



UNIVERSITÀ DEGLI STUDI DI FIRENZE

SCUOLA DI INGEGNERIA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

DIPARTIMENTO DI INGEGNERIA CIVILE E AMBIENTALE

Tesi di Laurea Magistrale in Ingegneria Informatica

**Sviluppo di un sistema basato su reinforcement
learning per la localizzazione di sorgenti inquinanti
in ambiente marino**

Candidato

Mattia Manneschi

Relatori

Prof. Giorgio Battistelli

Prof. Nicola Forti

Prof.ssa Irene Simonetti

Anno Accademico 2025/2026

Alla mia famiglia

Indice

1	Introduzione	1
1.1	Contesto e motivazione	1
1.2	Approcci al source seeking	2
1.3	Reinforcement Learning e scelta dell'algoritmo	2
1.4	Il progetto HYDRAS	4
1.5	Obiettivo e struttura della tesi	4
2	Il dataset	6
2.1	Dati utilizzati	6
2.1.1	Modello idrodinamico MIKE21	6
2.1.2	Forzanti e condizioni al contorno	7
2.1.3	Scenari idrodinamici e di vento	8
2.1.4	Sorgenti e formato dei dati	10
2.1.5	Suddivisione del dataset e augmentation temporale	11
2.2	Caricamento e gestione dei dati	12
2.2.1	Sincronizzazione dei dati	13
3	Composizione dell'ambiente	16
3.1	L'ambiente software	16
3.1.1	Architettura master-slave	17
3.1.2	Spazio delle azioni e dinamica dell'episodio	18
3.1.3	Action masking	19
3.1.4	Inizializzazione dell'agente	19
3.2	Il segnale di reward	24

4	Metodo del gradiente: Field Climbing Method	26
4.1	Introduzione	26
4.2	FCM puro	27
4.2.1	Stima del gradiente per minimi quadrati	27
4.2.2	Selezione dell'azione	28
4.2.3	Analisi del raggio di campionamento	28
4.3	FCM con passo adattivo (Adam)	31
4.3.1	Interpretazione del learning rate	33
4.3.2	Analisi del learning rate e risultati	33
5	Apprendimento per rinforzo: PPO	39
5.1	Motivazione	39
5.2	Introduzione a PPO	39
5.2.1	MaskablePPO	41
5.3	Spazio di osservazione	41
5.4	Addestramento	42
5.5	Risultati	44
5.5.1	Analisi sul raggio di campionamento crescente	45
5.5.2	Analisi sulla funzione di reward	46
5.5.3	Catena a velocità variabile	51
5.5.4	Formazione a singola vs. doppia corona	55
5.5.5	Test di modulazione: raggio di successo a 10 m	60
5.5.6	Heatmap di occupazione dell'agente	62
5.5.7	Analisi delle spawn map	64
5.5.8	Confronto con FCM puro e adattivo	68
5.5.9	Estensione all'intero range di velocità	70
6	Explainability della politica appresa	74
6.1	Motivazione	74
6.2	Metodi post-hoc: rassegna e criterio di scelta	75
6.2.1	Due accortezze tecniche	77
6.3	I gruppi semantici dell'osservazione	77

6.4	Ablation causale di gruppo	78
6.5	Valori di Shapley esatti	81
6.6	Analisi comportamentale	82
6.7	Il valore della formazione: ablazione per riaddestramento	84
6.8	Sintesi: che cosa la rete guarda, che cosa le serve	89
7	Conclusioni	91
7.1	Risultati principali	91
7.2	Sviluppi futuri	94
	Bibliografia	97

Capitolo 1

Introduzione

1.1 Contesto e motivazione

L'inquinamento delle acque costiere rappresenta una delle principali minacce ambientali per gli ecosistemi marini e per le attività economiche legate al mare. Sversamenti di idrocarburi, scarichi industriali e rilasci accidentali di sostanze chimiche generano *plume* di contaminante che si disperdono nell'ambiente seguendo la dinamica delle correnti e del vento. La localizzazione rapida della sorgente di un'emissione è cruciale per contenere il danno ambientale, garantire la sicurezza portuale e pianificare gli interventi di bonifica.

Il problema di localizzare una sorgente emittente sconosciuta a partire da osservazioni locali e sparse del campo che essa genera è noto in letteratura come *source seeking* [1]. L'impiego di sistemi robotici autonomi per questo compito è sempre più attraente grazie alla loro rapida dispiegabilità e alla possibilità di eliminare l'esposizione diretta degli operatori umani a sostanze pericolose. In ambito marino, il tracciamento di *plume* chimici è stato dimostrato tramite veicoli autonomi sottomarini (*Autonomous Underwater Vehicles*, AUV) e di superficie (*Autonomous Surface Vessels*, ASV) [2], anche in formazioni coordinate.

La natura del contaminante influenza fortemente la strategia di ricerca. Gli *sversamenti di idrocarburi* tendono a rimanere in superficie e sono rilevabili da remoto in mare aperto, ma non nelle aree portuali; gli *scarichi fognari*, ricchi di nutrienti e patogeni, hanno galleggiabilità neutra o negativa; i *contaminanti chimici* sono spesso invisibili, a galleggiabilità variabile e potenzialmente tossici. In tutti i casi, in ambiente costiero

e portuale la dispersione è governata da una dinamica complessa — correnti di marea, circolazione indotta dal vento, interazioni onda-corrente, apporti di acqua dolce, miscelamento turbolento e stratificazione — descritta matematicamente da equazioni alle derivate parziali. Il telerilevamento satellitare, incluso il radar ad apertura sintetica (SAR), offre risoluzione spaziale e temporale limitata ed è spesso compromesso da occlusioni atmosferiche, risultando inadeguato nelle zone costiere confinate. Ciò rende indispensabili soluzioni robotiche *in-situ* e adattive.

1.2 Approcci al source seeking

In letteratura il problema del source seeking è stato affrontato secondo diversi paradigmi di controllo. I metodi di **field climbing** (gradient ascent) *model-free* assumono che l'intensità del campo cresca avvicinandosi alla sorgente e guidano l'agente tramite la stima locale del gradiente: sono computazionalmente leggeri e non richiedono un modello dell'ambiente, ma soffrono di convergenza lenta, vulnerabilità ai massimi locali e necessità di formazioni dense per stimare accuratamente il gradiente. I metodi di *extremum seeking control* (ESC) stimano il gradiente indirettamente tramite perturbazioni oscillatorie. Gli approcci *model-based* di *source term estimation* (STE) formulano la localizzazione come un problema inverso, stimando i parametri della sorgente da misure rumorose tramite filtri di Kalman o particellari, ma richiedono modelli ambientali accurati e risorse di calcolo a bordo significative. Esistono inoltre approcci *information-theoretic*, ispirati a comportamenti biologici, e basati su *apprendimento*.

Dati i limiti dei metodi gradient-based e model-based, cresce l'interesse verso il **Reinforcement Learning** (RL) come alternativa flessibile e *model-free*, capace di apprendere strategie di navigazione robuste direttamente dall'interazione con ambienti complessi. È in questa direzione che si colloca il presente lavoro.

1.3 Reinforcement Learning e scelta dell'algoritmo

Il Reinforcement Learning formalizza il problema come un processo decisionale di Markov, in cui un agente sceglie azioni per massimizzare un ritorno cumulativo atteso, apprendendo per tentativi dall'interazione con l'ambiente [3]. Gli algoritmi si raggruppano

in poche famiglie. I metodi **value-based** — *Q-learning* e la sua versione con reti profonde, DQN [4] — apprendono una funzione azione-valore $Q(s, a)$ e ne derivano la politica per massimizzazione *greedy*: sono *off-policy* ed efficienti nel riuso dei dati (*replay buffer*), ma nativamente limitati ad azioni discrete e talvolta instabili. I metodi **policy-gradient** (REINFORCE, A2C/A3C) ottimizzano *direttamente* i parametri della politica lungo il gradiente del ritorno atteso, gestendo naturalmente politiche stocastiche. Gli approcci **actor-critic** combinano i due — una rete *attore* produce la politica, una rete *critico* ne stima il valore per ridurre la varianza del gradiente — e includono metodi off-policy per il controllo continuo come DDPG, TD3 e SAC [5]. Una linea distinta è infine quella dei metodi a **regione di fiducia**, che vincolano l'ampiezza di ogni aggiornamento della politica per garantirne la stabilità: TRPO [6] vi riesce risolvendo un problema di ottimizzazione vincolata, mentre **PPO** [7] ne è la semplificazione pratica, che ottiene lo stesso effetto con un semplice *clipping* del rapporto di probabilità.

In questa tesi si è scelto **Proximal Policy Optimization** (PPO) per un insieme di ragioni aderenti alla natura del problema. Anzitutto l'ambiente è un *simulatore*: i campioni sono abbondanti e a basso costo, condizione in cui la natura *on-policy* di PPO — di norma uno svantaggio di efficienza campionaria — non penalizza, mentre se ne guadagna la stabilità. In secondo luogo lo spazio d'azione è *discreto* (le otto direzioni, poi estese a 8×5 livelli di velocità; Capitolo 5): ciò esclude i metodi pensati per il controllo continuo (DDPG, TD3, SAC) e si sposa naturalmente con una politica categorica. Il vincolo di non uscire dal dominio o incagliarsi sulla costa è imposto tramite *action masking*, per cui esiste un'estensione diretta e consolidata di PPO — MaskablePPO [8]. Rispetto a TRPO, PPO ottiene una stabilità analoga con un'implementazione molto più semplice e pochi iperparametri; rispetto a DQN evita le note instabilità dell'off-policy con *replay buffer* in un ambiente per giunta *non stazionario* (il campo di concentrazione evolve durante l'episodio). PPO è inoltre lo standard de facto per il controllo tramite RL, con implementazioni affidabili e riproducibili [9], e la sua estensione multi-agente (MAPPO [10]) offre un percorso naturale verso la squadra cooperativa delineata fra gli sviluppi futuri. La descrizione dettagliata dell'algoritmo — l'obiettivo *clipped*, la stima del vantaggio e la nostra implementazione — è rimandata al Capitolo 5.

1.4 Il progetto HYDRAS

Questa tesi si inserisce nel progetto **HYDRAS** (*HYdrodynamic-aware Distributed Robots for mArine Source-seeking*), che ha come obiettivo lo sviluppo di un *framework* di nuova generazione per il source seeking autonomo in ambienti costieri e portuali complessi. L'elemento distintivo di HYDRAS è l'integrazione di **modelli idrodinamici ad alta risoluzione** site-specific, che simulano il trasporto dell'inquinante in condizioni di forcing realistiche, con strategie di controllo robotico distribuito. Tali simulazioni svolgono un duplice ruolo: generare ambienti fisicamente realistici per la valutazione delle strategie di ricerca e fornire terreni di addestramento per algoritmi di controllo basati su apprendimento.

Il dominio di studio è la baia di Cecina (costa toscana), modellata tramite simulazioni idrodinamiche **MIKE21** accoppiate ai dati meteo-oceanografici dei servizi **CMEMS** (*Copernicus Marine Environment Monitoring Service*) ed **ERA5**, descritte in dettaglio nel Capitolo 2.

Nella sua visione complessiva, HYDRAS adotta un'architettura *multi-agente*: una rete di robot mobili coordinati, ciascuno dotato di sensori di concentrazione, che collaborano per localizzare la sorgente con maggiore rapidità, copertura spaziale e robustezza rispetto a un sistema a singolo agente. In questa tesi si realizza una specifica istanza di tale architettura, basata su una **squadra di agenti** in formazione *master-slave*: un agente *master* si occupa della navigazione e della decisione di movimento, mentre otto agenti *slave* mantengono una formazione rigida attorno ad esso e hanno l'unico compito di campionare la concentrazione del contaminante nelle rispettive posizioni e riportarla al master. Le otto misurazioni degli slave, disposte lungo le direzioni cardinali e diagonali, costituiscono la base informativa con cui il master stima la direzione verso la sorgente.

1.5 Obiettivo e struttura della tesi

L'obiettivo della tesi è progettare, addestrare e valutare la politica di navigazione del master, confrontando due famiglie di approcci sul medesimo ambiente di simulazione: un **metodo del gradiente** (Field Climbing Method), che stima localmente il gradiente del campo di concentrazione a partire dalle misurazioni degli slave e si muove nella direzione

di salita più ripida senza alcuna fase di addestramento, adottato come *baseline* algoritmica; e un approccio di **Reinforcement Learning** basato sull'algoritmo *Proximal Policy Optimization* (PPO), in cui una rete neurale apprende una politica di navigazione ottimale tramite l'interazione ripetuta con l'ambiente simulato, che costituisce l'approccio proposto in questo lavoro. Il metodo del gradiente è puramente reattivo e privo di memoria, mentre l'approccio RL mira a superarne i limiti strutturali apprendendo strategie che sfruttano la storia dell'episodio e le informazioni contestuali sull'ambiente; il confronto, condotto a parità di condizioni su sorgenti mai viste in addestramento, quantifica il contributo dell'apprendimento rispetto all'approccio gradient-based classico. Coerentemente con questo obiettivo, la tesi è strutturata come segue: il Capitolo 2 descrive i dati utilizzati (simulazioni idrodinamiche MIKE21 della baia di Cecina, dati meteo-oceanografici di vento e corrente, sorgenti e scenari) e il modo in cui vengono caricati, sincronizzati e gestiti via software; il Capitolo 3 illustra la composizione dell'ambiente di simulazione condiviso dai due approcci — architettura master-slave, spazio delle azioni, action masking, inizializzazione dell'agente e segnale di reward; il Capitolo 4 presenta il metodo del gradiente, dalla stima del gradiente per minimi quadrati alla sua versione adattiva basata sull'ottimizzatore Adam, con la relativa analisi sperimentale e i risultati; il Capitolo 5 introduce l'algoritmo PPO, descrive la nostra implementazione (spazio di osservazione, funzione di reward, addestramento) e ne riporta i risultati, comprensivi della catena a velocità variabile, del confronto fra formazione a singola e doppia corona, dell'analisi delle *spawn map* e del confronto diretto con il metodo del gradiente a parità di velocità; il Capitolo 6 indaga poi la politica appresa dall'interno, chiedendosi su quali grandezze dell'osservazione si regga — quali la rete effettivamente usi e quali servano al compito — e quale strategia di navigazione abbia sviluppato; infine il Capitolo 7 riepiloga i risultati principali e delinea le possibili direzioni di sviluppo futuro.

Capitolo 2

Il dataset

Questo capitolo descrive i **dati** su cui si fonda l'intero lavoro. Si parte dai dati grezzi — le simulazioni idrodinamiche del campo di concentrazione e i dati meteo-oceanografici di vento e corrente (Sezione 2.1) — per poi illustrare come questi vengono caricati, sincronizzati e gestiti via software (Sezione 2.2). Su questi dati viene costruito l'ambiente di simulazione vero e proprio, oggetto del Capitolo 3 e condiviso dai due approcci confrontati nella tesi: ciò che cambia tra i due è unicamente la politica che governa l'agente, non i dati né l'ambiente.

2.1 Dati utilizzati

2.1.1 Modello idrodinamico MIKE21

Il campo di concentrazione del contaminante è ottenuto da simulazioni idrodinamiche realizzate con **MIKE21 Flow Model FM** (*Danish Hydraulic Institute, DHI*) [11], un codice ai volumi finiti basato sulla formulazione bidimensionale delle equazioni di Navier–Stokes per flussi incomprimibili che, sotto le ipotesi di Boussinesq e di distribuzione idrostatica della pressione, si riduce alle equazioni delle acque basse (*Shallow Water Equations*). Il codice include la simulazione dei processi di advezione e dispersione/diffusione di un inquinante non reattivo, trattato con approccio euleriano: il trasporto è descritto a partire da sorgenti di massa caratterizzate da una portata e da una concentrazione assegnate.

La discretizzazione spaziale del dominio è realizzata mediante una **griglia a elementi triangolari di dimensione variabile**, che consente un'elevata risoluzione nelle aree di maggiore interesse mantenendo una discretizzazione più grossolana altrove, al fine di contenere i tempi di calcolo. Il dominio di calcolo comprende l'area circostante il porto di Cecina (Figura 2.1), con un'estensione di circa 16×14 km e una dimensione caratteristica della griglia variabile fra circa 10 m in prossimità del porto (zona di interesse) e circa 150 m nelle aree più lontane.

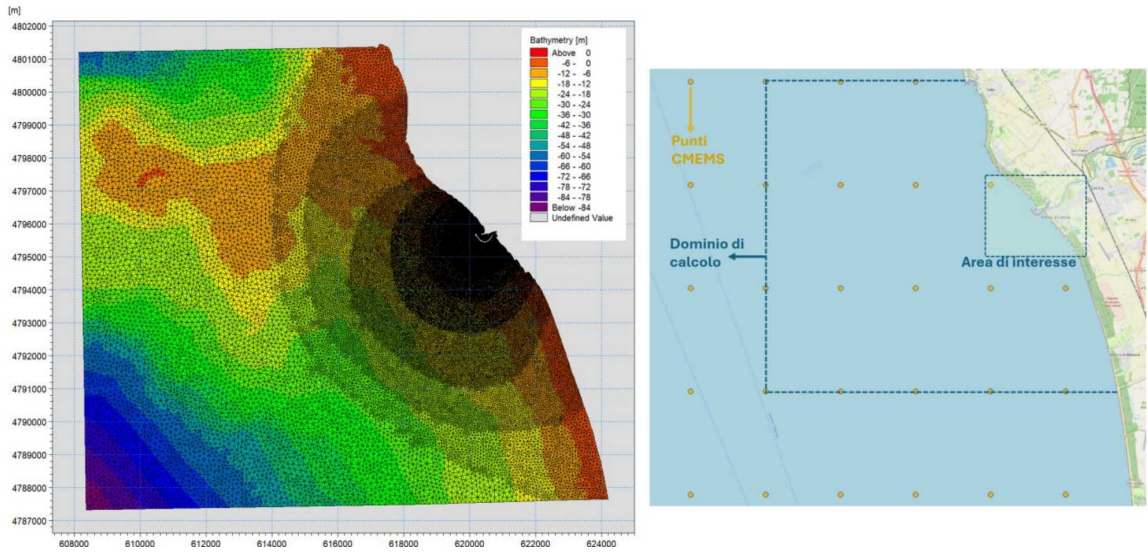


Figura 2.1: Dominio di calcolo del modello idrodinamico. A sinistra: batimetria e mesh a elementi triangolari di dimensione variabile (più fitta nella zona di interesse). A destra: estensione del dominio ($\sim 16 \times 14$ km), area di interesse (riquadro) da cui è estratto il dataset e nodi CMEMS lungo i bordi usati per le condizioni al contorno. Baia di Cecina (costa toscana).

2.1.2 Forzanti e condizioni al contorno

Il modello locale realizza un *downscaling idrodinamico* del bacino del Mediterraneo, incrementando la risoluzione spaziale da circa 4,5 km del modello a grande scala fino a circa 10 m nell'area di interesse. Le condizioni al contorno di livello della superficie libera e di velocità della corrente sono derivate dal prodotto CMEMS [12] `MEDSEA_ANALYSIS_FORECAST_PHY_006_013`, che fornisce dati orari delle componenti zonale e meridionale della velocità di corrente e del livello marino. Per ciascun nodo di calcolo ai bordi del dominio sono imposte condizioni al contorno di tipo *Flather* [13], mediante l'imposizione dei valori orari delle componenti (u, v) e dell'elevazione della superficie libera, interpolati linearmente dai nodi CMEMS disponibili lungo i bordi.

La forzante di vento è imposta come variabile nel tempo e costante sul dominio, ed è estratta dal dataset di *reanalysis* **ERA5** [14] dell'ECMWF (componenti del vento a 10 m, u_{10} , v_{10} , a risoluzione oraria). In questa fase non vengono considerati alcuni contributi al trasporto assunti come secondari: le correnti indotte dal moto ondoso (deriva litoranea, correnti di ritorno) e gli apporti di acqua dolce (scarichi fluviali, runoff superficiale).

2.1.3 Scenari idrodinamici e di vento

Le serie temporali CMEMS ai bordi del dominio (2024–2025) sono state classificate in finestre di 48 ore tramite descrittori sintetici (intensità media della corrente, persistenza direzionale, variabilità del livello marino) e raggruppate con un algoritmo di *clustering* k-means in 4 classi idrodinamiche di riferimento, che rappresentano regimi distinti in termini di intensità della corrente, persistenza direzionale e variabilità del livello: dalla Classe 1 (regime moderato con elevata variabilità del livello) alla Classe 4 (regime energetico e altamente persistente). Le simulazioni adottate in questa fase si riferiscono alla sola **Classe 2** (regime moderato e persistente, con correnti di intensità intermedia, direzione stabile e bassa variabilità del livello, ovvero condizioni relativamente stazionarie e ben organizzate), rappresentata dalla finestra del 19–20 maggio 2024.

Analogamente, l'analisi statistica delle serie di vento ERA5 (intensità, direzione, persistenza; Figura 2.2) ha portato a definire **4 classi di vento** rappresentative, sovrapposte alla condizione idrodinamica di riferimento:

- **V0** — *condizioni di calma / vento debole*. Vento di bassa intensità (circa 1,5–2 m/s), senza una direzione dominante significativa; rappresenta condizioni con forcing eolico trascurabile.
- **V1** — *vento persistente da NO*. Direzione prevalente compresa tra 310° e 330°, con velocità fino a 8–10 m/s; rappresenta uno dei regimi dominanti lungo costa.
- **V2** — *vento persistente da SO*. Direzione prevalente tra 230° e 250°, con velocità fino a 8–10 m/s; rappresenta il regime opposto rispetto al precedente, con effetti rilevanti sull'inversione del trasporto lungo costa.

- **V3** — *vento da settore orientale*. Direzione prevalente tra 110° e 130° , con velocità fino a 6–8 m/s; rappresenta un regime meno frequente ma significativo, associato a condizioni di trasporto trasversale o secondario.

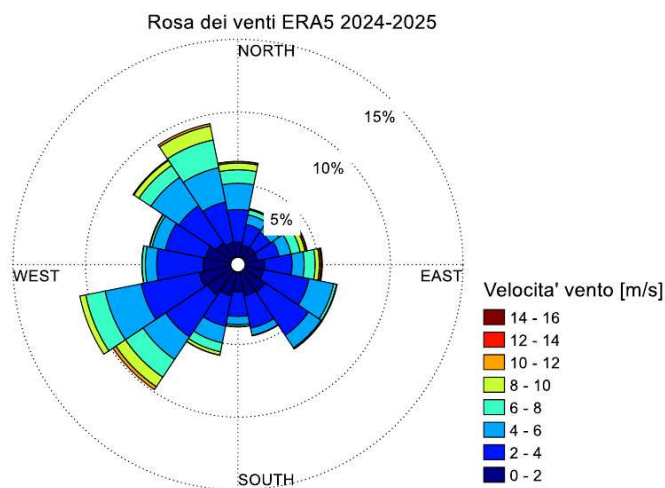


Figura 2.2: Rosa dei venti dell'area (dati ERA5 2024–2025): distribuzione di frequenza per direzione e intensità. I settori dominanti da NO e da SO — ben visibili — motivano le classi di vento V1 e V2; le classi V0–V3 sintetizzano in modo compatto questa climatologia eolica.

Per le classi V1–V3 il vento è modellato con un profilo temporale tipico: 8 ore di calma iniziale, 6 ore di aumento lineare fino all'intensità massima, 12 ore di intensità massima e 6 ore di diminuzione lineare. Ogni scenario genera plume con morfologie marcatamente diverse (Figura 2.3): da configurazioni compatte e direzionate a configurazioni diffuse e frammentate.

Morfologia del plume per scenario di vento — sorgente SRC114, chunk Q1/2

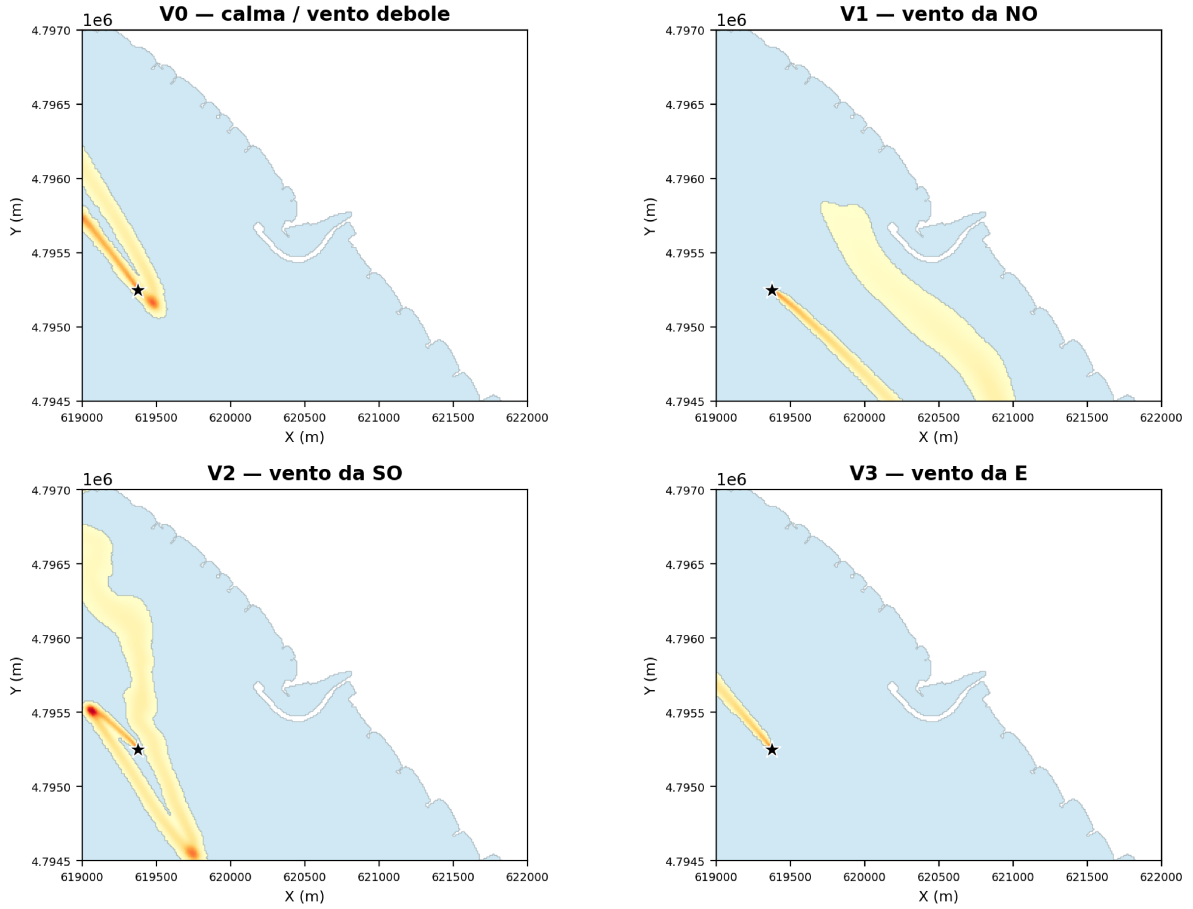


Figura 2.3: Campo di concentrazione del contaminante (plume) per la stessa sorgente (★) sotto i quattro scenari di vento, allo stesso istante (chunk Q1/2). In V0 (calma) il plume resta compatto attorno alla sorgente; con vento persistente (V1 da NO, V2 da SO) viene avvevato lungo costa e si allunga/diffonde; in V3 (da E) è sospinto verso il largo. Mare in azzurro, terraferma in bianco, concentrazione in scala gialla-rossa.

2.1.4 Sorgenti e formato dei dati

Nell'area di interesse sono distribuite **132 sorgenti** equispaziate di circa 200 m, ciascuna caratterizzata da una portata costante di 50 l/s e da una concentrazione di 1000 g/m³ dell'inquinante target. Ogni sorgente è simulata sotto i 4 scenari di vento, per un totale di $132 \times 4 = 528$ simulazioni. I campi di concentrazione risultanti sono estratti in formato **NetCDF** su una griglia regolare con passo di 10 m e intervallo temporale di 2 minuti, nell'area compresa fra le coordinate $x \in [619000, 622000]$ e $y \in [4794500, 4797000]$ m (UTM32N), corrispondente a un riquadro di $3 \times 2,5$ km (300×250 celle). I file seguono la convenzione di nome CL02_<vento>_SRC<num>_Conc_10mGrid.nc; i corrispondenti

campi di corrente (u, v), comuni a tutte le sorgenti di uno stesso scenario, seguono la convenzione CL02_<vento>_U_V_10mGrid.nc (i campi di corrente sono comuni a tutte le sorgenti di uno stesso scenario, in quanto la presenza della sorgente altera il campo di moto solo localmente). Sorgente e scenario di ciascun file sono ricavati dal nome tramite una semplice espressione regolare:

```
m = re.match(r"CL02_(V\d)_(SRC\d+)_Conc", filename)
version, source_id = m.group(1), m.group(2)    # es. ("V2", "SRC042")
```

2.1.5 Suddivisione del dataset e augmentation temporale

La suddivisione del dataset e l'augmentation temporale descritte di seguito sono **comuni a entrambi gli approcci** confrontati nella tesi (Capitoli 4 e 5), e fanno quindi parte dell'impostazione condivisa dell'ambiente di simulazione.

Per garantire una valutazione su sorgenti mai osservate durante l'addestramento del modello RL, le 132 sorgenti sono suddivise in un insieme di **training** (SRC001–SRC106, 80%, 106 sorgenti) e in un insieme di **inferenza** held-out (SRC107–SRC132, 20%, 26 sorgenti), riservato alla sola valutazione. Le stesse 26 sorgenti held-out sono usate per valutare sia il metodo del gradiente sia il modello RL, così da garantire un confronto equo tra i due approcci.

Poiché il campo evolve nel tempo, ogni simulazione viene inoltre suddivisa in **3 chunk temporali** che corrispondono a diversi stadi della dispersione del plume e fungono da condizioni iniziali distinte: **Q1/4** (inizio della propagazione, plume compatto), **Q1/2** (stadio intermedio, plume parzialmente disperso) e **Q3/4** (stadio avanzato, plume pesantemente disperso). Questa *augmentation temporale* consente di inizializzare l'agente in tre fasi diverse della dispersione, aumentando la variabilità delle condizioni. La combinazione di scenari di vento e chunk temporali definisce le *configurazioni* (v, q) su cui sono articolati i risultati nei capitoli seguenti. In codice, la suddivisione delle sorgenti e la mappatura chunk $\rightarrow$ frame iniziale si riducono a:

```
sources = sorted(discover_sources())          # SRC001 ...
SRC132
train = [s for s in sources if int(s[3:]) <= 106] # 80%
test  = [s for s in sources if int(s[3:]) > 106]  # 20% held-out
```

```

nt = field.n_timesteps                                # numero di frame
spawn_frame = {0: nt//4, 1: nt//2, 2: (nt*3)//4}[chunk_id]  # Q1/4,
    Q1/2, Q3/4

```

2.2 Caricamento e gestione dei dati

Il caricamento dei dati grezzi e la loro trasformazione in strutture interrogabili dall'ambiente sono gestiti dal modulo software `data_loader`, organizzato attorno a tre componenti principali.

Campo di concentrazione. Il componente `NetCDFLoader` legge un file NetCDF e ne costruisce un oggetto `ConcentrationField`, che incapsula i dati di concentrazione $[t, y, x]$ e fornisce query spazio-temporali continue. Data una posizione (x, y) e un istante t , il campo restituisce la concentrazione tramite interpolazione (bilineare nello spazio, lineare tra i due frame temporali adiacenti), così che l'agente non sia vincolato ai soli centri-cella della griglia. All'inizializzazione, dalla maschera di terra viene inoltre precalcolata — tramite *Euclidean Distance Transform* (EDT) — una mappa delle distanze dalla costa, usata sia per i vincoli di spawn sia per la penalità di prossimità alla terra. In pratica, caricamento e interrogazione del campo si riducono a:

```

field = NetCDFLoader(data_dir).load("CL02_V2_SRC042_Conc_10mGrid.nc")
field.set_time(t_idx)                                # seleziona il frame temporale
    corrente
c = field.get_concentration(x, y)                      # concentrazione interpolata
    nel punto (x, y)

```

Campi di vento e corrente. In modo analogo, `WindData` e `CurrentData` incapsulano rispettivamente la serie temporale di vento e il campo vettoriale di corrente, esponendo le componenti (u, v) in funzione del tempo (e della posizione, per la corrente). La sincronizzazione temporale con il campo di concentrazione è mantenuta in minuti simulati.

Gestione centralizzata. Il componente `DataManager` coordina l'accesso ai dati: alla creazione *scopre automaticamente* le sorgenti disponibili dai file presenti, e su richiesta fornisce all'ambiente il campo di concentrazione di una data coppia (sorgente, scenario) insieme ai relativi vento e corrente. Per evitare di diventare un collo di bottiglia di I/O durante l'addestramento — che richiede milioni di passi e quindi continue riletture — il `DataManager` mantiene in memoria una *cache* dei quattro scenari di vento e corrente, eliminando gli accessi al disco dopo il primo caricamento. Una singola chiamata restituisce il campo di concentrazione e le forzanti coerenti:

```
# Campo di una coppia (sorgente, scenario) + forzanti della STESSA
  versione
field, run_id = data_manager.get_random_field_for_source_version("
    SRC042", "V2")
wind      = data_manager.get_wind_data_for_run(run_id, wind_mapping)
           # da cache
current = data_manager.get_current_data_for_run(run_id,
    current_mapping) # da cache
```

2.2.1 Sincronizzazione dei dati

Affinché l'agente percepisca un ambiente fisicamente coerente, i tre flussi di dati — concentrazione, vento e corrente — devono essere allineati. Il `DataManager` garantisce due livelli di sincronizzazione.

Sincronizzazione di versione. La versione dello scenario di vento (V0–V3) è codificata nel nome di ogni file (es. CL02_V2_SRC042_Conc_10mGrid.nc). Al momento di fornire un campo, il `DataManager` estrae la versione dal nome del file di concentrazione e seleziona i corrispondenti campi di vento e corrente della *stessa* versione. In questo modo è impossibile un accoppiamento errato tra concentrazione e forzanti: il plume è sempre associato al vento e alla corrente che lo hanno effettivamente generato.

Sincronizzazione temporale. I tre flussi hanno cadenze di campionamento native diverse: il campo di concentrazione MIKE21 è prodotto con un *frame* ogni $\Delta t_c = 2$ min, mentre vento e corrente seguono le rispettive serie, ciascuna con il proprio passo (Δt_w

per il vento, Δt_u per la corrente). Se questi flussi non fossero allineati, l'agente percepirebbe un plume che avanza a una velocità incoerente con il vento e la corrente che lo trasportano. L'ambiente li riconduce perciò a un unico **asse temporale assoluto**, espresso in *minuti simulati*.

Istante assoluto. Ad ogni passo, dallo step corrente si ricava l'istante assoluto dell'episodio come somma dell'offset del chunk di partenza e del tempo trascorso:

$$t_{\text{abs}}(k) = \underbrace{f_0 \cdot \Delta t_c}_{\text{offset del chunk}} + \underbrace{k \cdot \Delta t / 60}_{\text{tempo trascorso}} \quad [\text{min}], \quad (2.1)$$

dove k è il numero di step compiuti, $\Delta t = 10$ s il passo dell'agente e f_0 il *frame* di spawn del chunk (Sezione 2.1.5). È essenziale che l'offset del chunk compaia in t_{abs} : così, quando l'episodio parte da Q3/4, non solo il plume ma anche il vento e la corrente sono presi al punto *corrispondente* del loro profilo temporale — l'agente incontra lo stato idrodinamico effettivamente contemporaneo a quello stadio di dispersione, non quello dell'inizio simulazione.

Query per flusso e interpolazione. L'istante t_{abs} viene poi convertito nell'indice, in generale *frazionario*, proprio di ciascun flusso; il valore restituito è l'**interpolazione lineare** tra i due campioni adiacenti:

- **concentrazione:** l'indice di frame è $f = f_0 + (t_{\text{abs}} - f_0 \Delta t_c) / \Delta t_c$; detta α la sua parte frazionaria, il campo restituito è la miscela $(1 - \alpha) C_{t_0} + \alpha C_{t_1}$ tra i frame adiacenti $t_0 = \lfloor f \rfloor$ e $t_1 = t_0 + 1$ — interpolazione temporale che si somma a quella bilineare nello spazio (Sezione 2.2);
- **vento e corrente:** l'indice è $t_{\text{abs}} / \Delta t_{w,u}$, con analogia miscela lineare tra i campioni delle rispettive serie.

Cadenza. Poiché $\Delta t_c / \Delta t = 2 \text{ min} / 10 \text{ s} = 12$, il campo di concentrazione avanza di un frame **ogni 12 step** dell'agente; nei passi intermedi l'agente non vede un fotogramma “congelato” ma una versione *interpolata* del campo. Ad esempio, dopo 11 minuti dallo spawn ($k = 66$) l'indice di frame vale $f_0 + 5,5$: il campo restituito è la media al 50% fra il frame f_0+5 e il frame f_0+6 (Figura 2.4).

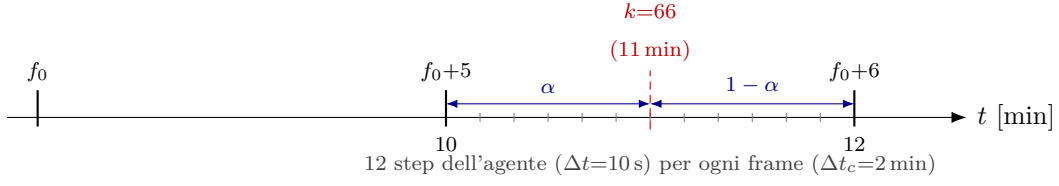


Figura 2.4: Sincronizzazione temporale della concentrazione. Tra due frame consecutivi (f_0+5 a 10 min, f_0+6 a 12 min) l'agente compie 12 step; allo step $k=66$ (11 min) il campo è la miscela lineare $(1 - \alpha)C_{f_0+5} + \alpha C_{f_0+6}$ con $\alpha = 0,5$. Vento e corrente sono interpolati in modo analogo sulle rispettive serie.

Gestione dei bordi. Un episodio dura al più 1080 step (= 180 min); partendo da un chunk avanzato, t_{abs} può oltrepassare la fine di una serie. In tal caso l'indice viene **saturato** all'ultimo campione disponibile (con un avviso una tantum per il vento), così che l'episodio resti sempre ben definito anche quando “sopravvive” ai dati. In codice, l'aggiornamento dei tre flussi ad ogni step si riduce a:

```
dt_c  = 2.0                                # min per frame di
    concentrazione
t_abs = f0 * dt_c + step * dt / 60.0        # istante assoluto
    in minuti simulati
field.set_time(f0 + (step * dt / 60.0) / dt_c) # frame float ->
    interp. temporale lineare
wind.set_time_from_minutes(t_abs)           # indice = t_abs /
    dt_wind  (clamp agli estremi)
current.set_time_from_minutes(t_abs)        # indice = t_abs /
    dt_curr  (clamp agli estremi)
```

Capitolo 3

Composizione dell'ambiente

A partire dai dati descritti nel Capitolo 2, questo capitolo illustra l'**ambiente di simulazione** vero e proprio: l'interfaccia software che trasforma i campi idrodinamici in un problema di controllo sequenziale, la struttura della squadra di agenti, lo spazio delle azioni, i vincoli di navigazione, la procedura di inizializzazione e il segnale di reward. Si tratta dell'infrastruttura **condivisa** dai due approcci confrontati nella tesi: sia il metodo del gradiente (Capitolo 4) sia l'agente PPO (Capitolo 5) operano su questo stesso ambiente, con la stessa dinamica e gli stessi vincoli. L'unica differenza risiede nel modulo che, ricevuta l'osservazione, sceglie l'azione: una regola analitica nel primo caso, una rete neurale addestrata nel secondo.

3.1 L'ambiente software

L'ambiente è implementato dalla classe `SourceSeekingEnv` secondo l'interfaccia standard `gymnasium.Env`. Esso riceve in ingresso un `ConcentrationField` (con i relativi `WindData` e `CurrentData`), oppure un `DataManager` da cui estrarre dinamicamente un campo casuale ad ogni episodio — modalità usata in addestramento per variare di continuo sorgente e scenario. Il dominio è la griglia regolare 300×250 celle a passo 10 m ($3 \times 2,5$ km) descritta nella Sezione 2.1.4.

L'oggetto ambiente viene quindi incapsulato in una catena di *wrapper*:

- un `ActionMasker` che applica l'action masking (Sezione 3.1.3), escludendo le azioni non ammissibili;

- un *vettorizzatore* (`DummyVecEnv`) che uniforma l'interfaccia a quella attesa dalla libreria di RL;
- per il solo addestramento RL, un `VecNormalize` che normalizza le osservazioni con media e varianza stimate online.

```
raw = SourceSeekingEnv(config, concentration_field=field,
                        wind_data=wind, current_data=current)
env = ActionMasker(raw, mask_fn)                # masking delle azioni
env = DummyVecEnv([lambda: env])                # vettorizzazione
env = VecNormalize(env, norm_reward=False)       # solo per il training
RL
```

3.1.1 Architettura master–slave

Il sistema è organizzato come una **squadra di agenti** in formazione *master–slave*. L'agente **master** è l'unico a decidere il movimento e a spostarsi attivamente sul dominio; attorno ad esso, otto agenti **slave** mantengono una formazione rigida (Figura 3.1), posizionandosi a distanza r_s (il *sensor range*) lungo le otto direzioni cardinali e diagonali $\hat{d}_i$ (N, S, E, O, NE, SE, NO, SO). Ciascuno slave campiona la concentrazione del contaminante nella propria posizione $\mathbf{p} + r_s \hat{d}_i$ e la riporta al master, senza alcuna autonomia decisionale propria: il loro unico compito è fornire le misurazioni e conservare la formazione rispetto al master mentre questo si muove.

Le otto misurazioni degli slave, insieme alla lettura del master nella posizione centrale, costituiscono l'informazione locale sul campo di concentrazione su cui si basano entrambi gli approcci. Nel metodo del gradiente tali letture vengono usate per stimare il gradiente locale; nell'approccio RL confluiscono nello spazio di osservazione della rete neurale che governa il master. Le posizioni degli slave e il relativo campionamento si esprimono come:

```
DIAG = 1 / np.sqrt(2)
DIRECTIONS = np.array([(0,1),(0,-1),(1,0),(-1,0),
                        0,
                        (DIAG,DIAG),(DIAG,-DIAG),
                        (-DIAG,DIAG),(-DIAG,-DIAG)])
                                #
                                diagonali
```

```
# ogni slave campiona a distanza  $r_s$  lungo la propria direzione
readings = [field.get_concentration(*(master_pos + r_s * d)) for d in
             DIRECTIONS]
```

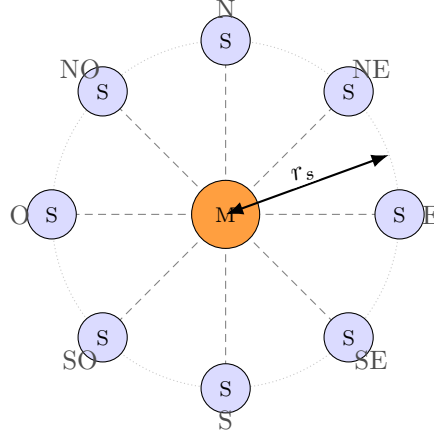


Figura 3.1: Formazione master-slave. L'agente **master** (M, arancione) occupa il centro; otto agenti **slave** (S, blu) sono disposti a distanza r_s (il *sensor range*) lungo le direzioni cardinali e diagonali. Ogni slave campiona la concentrazione nella propria posizione e la riporta al master; le stesse otto direzioni sono anche le azioni di movimento disponibili.

Questa unica corona di 8 sensori è la configurazione base (*singola corona*); il Capitolo 5 valuta anche una variante a **doppia corona** che aggiunge una seconda formazione di 8 slave a $r_{s2} = 50$ m, per dare al master informazione sul plume a scala spaziale più larga.

3.1.2 Spazio delle azioni e dinamica dell'episodio

Il master si muove con uno spazio delle azioni discreto a **8 direzioni** (le stesse $\hat{d}_i$ della formazione), con velocità nominale di 1 m/s e passo temporale $\Delta t = 10$ s, spostandosi quindi di 10 m per azione (o ~ 7 m per componente nelle direzioni diagonali). Ogni episodio ha durata massima di 1080 step (= 3 h di tempo simulato) e termina con **successo** quando il master raggiunge la sorgente (distanza ≤ 50 m). Il campo di concentrazione evolve durante l'episodio: ogni alcuni step dell'agente il campo MIKE21 avanza di un frame temporale, mantenendo vento e correnti sincronizzati. La conversione azione $\rightarrow$ spostamento è immediata:

```
ACTION_MAP = {0:(0,1), 1:(0,-1), 2:(1,0), 3:(-1,0),           # N, S, E,
               4:(0,0), 5:(1,1), 6:(1,-1), 7:(-1,-1)}        # O
```

```

        4:(DIAG,DIAG), 5:(DIAG,-DIAG),
        6:(-DIAG,DIAG), 7:(-DIAG,-DIAG)}           # diagonali
dx, dy = ACTION_MAP[action]
step = max_velocity * dt                            # 1 m/s * 10 s = 10 m
master_pos += np.array([dx, dy]) * step

```

Questo spazio d'azione a passo fisso è il punto di partenza; il Capitolo 5 lo estende a **velocità variabile** rendendo la velocità stessa una scelta dell'agente (spazio Discrete($8 \times K$)), così che il master decida a ogni passo non solo la direzione ma anche l'entità dello spostamento.

3.1.3 Action masking

Per impedire le collisioni con la costa, l'ambiente calcola ad ogni passo una *maschera* delle azioni: le direzioni che porterebbero il master sulla terraferma o fuori dal dominio vengono escluse dall'insieme delle azioni ammissibili. Questo meccanismo, sfruttato sia dal metodo del gradiente sia dall'agente RL, garantisce che vengano percorse soltanto traiettorie valide senza dover penalizzare a posteriori le azioni non consentite.

```

def action_masks(self):
    masks = np.ones(8, dtype=bool)
    step = self.max_velocity * self.dt
    for a, (dx, dy) in ACTION_MAP.items():
        nx, ny = self.x + dx*step, self.y + dy*step
        if out_of_domain(nx, ny) or self.field.is_land(nx, ny):
            masks[a] = False           # azione invalida
    return masks

```

La maschera è costruita a partire dalla *land mask* del campo e dalla mappa delle distanze dalla costa precalcolata via *Euclidean Distance Transform* (EDT, Sezione 2.2): la stessa mappa fornisce anche la penalità di prossimità alla terra usata nel segnale di reward (Sezione 3.2).

3.1.4 Inizializzazione dell'agente

Il master viene inizializzato su una cella appartenente al plume (concentrazione $c \geq 0,5$), a distanza dalla costa ≥ 50 m e a una distanza dalla sorgente controllata. Nel corso del

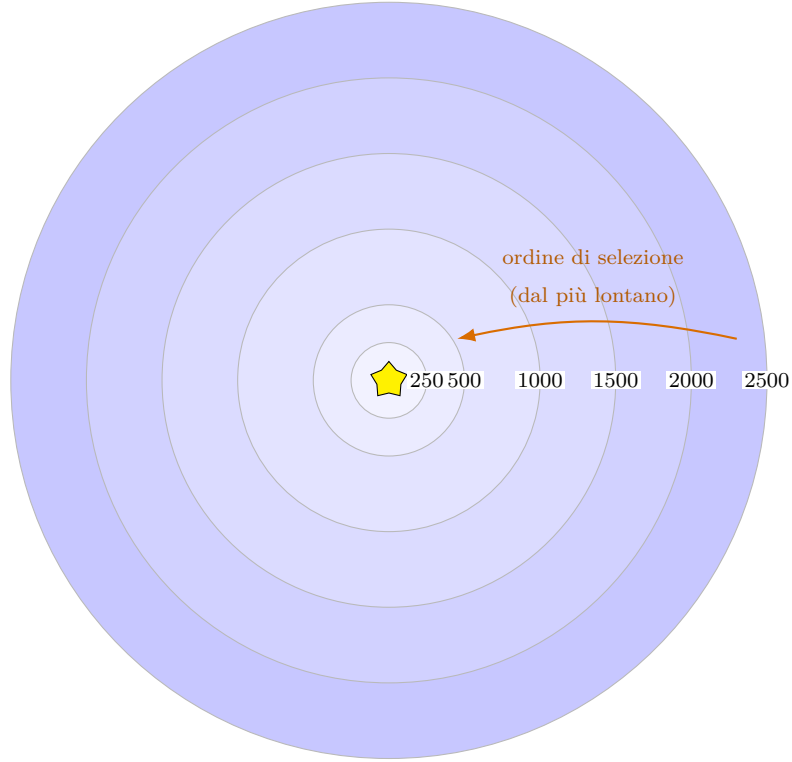


Figura 3.2: Cascata di anelli concentrici per l'inizializzazione. Il master è collocato in un anello attorno alla sorgente ($\star$), partendo dal più lontano (2000–2500 m) e scendendo verso i più vicini solo se nessuna cella dell'anello corrente soddisfa i vincoli (concentrazione $\geq 0,5$, distanza dalla terra ≥ 50 m). Le distanze sono in metri dalla sorgente.

Pro. Lo schema è semplice e deterministico nell'ordinamento, e privilegia in modo naturale gli spawn lontani: con plume esteso l'agente parte quasi sempre dal bordo esterno, affrontando il compito nella sua versione più difficile e informativa. **Contro.** Restituendo sempre il *primo* anello popolato, gli spawn si **addensano quasi tutti alla stessa distanza** (l'anello esterno del plume del momento): la distribuzione della distanza iniziale è di fatto degenerare e copre male l'intervallo intermedio. Le soglie sono inoltre *fisse*, e non si adattano a plume più compatti o più diffusi del previsto.

Secondo algoritmo (dall'analisi a velocità variabile): disco adattivo $[d_{\min}, d_{\max}]$.

Per distribuire i punti di partenza su tutto l'intervallo utile, la versione definitiva costruisce un *disco di spawn* $[d_{\min}, d_{\max}]$ che si adatta allo scenario e campiona in modo **uniforme rispetto alla distanza** (Figura 3.3). Detto $\{d_i\}$ l'insieme delle distanze dalla sorgente delle celle di plume ammissibili ($c \geq 0,5$, distanza dalla terra ≥ 50 m):

- $d_{\max} = \min(3000 \text{ m}, \max_i d_i)$: il tetto di 3 km, abbassato all'estensione massima del plume quando questo non arriva così lontano;

- $d_{\min} = \max(300 \text{ m}, \text{perc}_{50}(\{d_i\}))$: la mediana delle distanze delle celle di plume, con un *floor* assoluto a 300 m. Così $d_{\min}$ **scala con la diffusione del plume**: plume molto esteso $\Rightarrow$ celle lontane $\Rightarrow$ mediana alta $\Rightarrow d_{\min}$ alta; plume compatto $\Rightarrow d_{\min}$ bassa (una soglia alta sarebbe irraggiungibile). Il floor evita gli spawn già dentro il raggio di successo, tranne quando l'intero plume è più piccolo del floor stesso, nel qual caso il disco si restringe alla sua estensione;
- se $d_{\max} - d_{\min} < 500 \text{ m}$ si abbassa $d_{\min}$ (fino al floor) per garantire un intervallo abbastanza ampio da non far ripartire tutti gli agenti dalla stessa distanza.

Si estrae quindi una distanza obiettivo $d^* \sim \mathcal{U}(d_{\min}, d_{\max})$ e si sceglie a caso una cella valida entro $\pm 50 \text{ m}$ da d^* (con piccolo jitter), ripetendo finché i vincoli di dominio, concentrazione e distanza dalla costa sono soddisfatti. Campionando in modo uniforme d^* , gli spawn si **spalmano su tutto** $[d_{\min}, d_{\max}]$ invece di concentrarsi sull'anello esterno.

```
d = dist_from_source(plume_cells)          # celle con c>=0.5 e land
      >=50 m
d_disk = d[d <= 3000]                      # tetto d_cap = 3 km
d_max = min(3000, d_disk.max())
d_min = max(300, percentile(d_disk, 50))   # mediana, floor 300 m
if d_max - d_min < 500:                    # ampiezza minima dell'
    intervallo
    d_min = max(300, d_max - 500)
target = uniform(d_min, d_max)             # UNIFORME in distanza
band = plume_cells[abs(d_disk - target) <= 50] # cella entro +-50
      m
return random.choice(band)                 # (con jitter +-5 m)
```

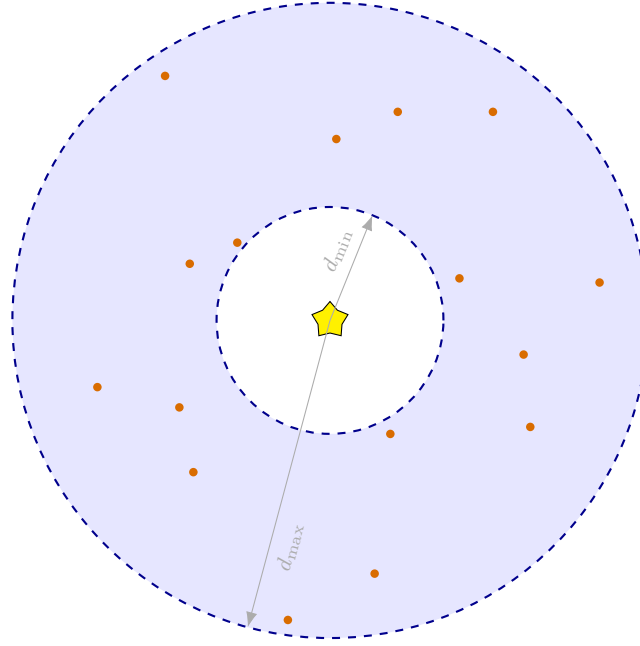


Figura 3.3: Disco di spawn adattivo $[d_{\min}, d_{\max}]$ (versione adottata). $d_{\max}$ è il tetto di 3 km ridotto all'estensione del plume; $d_{\min}$ è la mediana delle distanze delle celle di plume (*floor* 300 m) e scala con la diffusione. I punti di partenza (arancioni) sono estratti in modo *uniforme rispetto alla distanza* nell'anello, coprendo l'intero intervallo anziché addensarsi sul bordo esterno.

Pro. Il disco è *adattivo* (si ridimensiona con l'estensione del plume) e distribuisce gli spawn in modo uniforme su tutto $[d_{\min}, d_{\max}]$: la distanza iniziale copre l'intero intervallo, rendendo il confronto fra modelli più equo e statisticamente più solido. Il *floor* a 300 m e l'ampiezza minima mantengono comunque il compito non banale, senza inizializzazioni già prossime al bersaglio. **Contro.** La logica è più complessa (percentile, banda di tolleranza, fallback), e la distanza tipica di partenza non è un valore fisso ma dipende dalla mediana delle celle di plume del singolo scenario, quindi varia da caso a caso. Soprattutto, campionando in modo uniforme su $[d_{\min}, d_{\max}]$, l'agente può partire anche da **distanze brevi** (prossime a $d_{\min}$), mentre la cascata sceglieva sempre il punto valido più lontano: alcuni episodi risultano quindi mediamente più facili rispetto al primo algoritmo.

Perché il passaggio, e quale algoritmo usa cosa. Con la velocità fissa la cascata era adeguata: il compito era già difficile e la distanza di partenza, pur concentrata, risultava paragonabile fra episodi. Introducendo la **velocità variabile** (Sezione 5.5.3) il confronto avviene però *fra* modelli e *fra* velocità massime diverse: una distribuzione di

spawn degenerare avrebbe reso i confronti sensibili al particolare anello estratto più che alla reale capacità dell'agente. Il disco adattivo, uniforme in distanza e indipendente dalla forma istantanea del plume, offre una base di valutazione controllata e riproducibile. In pratica: i modelli a **velocità fissa** dei primi esperimenti usano la **cascata**; tutti i modelli e le analisi a **velocità variabile** — l'intero range di velocità (Sezione 5.5.3), i $v_{\max}$ intermedi, il confronto fra corona singola e doppia e il test a singolo agente senza formazione — usano il **disco adattivo**.

L'analisi delle *spawn map* (Sezione 5.5.7) verifica infine, per il disco, che l'esito di un episodio non dipenda dalla particolare posizione di partenza.

3.2 Il segnale di reward

Oltre a osservazione e dinamica, l'ambiente calcola ad ogni passo un **segnale di reward** scalare r_t . Esso è usato *unicamente* dall'agente RL, che ne massimizza il ritorno atteso in fase di addestramento (Capitolo 5); il metodo del gradiente lo ignora, poiché decide l'azione analiticamente dalle sole letture correnti. Nella sua forma finale, adottata come riferimento, il reward combina un piccolo insieme di segnali essenziali:

$$r_t = \underbrace{r_t^{\text{reach}}}_{+100 \text{ a successo}} + \underbrace{r_t^{\text{dist}}}_{\text{avvicinamento}} + \underbrace{r_t^{\text{grad}}}_{\pm 0,05} + \underbrace{r_t^{\text{time}}}_{-0,1/\text{step}} + \underbrace{r_t^{\text{land}}}_{\text{costa}} + \underbrace{r_t^{\text{bound}}}_{-10 \text{ fuori dominio}} \quad (3.1)$$

dove: $r_t^{\text{reach}} = +100$ (più un piccolo bonus per l'arrivo anticipato) premia il raggiungimento della sorgente; r_t^{dist} è proporzionale alla *riduzione* di distanza dalla sorgente nel passo (reward di distanza lineare); $r_t^{\text{grad}} = \pm 0,05$ premia il muoversi lungo il gradiente crescente di concentrazione; $r_t^{\text{time}} = -0,1$ per step incentiva a raggiungere la sorgente rapidamente; r_t^{land} penalizza la prossimità eccessiva alla costa (via la mappa EDT); $r_t^{\text{bound}} = -10$ penalizza l'uscita dal dominio. In codice, il reward si assembla ad ogni passo come somma di questi contributi:

```
def _compute_reward(self):
    r = 0.0
    if self._source_reached():
        # 1) raggiungimento
        (+ bonus di tempo)
        r += 100.0 + max(0, (N_max - k) / N_max * 50)
```

```
if self._out_of_bounds():                                # 2) uscita dal
    dominio
    r += -10.0
r += (prev_dist - curr_dist) * K_dist                    # 3) distanza (
    lineare)
r += 0.05 if conc > prev_conc else -0.05                 # 4) gradiente di
    concentrazione
r += -0.1                                                 # 5) penalita' di
    tempo (per step)
r += land_proximity_penalty(land_dist)                   # 6) prossimita'
    alla costa (EDT)
return r
```

La scelta di questa forma *minimale* non è a priori: è il risultato di uno studio di ablation (Sezione 5.5.2) che ha confrontato varianti più ricche — con reward di allineamento a vento e corrente, distanza quadratica, *zone multiplier*, *retreat asymmetry* e penalità di stagnazione — mostrando che i segnali aggiuntivi non migliorano, e in alcuni casi peggiorano, le prestazioni. Il gradiente di concentrazione, calcolato direttamente sul campo simulato, incorpora già implicitamente l'effetto di vento e corrente.

Capitolo 4

Metodo del gradiente: Field Climbing Method

4.1 Introduzione

Il **Field Climbing Method** (FCM) è l’approccio gradient-based classico al problema del source seeking, adottato in questa tesi come *baseline* algoritmica di confronto. L’idea è diretta: se l’agente conosce la direzione lungo cui la concentrazione del contaminante cresce più rapidamente, può muoversi in quella direzione e, ripetendo il procedimento ad ogni passo, risalire il plume fino alla sorgente. A differenza dell’approccio basato su Reinforcement Learning (Capitolo 5), FCM *non richiede una fase di addestramento*: è un algoritmo puramente reattivo che decide l’azione ad ogni istante basandosi esclusivamente sulle letture correnti dei sensori.

Il capitolo presenta due varianti del metodo, ciascuna con la relativa analisi sperimentale: il **FCM puro** a passo fisso (Sezione 4.2), con l’analisi sul raggio di campionamento, e il **FCM con passo adattivo** basato sull’ottimizzatore Adam (Sezione 4.3), con l’analisi sul learning rate.

Protocollo di valutazione. Tutte le analisi sperimentali di questo capitolo sono condotte sulle 26 sorgenti di test (SRC107–SRC132) — riservate alla sola valutazione e impiegate anche per il modello RL del Capitolo 5, così da garantire un confronto equo — su 4 scenari di vento (V0–V3) e 3 chunk temporali, ossia tre istanti della simulazione

usati come condizioni iniziali distinte (Q1/4, Q1/2, Q3/4), con 2 episodi per scenario: in totale 624 episodi per ogni configurazione di parametri.

4.2 FCM puro

4.2.1 Stima del gradiente per minimi quadrati

La squadra di robot legge la concentrazione del contaminante in **9 punti**: la posizione del master $\mathbf{p}$ (concentrazione centrale C_0) e gli 8 agenti slave che, in formazione attorno al master (Sezione 3.1.1), campionano $C(\mathbf{p} + \boldsymbol{\delta}_i)$ con offset $\boldsymbol{\delta}_i = r_s \hat{d}_i$, dove r_s è il *sensor range* e $\hat{d}_i$ è uno degli 8 versori cardinali e diagonali (N, S, E, O, NE, SE, NO, SO). Tali direzioni coincidono con le 8 azioni discrete disponibili al master.

Assumendo un'approssimazione lineare locale del campo (sviluppo di Taylor al primo ordine):

$$C(\mathbf{p} + \boldsymbol{\delta}_i) \approx C(\mathbf{p}) + \nabla C \cdot \boldsymbol{\delta}_i \quad (4.1)$$

Sottraendo C_0 da ciascuna lettura e normalizzando per r_s , la relazione (4.1) diventa $\hat{d}_i \cdot \nabla C \approx b_i$ per ogni sensore i , con:

$$b_i = \frac{C_i - C_0}{r_s}, \quad C_i \equiv C(\mathbf{p} + \boldsymbol{\delta}_i) \quad (4.2)$$

Raccogliendo gli 8 vincoli in forma matriciale ($\mathbf{D} \in \mathbb{R}^{8 \times 2}$, $D_{ij} = (\hat{d}_i)_j$) si ottiene il sistema sovradeterminato $\mathbf{D} \nabla C \approx \mathbf{b}$, risolto per minimi quadrati. La disposizione simmetrica delle 8 direzioni (per ogni $\hat{d}_i$ è presente anche $-\hat{d}_i$) implica $\mathbf{D}^\top \mathbf{D} = 4 \mathbf{I}_2$: il sistema è perfettamente condizionato e la stima del gradiente ha forma chiusa:

$$\hat{\nabla} C = \frac{1}{4} \mathbf{D}^\top \mathbf{b} = \frac{1}{4} \sum_{i=1}^8 \hat{d}_i b_i \quad (4.3)$$

In termini implementativi, la stima si riduce a poche righe: dalle 9 letture si costruisce il vettore $\mathbf{b}$ e si risolve il sistema ai minimi quadrati con `numpy`.

```
def estimate_gradient(obs, r_s, D):    # D: matrice 8x2 delle
    # direzioni d_i
    c0 = obs[0]                       # lettura del master (
    # concentrazione centrale)
```

```

sensors = obs[28:36]                # 8 letture riportate dagli
    slave

b = (sensors - c0) / r_s             # gradiente proiettato sulle
    direzioni (eq. 4.2)

grad, *_ = np.linalg.lstsq(D, b, rcond=None) # soluzione ai
    minimi quadrati (eq. 4.3)

return grad

```

4.2.2 Selezione dell'azione

Le 8 azioni discrete corrispondono esattamente alle 8 direzioni $\hat{d}_i$ usate dai sensori. Nel FCM puro il passo dell'agente è fisso e pari a $\Delta x = v_{\max} \Delta t = 1 \text{ m/s} \times 10 \text{ s} = 10 \text{ m}$. Avendo già stimato il gradiente $\hat{\nabla}C$ tramite la (4.3), si sceglie la direzione discreta più allineata ad esso:

$$a^* = \underset{i \in \{1, \dots, 8\}}{\operatorname{argmax}} \hat{d}_i \cdot \hat{\nabla}C \quad (4.4)$$

Se $\|\hat{\nabla}C\| < 10^{-12}$ (campo localmente piatto), l'azione viene scelta uniformemente a caso tra le direzioni valide. In entrambi i casi, le direzioni non ammissibili (collisione con la costa) sono escluse tramite l'*action masking* descritto nella Sezione 3.1.3.

4.2.3 Analisi del raggio di campionamento

Il parametro critico del FCM è il **raggio di campionamento** (*sensor range*) r_s , ovvero la distanza a cui gli agenti slave si posizionano rispetto al master per misurare la concentrazione (Sezione 3.1.1). La sua scelta è un compromesso: un raggio troppo piccolo rende le letture insensibili al gradiente, poiché le differenze di concentrazione su scala locale diventano trascurabili e dominate dal rumore; un raggio troppo grande campiona zone distanti dalla posizione corrente, riducendo la validità dell'approssimazione lineare (4.1) su cui si fonda la stima del gradiente e aumentando il tempo di convergenza.

Per individuare il valore ottimale sono stati testati cinque raggi $r_s \in \{20, 50, 100, 150, 200\} \text{ m}$ con il FCM a passo fisso, su 624 episodi per valore. La Tabella 4.1 riporta i risultati.

r_s	SR%	Step medi	V0	V1	V2	V3	Q1/4	Q1/2	Q3/4
20 m	19,4	274,5	17,9	10,3	19,2	30,1	41,3	1,0	15,9
50 m	32,9	182,7	39,7	16,0	23,7	51,9	50,5	11,5	36,5
100 m	18,8	411,6	28,2	8,3	16,0	22,4	28,8	1,4	26,0
150 m	13,6	546,4	16,0	5,1	12,2	21,2	16,8	1,0	23,1
200 m	13,8	398,7	6,4	6,4	13,5	28,8	11,1	7,7	22,6

Tabella 4.1: Sweep sul raggio di campionamento r_s (FCM a passo fisso, 624 episodi per valore). In verde il valore ottimale.

Il valore ottimale risulta $r_s = 50$ m, con un success rate globale del 32,9% e il tempo medio al successo più basso (~ 30 min). Per raggi superiori il success rate crolla e il tempo di convergenza aumenta, a causa dell'approssimazione lineare sempre meno valida su scale spaziali grandi rispetto alla struttura del plume; per $r_s = 20$ m, viceversa, il segnale locale è troppo debole. Il valore $r_s = 50$ m viene quindi adottato in tutto il seguito.

Le Figure 4.1–4.3 analizzano il comportamento del FCM puro alla configurazione ottimale ($r_s = 50$ m) sui 624 episodi.

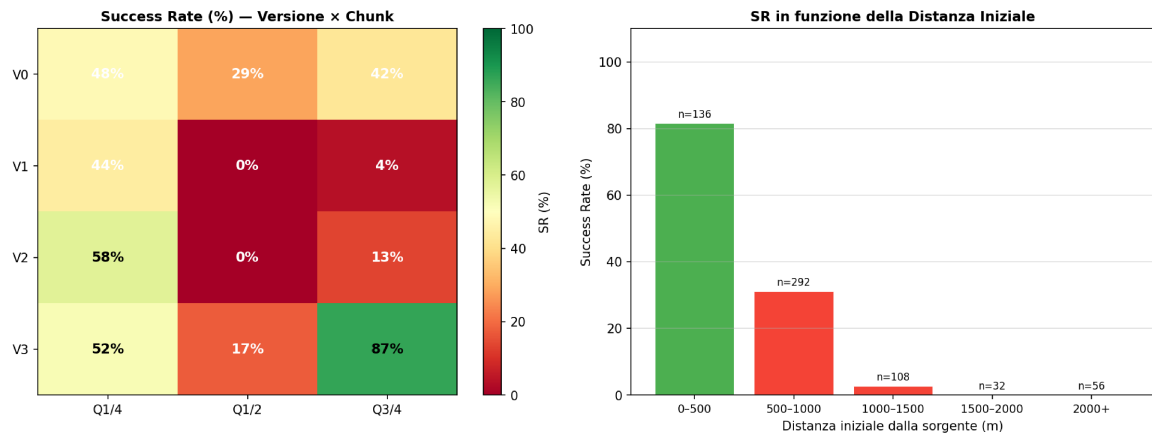


Figura 4.1: Success rate per scenario di vento e chunk temporale — FCM puro $r_s = 50$ m. Si evince che i fallimenti si concentrano negli scenari a plume diffuso (V1, V2) e nel chunk intermedio Q1/2, mentre V0/V3 e Q1/4 sono i più favorevoli.

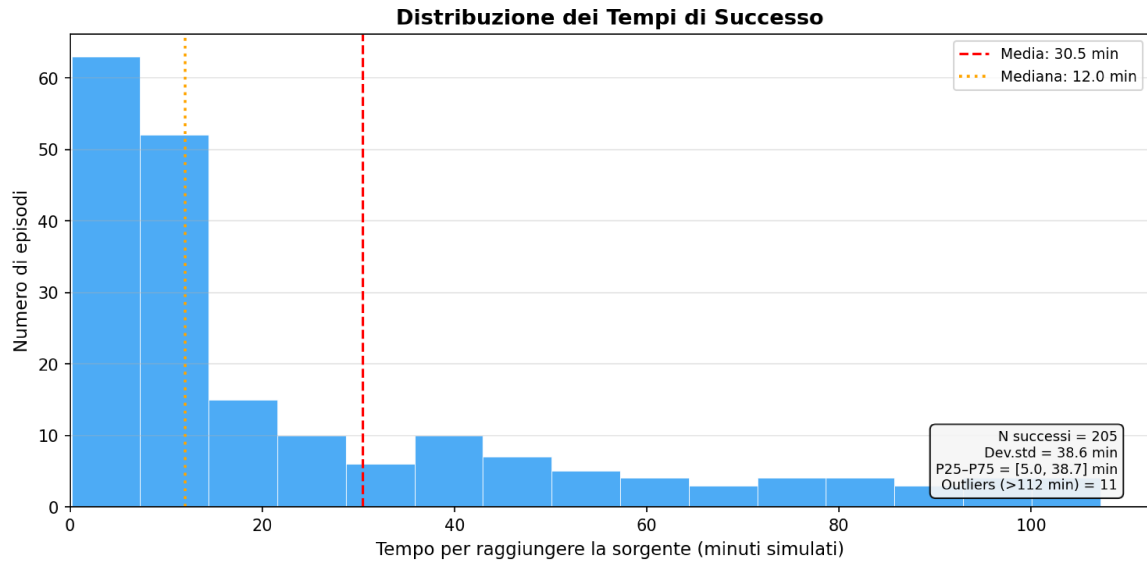


Figura 4.2: Distribuzione del tempo al successo — FCM puro $r_s = 50$ m. La maggior parte dei successi avviene nei primi minuti, ma la coda lunga rivela numerosi episodi in cui l'agente vaga a lungo prima di convergere.

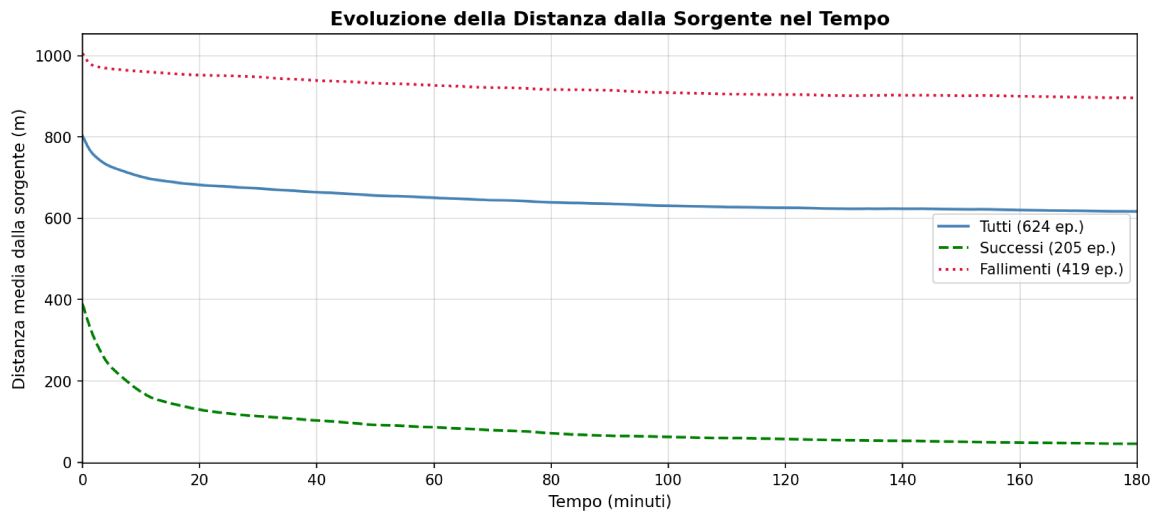


Figura 4.3: Distanza media dalla sorgente nel tempo — FCM puro $r_s = 50$ m. Gli episodi di successo riducono progressivamente la distanza, mentre i fallimenti restano stabili a grande distanza: il metodo reattivo entra in stallo senza riuscire ad avvicinarsi.

Limiti del FCM puro. Anche al raggio ottimale, il success rate globale del FCM puro si ferma al 32,9%. La causa principale è il *passo fisso*, indipendente dalla storia del gradiente: l'agente reagisce unicamente alla lettura corrente, senza memoria delle direzioni percorse. In presenza di segnale rumoroso o plume diffuso ciò produce traiettorie oscillanti e convergenza lenta, e il metodo tende a bloccarsi sui massimi locali di concentrazione frequenti in ambiente costiero. Questi limiti motivano la variante adattiva presentata nella sezione seguente.

4.3 FCM con passo adattivo (Adam)

Per superare i limiti del passo fisso, si sostituisce la regola di selezione (4.4) con un aggiornamento basato sull'ottimizzatore **Adam** (*Adaptive Moment Estimation*) [15]. Adam accumula un *momentum* sul gradiente stimato e normalizza il passo per la varianza storica del gradiente stesso, ottenendo un comportamento più robusto al rumore e più direzionato.

Ad ogni timestep t l'agente stima il gradiente locale $\hat{\nabla}C_t \in \mathbb{R}^2$ tramite la (4.3) e calcola il vettore di passo secondo la regola di aggiornamento Adam:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \hat{\nabla}C_t \quad (4.5)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \hat{\nabla}C_t^2 \quad (4.6)$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \quad (4.7)$$

$$\mathbf{s}_t = lr \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon} \quad (4.8)$$

dove $\mathbf{m}_t, \mathbf{v}_t \in \mathbb{R}^2$ sono il primo e il secondo momento del gradiente (inizializzati a $\mathbf{0}$ all'inizio di ogni episodio), $\beta_1 = 0,9$, $\beta_2 = 0,999$, $\varepsilon = 10^{-8}$, e lr è il *learning rate*, che rappresenta il passo base in metri. Il passo fisico effettivo è limitato superiormente da $\Delta x_{\max} = 50$ m, e l'azione discreta viene scelta come la direzione $\hat{d}_i$ più allineata al vettore $\mathbf{s}_t$:

$$\Delta x_t = \min(\|\mathbf{s}_t\|, \Delta x_{\max}), \quad a_t = \operatorname{argmax}_{i \in \{1, \dots, 8\}} \hat{d}_i \cdot \mathbf{s}_t \quad (4.9)$$

Il momentum (β_1) mantiene la direzione coerente su più timestep, smorzando le oscillazioni dovute al rumore del sensore; la normalizzazione per $\sqrt{\hat{\mathbf{v}}_t}$ riduce il passo nelle di-

rezioni in cui il gradiente è inconsistente nel tempo. Lo pseudocodice dell'aggiornamento è riportato di seguito.

FCM Adam: aggiornamento del passo per ogni timestep dell'episodio

— *Inizio episodio* —

$\mathbf{m} = \mathbf{0}_2; \quad \mathbf{v} = \mathbf{0}_2; \quad t = 0;$

— *Per ogni timestep* —

$\hat{\nabla}C = \text{stima_gradiente}(\text{obs}); \quad (\text{minimi quadrati, 8 sensori})$

$t = t + 1;$

$\mathbf{m} = \beta_1 \cdot \mathbf{m} + (1 - \beta_1) \cdot \hat{\nabla}C;$

$\mathbf{v} = \beta_2 \cdot \mathbf{v} + (1 - \beta_2) \cdot \hat{\nabla}C^2;$

$\hat{\mathbf{m}} = \mathbf{m} / (1 - \beta_1^t); \quad \hat{\mathbf{v}} = \mathbf{v} / (1 - \beta_2^t);$

$\mathbf{s} = lr \cdot \hat{\mathbf{m}} / (\sqrt{\hat{\mathbf{v}}} + \varepsilon);$

$\Delta x = \min(\|\mathbf{s}\|, \Delta x_{\max});$

$a = \operatorname{argmax}_i (\hat{d}_i \cdot \mathbf{s});$

Concretamente, ad ogni timestep la selezione dell'azione si traduce in:

```
self.t += 1
self.m = b1 * self.m + (1 - b1) * grad           # primo momento
self.v = b2 * self.v + (1 - b2) * grad**2        # secondo momento
m_hat = self.m / (1 - b1**self.t)                # bias correction
v_hat = self.v / (1 - b2**self.t)
step = lr * m_hat / (np.sqrt(v_hat) + eps)        # vettore di passo
Adam
```

```

action = int(np.argmax(D @ step))          # direzione piu'
allineata

```

4.3.1 Interpretazione del learning rate

Il parametro lr rappresenta il passo asintotico per direzioni cardinali del gradiente: a convergenza dei momenti Adam, $\|\mathbf{s}_t\| \rightarrow lr$ per gradienti puramente orizzontali o verticali, e fino a $lr\sqrt{2}$ per gradienti diagonali (comunque limitato a $\Delta x_{\max}$). Con $\Delta t = 10$ s, un passo di $lr = 10$ m corrisponde a una velocità di 1 m/s, compatibile con la velocità nominale del veicolo; valori maggiori di lr implicano velocità proporzionalmente più alte (fino a 5 m/s per $lr = 50$ m), a scapito del realismo fisico.

4.3.2 Analisi del learning rate e risultati

Il parametro lr è stato ottimizzato tramite uno sweep su $lr \in \{10, 20, 30, 40, 50\}$ m, con 624 episodi per valore e sensor range $r_s = 50$ m. La Tabella 4.2 riporta il success rate globale, gli step medi al successo e il dettaglio per scenario di vento e chunk temporale, confrontati con la baseline a passo fisso (10 m).

lr	SR%	Step medi	V0	V1	V2	V3	Q1/4	Q1/2	Q3/4
FCM puro	32,9	182,7	39,7	16,0	23,7	51,9	50,5	11,5	36,5
10 m	54,6	174,0	73,7	29,5	35,9	79,5	77,9	30,3	55,8
20 m	58,2	131,5	80,1	32,1	34,6	85,9	81,7	38,0	54,8
30 m	60,7	110,4	84,6	30,8	37,2	90,4	86,1	39,4	56,7
40 m	63,5	115,1	84,6	35,3	41,7	92,3	90,9	44,2	55,3
50 m	62,2	82,9	84,6	34,6	37,2	92,3	89,4	41,8	55,3

Tabella 4.2: Sweep su lr per FCM Adam ($\beta_1=0,9$, $\beta_2=0,999$, $\Delta x_{\max}=50$ m, $r_s=50$ m, 624 episodi per valore). In verde la configurazione con il success rate più alto.

Il FCM Adam con $lr = 40$ m raggiunge il success rate globale più alto, **63,5%**, con un guadagno di **+30,6 punti percentuali** rispetto al FCM puro a passo fisso (32,9%). Il success rate **non cresce indefinitamente** con lr : sale da 54,6% ($lr = 10$ m) fino al massimo a 40 m, per poi ripiegare leggermente a 62,2% a 50 m. Esiste dunque un passo ottimale, oltre il quale falcate troppo ampie iniziano a scavalcare la struttura del plume anziché seguirla. Il miglioramento rispetto al FCM puro è comunque diffuso su tutti gli scenari, con V3 e Q1/4 che beneficiano maggiormente del momentum, mentre V1, V2 e Q1/2 restano i più critici.

Gli step medi al successo calano complessivamente all'aumentare di lr (da 174 a 83): passi più ampi riducono il numero di timestep necessari a raggiungere la sorgente. L'andamento non è però regolare — a $lr = 40$ m gli step risalgono a 115,1 contro i 110,4 di 30 m — e la spiegazione più naturale è un effetto di *selezione*: è proprio a 40 m che il metodo porta a termine il maggior numero di episodi, inclusi quelli difficili e lunghi che alle altre configurazioni finivano in timeout e non entravano nella media. Un success rate più alto include successi più faticosi, e la media degli step ne risente.

Si ricorda infine (Sezione 4.3) che valori elevati di lr comportano velocità non realistiche: il valore $lr = 10$ m, fisicamente compatibile con la velocità nominale del veicolo, raggiunge già il 54,6% di SR.

Comparabilità degli spawn. Va segnalato che il FCM puro e il FCM adattivo non condividono lo stesso algoritmo di inizializzazione (Sezione 3.1.4): il primo, a passo fisso, adotta la *cascata di anelli* (distanza media di spawn ~ 800 m), mentre il secondo, a passo variabile, adotta il *disco adattivo*, che produce spawn in media più vicini alla sorgente. Una parte del divario fra le due varianti riflette quindi la diversa distribuzione delle distanze iniziali, e non il solo effetto dell'ottimizzatore Adam. Il confronto più rigoroso, a parità di condizioni, è quello fra RL e FCM adattivo alla stessa velocità massima (Sezione 5.5.9), in cui entrambi i metodi condividono il disco adattivo.

Le Figure 4.4–4.6 analizzano il comportamento del modello migliore ($lr = 40$ m) sui 624 episodi.

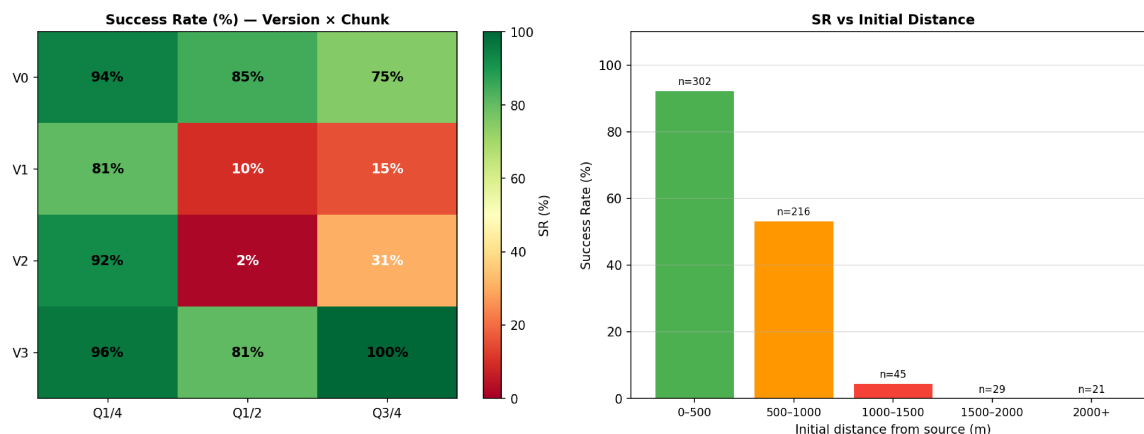


Figura 4.4: Success rate per scenario di vento e chunk temporale — FCM Adam $lr = 40$ m. Rispetto al FCM puro il miglioramento è diffuso su tutti gli scenari, ma V1, V2 e Q1/2 rimangono i più critici: il momentum mitiga ma non elimina la difficoltà del plume diffuso.

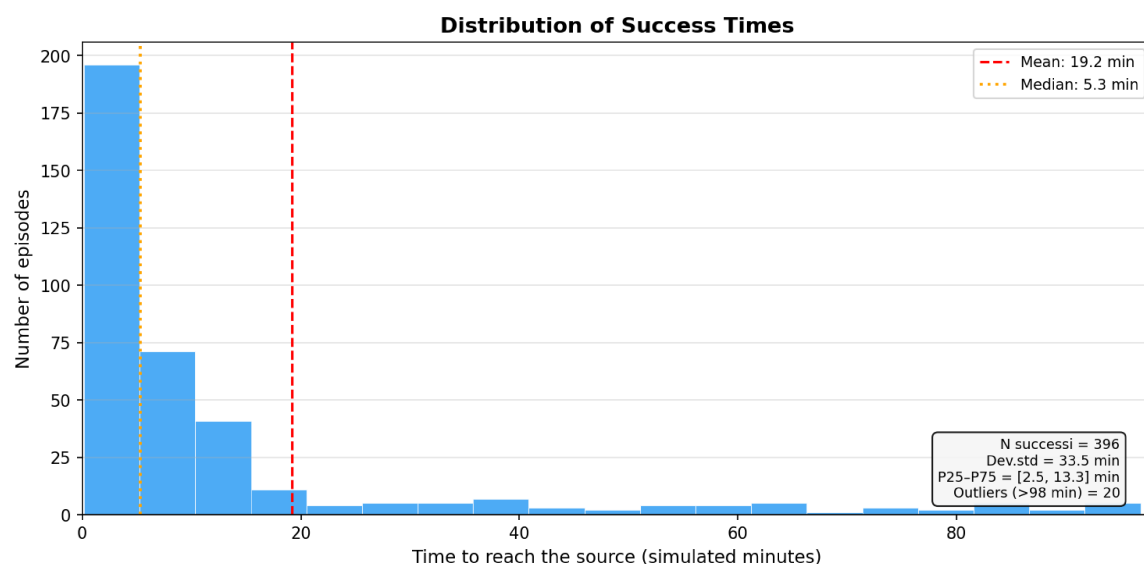


Figura 4.5: Distribuzione del tempo al successo — FCM Adam $lr = 40$ m. La distribuzione è più concentrata sui tempi brevi rispetto al FCM puro: il passo adattivo riduce il numero di timestep necessari a raggiungere la sorgente.

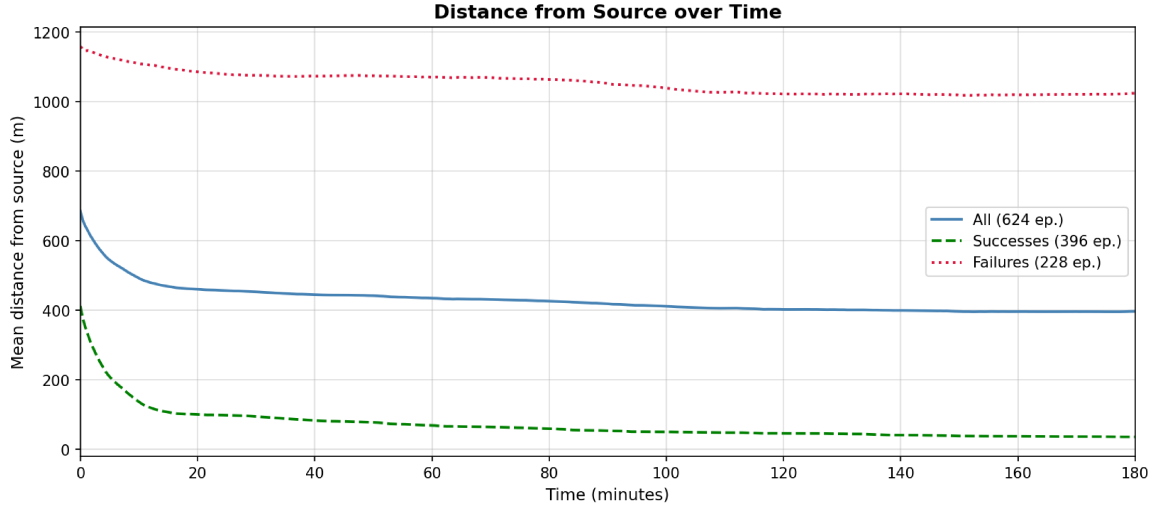
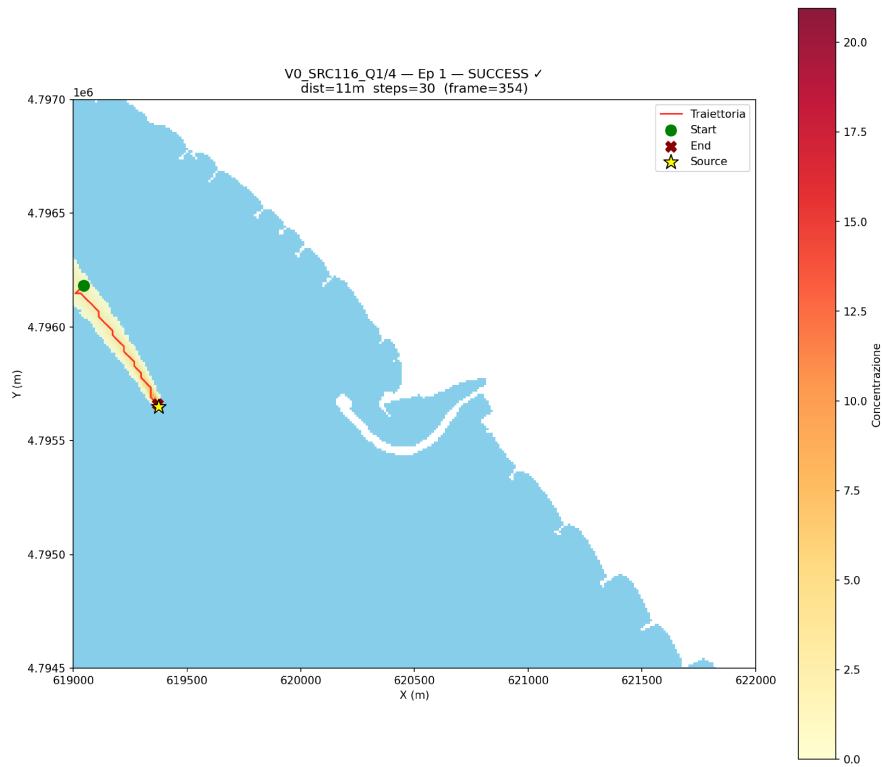
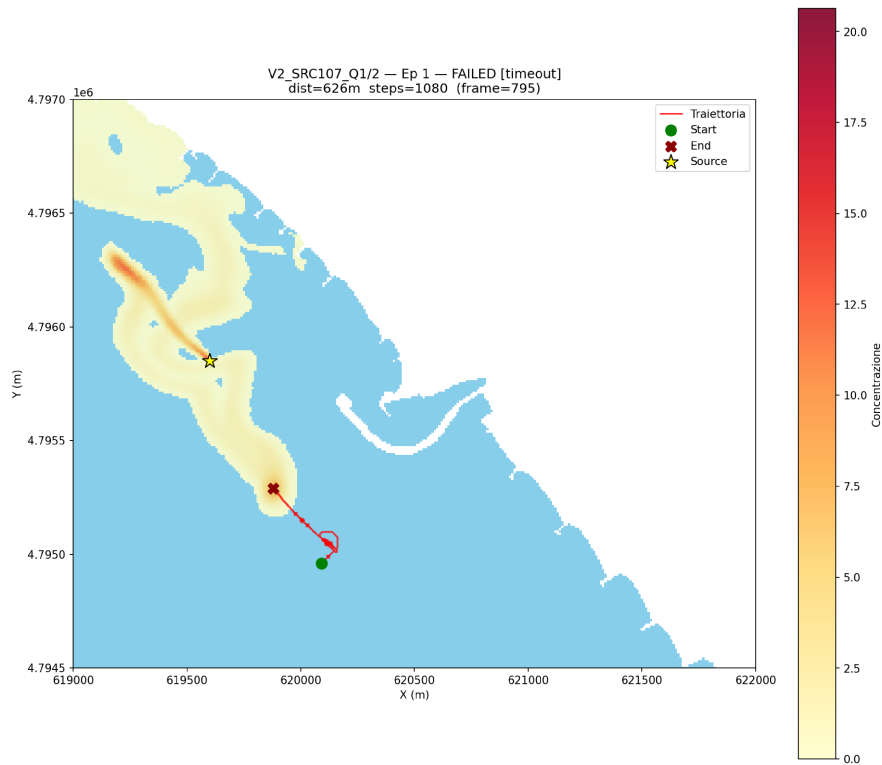


Figura 4.6: Distanza media dalla sorgente nel tempo — FCM Adam $lr = 40$ m. La curva dei successi cala più rapidamente che nel FCM puro; resta comunque una quota di fallimenti stabili a grande distanza negli scenari più difficili.

La Figura 4.7 mostra due episodi rappresentativi, tratti dalla configurazione $lr = 50$ m: un successo in condizioni di plume compatto (V0) e un fallimento in condizioni di plume disperso (V2). Nel caso di fallimento, il campo diffuso annulla il gradiente locale e l'agente non riesce a mantenere una direzione affidabile verso la sorgente, terminando in timeout — un limite qualitativo della risalita di gradiente che non dipende dal particolare valore di lr .



(a) Successo (V0, plume compatto).



(b) Fallimento (V2, plume disperso).

Figura 4.7: Traiettorie rappresentative del FCM Adam $lr = 50$ m. Nel successo (a) l'agente risale il plume compatto con traiettoria diretta; nel fallimento (b) il plume disperso annulla il gradiente locale e l'agente vaga senza convergere, terminando in timeout.

In sintesi, il metodo del gradiente adattivo migliora sensibilmente la baseline a passo fisso, ma resta limitato dalla sua natura reattiva: negli scenari a plume diffuso (V1, V2, Q1/2) il gradiente locale degenera e il success rate rimane basso. Questo limite strutturale motiva l'approccio basato su Reinforcement Learning presentato nel capitolo seguente.

Capitolo 5

Apprendimento per rinforzo: PPO

5.1 Motivazione

Il metodo del gradiente presentato nel Capitolo 4 è per natura *locale e reattivo*: decide l'azione basandosi solo sulle letture correnti degli slave, senza memoria dell'episodio né capacità di pianificazione. Anche nella sua versione adattiva con Adam, esso resta limitato negli scenari a plume diffuso, dove il gradiente locale degenera e il metodo si blocca su massimi locali.

L'approccio proposto in questo lavoro si basa sul **Reinforcement Learning** (RL), che mira a superare questi limiti apprendendo, tramite l'interazione ripetuta con l'ambiente [3], una politica di navigazione che sfrutta la storia delle osservazioni e le informazioni contestuali sul vento e sulle correnti. A differenza degli approcci gradient-based e model-based, l'RL è *model-free* e non richiede un modello esplicito dell'ambiente: la strategia emerge dall'ottimizzazione su milioni di episodi simulati. L'obiettivo è apprendere la politica del master a partire dalle stesse misurazioni della squadra di slave usate dal FCM, verificando se l'apprendimento produca strategie più robuste del gradiente reattivo.

5.2 Introduzione a PPO

Proximal Policy Optimization (PPO) [7] è un algoritmo di reinforcement learning *on-policy* appartenente alla famiglia dei *Policy Gradient Methods*. L'architettura segue lo

schema *actor-critic*: una rete (la *policy* o attore) produce la distribuzione di probabilità sulle azioni $\pi_\theta(a|s)$, mentre una seconda rete (la *value function* o critico) stima la funzione valore $V(s)$, ovvero il *ritorno atteso* (la somma scontata delle ricompense future) a partire dallo stato s seguendo la politica corrente.

In forma esplicita: poiché lo spazio d'azione di questo lavoro è *discreto*, l'attore produce un vettore di *logit* $z_\theta(s)$ e la politica è la distribuzione categorica ottenuta con la funzione *softmax*,

$$\pi_\theta(a | s) = \frac{\exp(z_\theta(s)_a)}{\sum_{a'} \exp(z_\theta(s)_{a'})}, \quad (5.1)$$

mentre la funzione valore è definita come il ritorno atteso scontato a partire da s ,

$$V^{\pi_\theta}(s) = \mathbb{E}_{\pi_\theta} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s \right], \quad (5.2)$$

con $\gamma \in (0, 1]$ il fattore di sconto (Tabella 5.3) e r_t la ricompensa al passo t ; il critico ne apprende un'approssimazione $V_\phi(s)$ tramite una seconda rete.

Il meccanismo chiave di PPO è il *clipped surrogate objective*: l'algoritmo massimizza il miglioramento della policy ma limita la deviazione rispetto alla policy precedente tramite il clipping del rapporto di probabilità $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (5.3)$$

dove $\hat{A}_t$ è la stima del vantaggio (*advantage*) e ϵ è il parametro di clipping. L'obiettivo complessivo combina tre contributi: la *policy loss* (clipped), la *value loss* che allinea la stima $V(s)$ ai ritorni osservati, e un termine di *entropy bonus* che mantiene l'esplorazione:

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 H[\pi_\theta] \quad (5.4)$$

La stima del vantaggio è ottenuta tramite *Generalized Advantage Estimation* (GAE) [16], che offre un buon compromesso tra bias e varianza. Durante il training la policy è stocastica (campionamento da π_θ); in inferenza può essere resa deterministica scegliendo l'azione a probabilità massima.

5.2.1 MaskablePPO

L'implementazione utilizzata è **MaskablePPO** della libreria *sb3-contrib*, estensione di PPO che integra l'*action masking* [8] descritto nella Sezione 3.1.3. Ad ogni passo, le azioni che porterebbero il master sulla terraferma o fuori dal dominio vengono escluse dalla distribuzione di probabilità, impedendo le collisioni con la costa senza penalità a posteriori e garantendo l'esplorazione di sole traiettorie ammissibili.

L'architettura della policy è una rete *Multi-Layer Perceptron* (MLP) con due reti separate per attore e critico, ciascuna composta da due strati nascosti da 256 neuroni con attivazione tanh. La libreria *Stable-Baselines3* [9], costruita su PyTorch, fornisce l'implementazione di base e ne rende l'uso riproducibile. La creazione dell'agente e l'avvio dell'addestramento si riducono a poche righe:

```
env = ActionMasker(SourceSeekingEnv(config), mask_fn)    # masking
               delle azioni
model = MaskablePPO("MlpPolicy", env,
                    policy_kwargs=dict(net_arch=[256, 256],
                                       activation_fn=nn.Tanh),
                    n_steps=4096, batch_size=128, gamma=0.99,
                                       clip_range=0.15)
model.learn(total_timesteps=6_000_000)
```

5.3 Spazio di osservazione

L'osservazione fornita alla rete neurale rappresenta il campo chimico locale, la cinematica dell'agente e le condizioni ambientali. Tutte le feature sono normalizzate. Se ne considerano due configurazioni, confrontate nella Sezione 5.5.4: la **singola corona (116 dimensioni)**, in cui gli slave formano un'unica corona di 8 sensori a $r_s = 20$ m attorno al master, e la **doppia corona (196 dimensioni)**, che aggiunge una seconda corona a $r_{s2} = 50$ m per dare all'agente informazione sul plume a scala spaziale più larga. La struttura è riportata nella Tabella 5.1.

Componente	Singola	Doppia	Descrizione
Concentrazione corrente	1	1	Concentrazione nella posizione attuale
Memoria concentrazione	9	9	Ultime 9 letture di concentrazione
Memoria spostamenti	18	18	Ultimi 9 spostamenti ($\Delta x, \Delta y$)
Sensori corona 1	8	8	Concentrazione nelle 8 direzioni a 20 m
Sensori corona 2	–	8	Concentrazione nelle 8 direzioni a 50 m
Memoria corona 1	72	72	Corona 1 per i 9 timestep passati
Memoria corona 2	–	72	Corona 2 per i 9 timestep passati
Vento	2	2	Componenti u, v [m/s]
Corrente	2	2	Componenti u, v [m/s]
Informazione posizionale	4	4	Direzione e valore della max concentrazione vista, step dall'ultimo contatto col plume
Totale	116	196	

Tabella 5.1: Spazio di osservazione della rete neurale: singola corona (116 dimensioni) e doppia corona (196 dimensioni).

5.4 Addestramento

I modelli sono addestrati per 6×10^6 step con gli iperparametri riportati nella Tabella 5.3, sulla suddivisione del dataset e con l'augmentation temporale descritte nella

Sezione 2.1.5.

Learning rate decrescente. Il learning rate segue uno **schedule a gradini decrescenti** (Tabella 5.2, sinistra): si parte da un valore relativamente alto (3×10^{-4}) nelle prime fasi, quando la policy è ancora lontana dall’ottimo e conviene compiere aggiornamenti ampi per esplorare rapidamente lo spazio delle soluzioni, e lo si riduce progressivamente fino a 1×10^{-5} nelle fasi finali, quando servono aggiustamenti più fini per affinare la politica senza destabilizzare quanto già appreso.

Curriculum sugli scenari. La distribuzione degli scenari di addestramento evolve tramite un parametro $\alpha \in [0, 1]$ (Tabella 5.2, destra): per $\alpha = 0$ gli scenari sono estratti in modo **uniforme**, mentre per $\alpha \rightarrow 1$ il peso si concentra progressivamente sugli scenari più difficili — lo scenario di vento V2 e i chunk temporali centrali (Q1/2), caratterizzati da plume più diffuso. In questo modo l’agente apprende prima le situazioni più semplici e, una volta acquisita una politica di base, viene esposto in misura crescente ai casi critici, secondo la logica del *curriculum learning*.

Step	LR	Step	α
0 – 2 000 000	3×10^{-4}	0 – 2 000 000	0,0
2 000 000 – 4 000 000	1×10^{-4}	2 000 000 – 4 000 000	0,5
4 000 000 – 5 000 000	5×10^{-5}	4 000 000 – 6 000 000	0,8
5 000 000 – 6 000 000	1×10^{-5}		

Tabella 5.2: Schedule del learning rate (sinistra) e curriculum sugli scenari (destra).

Parametro	Valore
Algoritmo	MaskablePPO
Policy	MLP (256×256 , tanh)
Step totali	6 000 000
<i>n_steps</i>	4096
Batch size	128
Epoche	10
γ	0,99
λ (GAE)	0,95
Clip range	0,15
Coeff. value function	0,3
Coeff. entropia	0,01
Max grad norm	0,5
Target KL	0,02

Tabella 5.3: Iperparametri di training del modello PPO.

5.5 Risultati

La valutazione è condotta sulle 26 sorgenti held-out (SRC107–SRC132) su 4 scenari di vento e 3 chunk temporali, in modalità deterministica, per un totale di 1560 episodi per modello. Si analizzano in sequenza l’effetto del raggio di campionamento (Sezione 5.5.1) e della funzione di reward (Sezione 5.5.2), che fissano il modello di riferimento **minimal**. Da questo si sviluppa la **catena a velocità variabile** (Sezione 5.5.3), in cui la velocità diventa una scelta dell’agente, e su ciascuno stadio della catena si valuta la **doppia corona** (Sezione 5.5.4). Seguono due approfondimenti sulla politica così ottenuta — il restringimento del raggio di successo (Sezione 5.5.5) e le heatmap di occupazione (Sezione 5.5.6) — quindi l’analisi delle spawn map (Sezione 5.5.7) e il confronto diretto con il metodo del gradiente (Sezione 5.5.8), esteso infine all’intero range di velocità operative (Sezione 5.5.9).

5.5.1 Analisi sul raggio di campionamento crescente

Analogamente a quanto fatto per l'FCM (Sezione 4.2.3), si è indagato l'effetto del raggio di campionamento r_s — la distanza a cui gli slave si posizionano attorno al master — anche per l'agente RL. A partire da un modello base addestrato con $r_s = 20$ m (success rate 94%), è stato condotto un *fine-tuning progressivo a catena* con raggi crescenti $r_s \in \{50, 100, 150, 200\}$ m: ogni modello è stato ottenuto rifinando per 2×10^6 step il modello dello stadio precedente ($20 \rightarrow 50 \rightarrow 100 \rightarrow 150 \rightarrow 200$ m), con distribuzione uniforme di scenari e chunk. La Tabella 5.4 riporta i risultati.

r_s	SR%	V0	V1	V2	V3	Q1/4	Q1/2	Q3/4
20 m	94	100	82	87	99	100	84	96
50 m	93	100	82	91	100	100	84	96
100 m	87	100	85	82	82	100	81	81
150 m	63	65	60	61	66	61	56	72
200 m	52	47	45	39	75	49	39	66

Tabella 5.4: Sweep sul raggio di campionamento per l'agente RL (fine-tuning progressivo dal modello base a 20 m, 1560 episodi per valore). In verde il valore ottimale.

Si osserva una **degradazione monotona** delle prestazioni all'aumentare del raggio, con il calo più netto tra 100 m (87%) e 150 m (63%). La spiegazione è di natura fisica e coincide con quella dell'FCM: il criterio di successo richiede di arrivare entro 50 m dalla sorgente, un obiettivo di navigazione *locale*; sensori troppo distanti (es. 200 m) campionano punti oltre la sorgente stessa, invertendo o annullando il gradiente direzionale nella fase critica di avvicinamento. Il raggio ottimale per questo task e dataset è dunque $r_s \approx 20$ m, fissato dalla scala caratteristica del plume nelle simulazioni MIKE21; aumentarlo sostituisce informazione locale utile con informazione irrilevante. Il risultato

è indipendente dall'architettura della rete e dalla funzione di reward, identiche per tutti i modelli testati. Tutti i modelli successivi adottano quindi $r_s = 20$ m.

5.5.2 Analisi sulla funzione di reward

Fissato il raggio ottimale, si è studiato l'effetto della **funzione di reward**. Ad ogni passo l'ambiente calcola una ricompensa scalare come *somma* di più componenti; le tre varianti confrontate differiscono per quali componenti sono attive e per la *forma* del termine di distanza. Se ne descrivono di seguito i singoli componenti.

- **Bonus di raggiungimento:** +100 all'arrivo (distanza ≤ 50 m), più un bonus di tempo residuo $\max(0, (N_{\max} - k)/N_{\max} \cdot 50)$ che premia l'arrivare in anticipo.
- **Penalità di uscita dal dominio:** -10 se il passo porta fuori dai bordi.
- **Reward di distanza** (segnale dominante): premia l'avvicinamento alla sorgente. In forma *lineare* è proporzionale alla riduzione di distanza nel passo, $(d_{t-1} - d_t) K/3000$; in forma *quadratica* (variante **full**) usa un potenziale $\propto (Q - d)^2$ modulato da uno *zone multiplier* (che pesa di più i progressi vicino alla sorgente) e da una *retreat asymmetry* (il moto di allontanamento è penalizzato $3\times$).
- **Gradiente di concentrazione:** $\pm 0,05$ a seconda che la concentrazione sotto l'agente aumenti o diminuisca nel passo.
- **Allineamento a vento e corrente:** $\pm 0,05$ per ciascuno, che premia il muoversi *controvento* e *controcorrente* (assunti diretti verso la sorgente).
- **Penalità di tempo:** -0,1 per passo, per incentivare la rapidità.
- **Prossimità alla costa:** rampa lineare da 0 (a 30 m dalla terra) fino a -15 (sul bordo).
- **Stagnazione** (solo **full**): se in una finestra di passi la distanza minima non migliora abbastanza, una penalità scoraggia il vagare; una variante *direzionale* penalizza i percorsi in cui lo spostamento netto è piccolo rispetto a quello totale (moto inefficiente).

Le tre varianti si ottengono attivando o disattivando questi componenti (parametro `reward_mode`):

full

tutti i componenti: distanza *quadratica* con zone multiplier e retreat asymmetry, allineamento a vento/corrente e le penalità di stagnazione.

base

distanza *lineare* e simmetrica; rimossi zone multiplier, retreat asymmetry e stagnazione. Allineamento a vento e corrente ancora attivi.

minimal

come **base**, ma con l'allineamento a vento e corrente *rimosso*. Restano i soli segnali essenziali: raggiungimento (+100), uscita dal dominio (-10), distanza lineare, gradiente di concentrazione ($\pm 0,05$), tempo (-0,1/step) e prossimità alla costa. Nel codice questa variante corrisponde a `reward_mode = "base_no_wind_reward"`.

In codice, l'assemblaggio del reward e i punti in cui le varianti divergono sono:

```
def _compute_reward(self):
    r = 0.0
    if self._source_reached():                # 1) raggiungimento
        r += 100.0 + max(0, (N_max - k) / N_max * 50)  # (+ bonus di
                                                    tempo)
    if self._out_of_bounds():                  # 2) fuori dominio
        r += -10.0
    r += self._distance_reward(d, d_prev)      # 3) distanza
    r += 0.05 if conc > prev_conc else -0.05   # 4) gradiente conc
    .
    if self.reward_mode != "base_no_wind_reward": # 5) allineamento (
        no in minimal)
        r += self._alignment_reward(v, wind, current)
    r += -0.1                                # 6) tempo
    r += self._land_proximity_penalty(d_land)  # 7) prossimità'
        costa
    if self.reward_mode == "full":            # 8) stagnazione (
        solo full)
        r += self._stagnation_penalty(dist_history)
    return r
```

Il termine di distanza è l'unico a cambiare *forma* tra le varianti:

```

def _distance_reward(self, d, d_prev):
    K = self.distance_reward_multiplier          # = 100
    if self.reward_mode == "full":              # potenziale
        quadratico
        QMAX = 3000.0
        raw = ((QMAX - d)**2 - (QMAX - d_prev)**2) / QMAX**2
        zone = self._distance_zone_multiplier(d)    # pesa di piu'
        vicino alla sorgente
        mult = 1.0 if d <= d_prev else self.
            retreat_penalty_multiplier    # 3x se si allontana
        return raw * zone * mult * K
    else:                                         # base/minimal:
        LINEARE, simmetrico
        return (d_prev - d) * K / 3000.0

```

L'allineamento a vento e corrente — presente in **full** e **base**, assente in **minimal** — premia il moto *contro* il campo (controvento/controcorrente):

```

def _alignment_reward(self, v, wind, current):
    r = 0.0
    for field in (wind, current):
        n = np.linalg.norm(field)
        if n > 1e-6:
            align = np.dot(v, field) / n          # <0 = contro il
            campo = verso la sorgente
            r += 0.05 if align < 0 else -0.05
    return r

```

I tre modelli sono stati addestrati da zero con configurazione identica e valutati su 1560 episodi ciascuno. La Tabella 5.5 riporta il confronto.

Metrica	base	full	minimal
SR globale	88,3%	95,6%	96,9%
Step medi (succ.)	113,8	129,6	129,2
V0	99,5%	98,0%	100,0%
V1	95,4%	99,0%	99,0%
V2	67,9%	88,5%	88,7%
V3	90,3%	97,0%	99,7%
Q1/4	100,0%	100,0%	100,0%
Q1/2	74,4%	93,7%	93,5%
Q3/4	90,4%	93,0%	97,1%

Tabella 5.5: Confronto delle tre varianti di reward su 1560 episodi ciascuna.

Il modello **minimal** ottiene il success rate più alto (**96,9%**), superando sia **full** (95,6%) sia **base** (88,3%). Due osservazioni emergono. Primo, i componenti aggiuntivi della reward **full** (stagnation penalty, zone multiplier, retreat asymmetry) non producono un guadagno misurabile rispetto a una reward lineare più semplice. Secondo, il divario più significativo è tra **base** e **minimal** ($-8,6\%$ globale), imputabile ai reward di allineamento a vento e corrente: tali segnali assumono che la sorgente si trovi sempre controvento/controcorrente, ipotesi non valida in ambiente costiero complesso, e il segnale risultante entra in conflitto con il gradiente chimico in una frazione rilevante degli scenari. Il gradiente di concentrazione, calcolato direttamente sul campo simulato, incorpora già implicitamente l'effetto di vento e corrente. Il modello **minimal** è quindi adottato come modello di riferimento; le Figure 5.1–5.4 ne riportano l'analisi quantitativa.

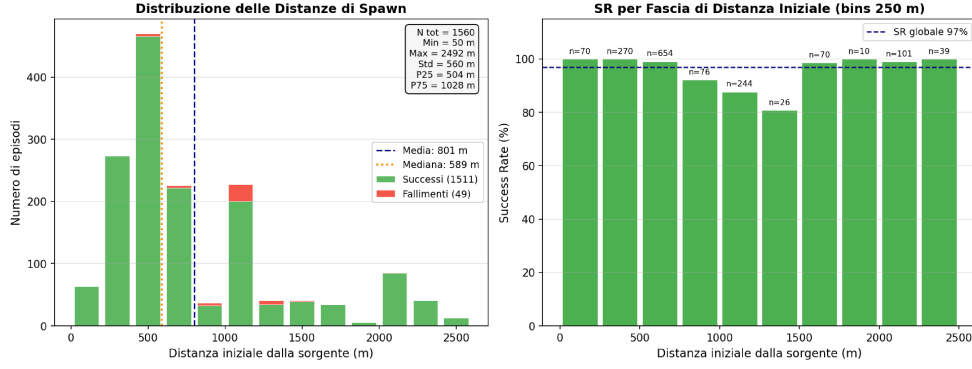


Figura 5.1: Distribuzione delle distanze iniziali di spawn — modello *minimal*. La success rate resta elevata su tutte le fasce di distanza iniziale (anche oltre 2000 m), a conferma che la politica generalizza indipendentemente dal punto di partenza.

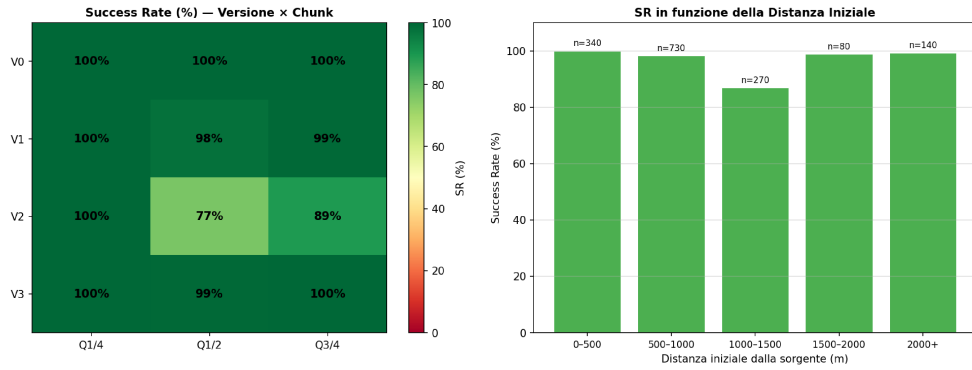


Figura 5.2: Success rate per scenario di vento e chunk temporale — modello *minimal*. Quasi tutte le combinazioni sono risolte vicino al 100%; le uniche difficoltà residue — comunque lievi — restano su V2 e Q1/2, gli stessi scenari critici per l'FCM.

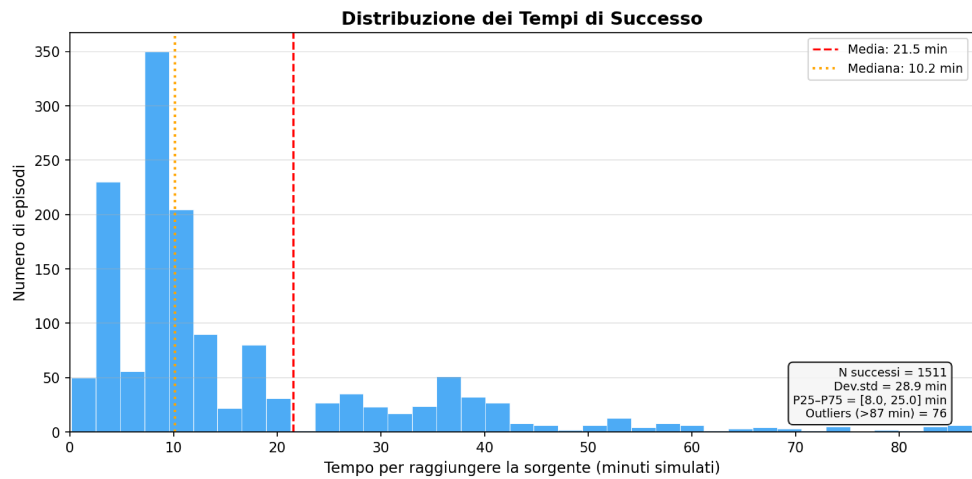


Figura 5.3: Distribuzione del tempo al successo — modello *minimal*. La gran parte degli episodi converge entro pochi minuti simulati, con una coda contenuta: l'agente raggiunge la sorgente in modo rapido e consistente.

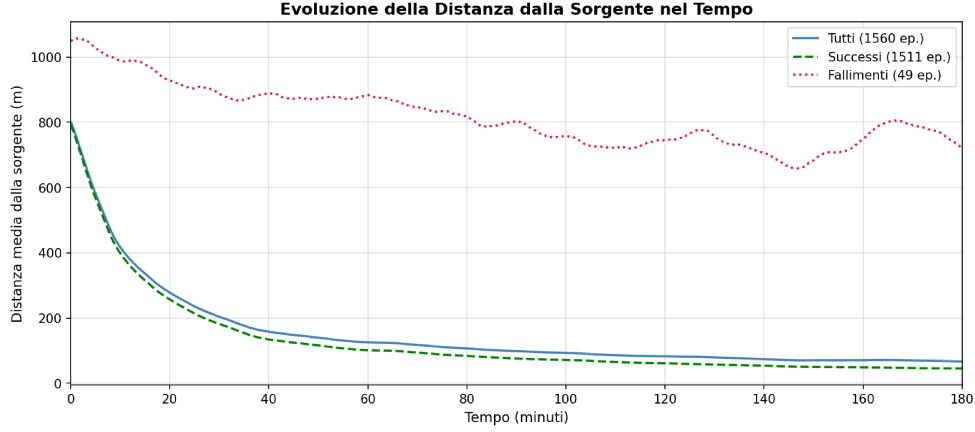


Figura 5.4: Distanza media dalla sorgente nel tempo — modello `minimal`. La curva dei successi decresce in modo netto e monotono fino all’azzeramento; i rari fallimenti restano una frazione marginale, in netto contrasto con i metodi del gradiente.

5.5.3 Catena a velocità variabile

Nei modelli visti finora la velocità è un parametro fisso del veicolo e l’agente sceglie solo la direzione. È una limitazione: un AUV reale può modulare la spinta, e non è detto che andare sempre al massimo sia ottimale. Si è quindi reso la velocità una **scelta dell’agente**.

Spazio d’azione. Lo spazio d’azione passa da 8 direzioni (passo fisso) a `Discrete(8 × K)` con $K = 5$: ogni azione codifica *congiuntamente* una delle 8 direzioni e uno dei 5 livelli di velocità $\{0, 2, 4, \dots, 10\} \cdot v_{\max}$. L’agente sceglie così a ogni passo non solo *dove* andare ma anche *quanto* spingere. La velocità ottimale non è imposta da una regola: emerge **implicitamente** dall’ottimizzazione del ritorno — l’agente potrebbe apprendere, ad esempio, che rallentare in prossimità della sorgente o in controcorrente conviene. La singola azione intera viene decodificata in una coppia (direzione, velocità) tramite divisione e resto per K :

```
def _decode_action(self, action):          # action in {0, ..., 8K-1}
    K = self.config.n_velocity_levels      # K = 5 livelli
    dir_idx = action // K                  # una delle 8 direzioni
    vel_idx = action % K                    # livello di velocità
    velocity = (vel_idx + 1) / K * self.config.max_velocity # in
    (0, v_max]
```

```

return dir_idx, velocity          # dove andare + quanto
    spingere

```

Perché una catena di warm-start. Passare direttamente da un modello a passo fisso (1 m/s) a uno con velocità libera fino a 5 m/s è un salto difficile: lo spazio d'azione si quintuplica (da 8 a 40 azioni) e la dinamica dell'episodio cambia. L'addestramento è perciò organizzato a **catena**, in modo che ogni incremento sia un piccolo aggiustamento su una policy già competente anziché un apprendimento da zero:

- il primo stadio ($v_{\max}=1$) parte dal modello **minimal** a passo fisso, ne **trapianta il corpo** (le rappresentazioni già apprese) ed espande la testa d'azione da 8 a 40 uscite; l'agente impara a modulare la velocità entro lo stesso tetto di 1 m/s;
- ogni stadio successivo ($v_{\max}=2, \dots, 5$) riparte dal precedente (fine-tuning) e alza progressivamente il tetto di velocità.

Ogni stadio è valutato su 1560 episodi. Il trapianto della testa d'azione replica i logit di ciascuna direzione sui K livelli, favorendo all'avvio il livello $v_{\max}$ tramite il bias (variante *max-bias*), così che la policy parta equivalente a quella a passo fisso:

```

# corpo (MLP 256x256) e critico: copiati invariati dal modello
    minimal.
# testa azione (8 -> 8*K): ogni direzione replicata sui K livelli di
    velocita'.
for d in range(n_directions):          # 8 direzioni
    for k in range(K):                 # 5 livelli di
        velocita'
        row = d * K + k
        W[row] = w_min[d]              # stessi pesi della
            direzione d
        B[row] = b_min[d] - (0.0 if k == K-1 else suppress) # bias
            sul livello v_max

```

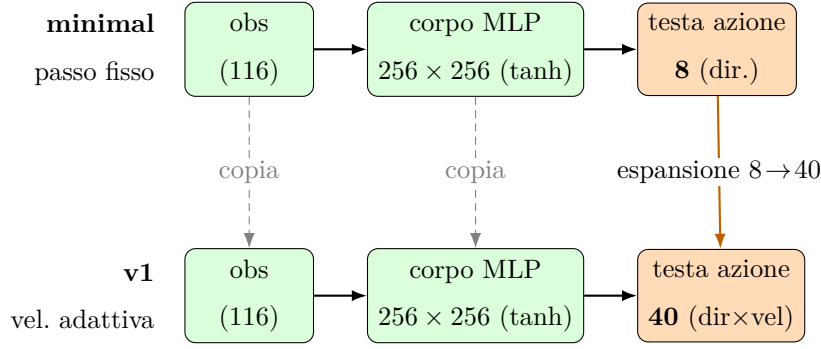



Figura 5.5: Warm-start del primo stadio (*minimal* → *v1*). Il **corpo** della rete (ingresso a 116 dim. e MLP 256×256) viene **copiato** invariato (verde); solo la **testa d'azione** viene **espansa** da 8 a 40 uscite (arancio): ogni direzione viene replicata sui 5 livelli di velocità, con bias iniziale verso $v_{\max}$ (all'avvio la policy equivale a quella a passo fisso).

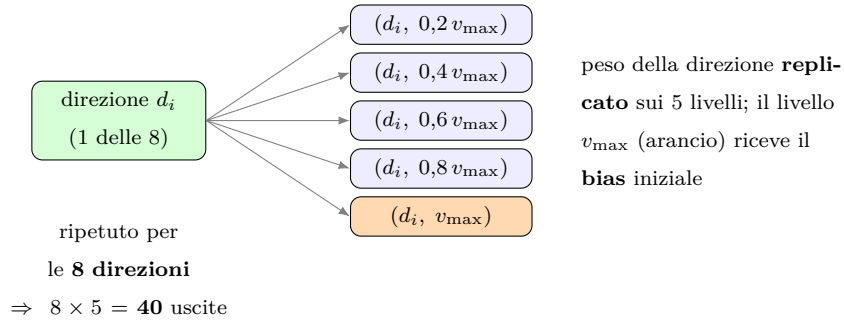


Figura 5.6: Focus sulla testa d'azione (espansione $8 \rightarrow 40$). Ciascuna delle 8 direzioni della testa *minimal* viene sdoppiata nei 5 livelli di velocità (d_i, v_k): i pesi sono replicati e il livello $v_{\max}$ è favorito all'avvio, così la policy parte equivalente a quella a passo fisso.

$v_{\max}$	SR%	Step (min)	V0	V1	V2	V3	Q1/4	Q1/2	Q3/4
1 m/s	97,1	119,6 (19,9)	99,5	100,0	89,7	99,2	100,0	95,0	96,3
2 m/s	99,7	69,5 (11,6)	100,0	99,5	99,2	100,0	100,0	99,0	100,0
3 m/s	99,9	46,9 (7,8)	100,0	100,0	99,7	100,0	100,0	99,8	100,0
4 m/s	99,9	38,2 (6,4)	100,0	100,0	99,5	100,0	100,0	100,0	99,6
5 m/s	99,9	27,8 (4,6)	100,0	100,0	99,7	100,0	100,0	100,0	99,8

Tabella 5.6: Catena PPO a velocità variabile (1560 episodi per stadio, $K=5$). “Step (min)” riporta gli step medi al successo e, tra parentesi, i minuti corrispondenti (1 step = 10 s). All’aumentare di $v_{\max}$ crescono SR e velocità (step medi da ~ 20 a ~ 5 min).

Già il primo stadio ($v_{\max}=1$, equivalente in velocità al modello a passo fisso) raggiunge il 97,1%, superando il 96,9% di **minimal**: rendere la velocità una scelta non costa nulla nemmeno a parità di tetto. Alzare $v_{\max}$ produce due benefici congiunti: l'SR sale al 99,7–99,9% e gli step medi al successo crollano da 120 a 28 (oltre $4\times$ più veloce). Il guadagno maggiore è su **V2**, lo scenario a corrente forte: la velocità netta dell'agente è $v_{\text{agente}} - v_{\text{corrente}}$, e con più velocità disponibile l'agente batte più facilmente la corrente contraria, passando da 89,7% a 99,7%. A partire da $v_{\max}=3$ le prestazioni sono sostanzialmente sature ($\approx 99,9\%$): l'ulteriore margine di velocità serve solo a ridurre il tempo al successo, non l'affidabilità.

Andamento della velocità media. In media l'agente resta prossimo a $v_{\max}$ a ogni stadio; la modulazione (gli avvallamenti della media e l'ampiezza della banda) **non è monotona** in $v_{\max}$: è **massima agli stadi intermedi** ($v_{\max}=2, 3$) e **quasi assente agli estremi**. A $v_{\max}=1$ il passo pieno (10 m) è molto minore del raggio di successo (50 m): non c'è rischio di scavalcare la sorgente, andare al massimo è quasi sempre sicuro e la media resta piatta. A $v_{\max}=5$ il passo pieno (50 m) eguaglia il raggio e, grazie alla cattura sul segmento percorso, una falcata piena centra comunque il bersaglio: di nuovo conviene il massimo. Ai valori intermedi, invece, il passo diventa confrontabile con la scala di avvicinamento e i rallentamenti correttivi si fanno più evidenti.

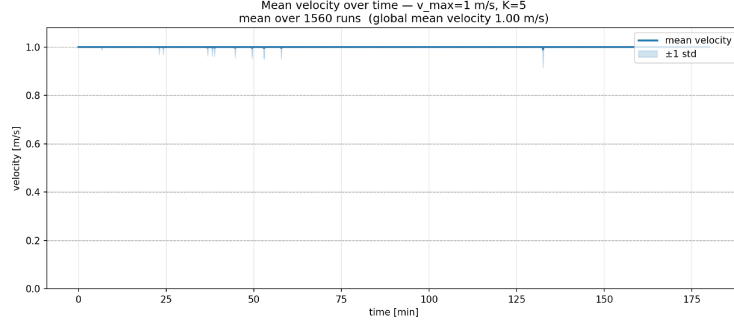
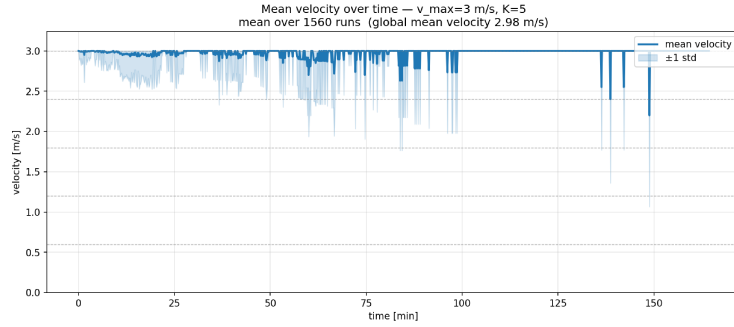
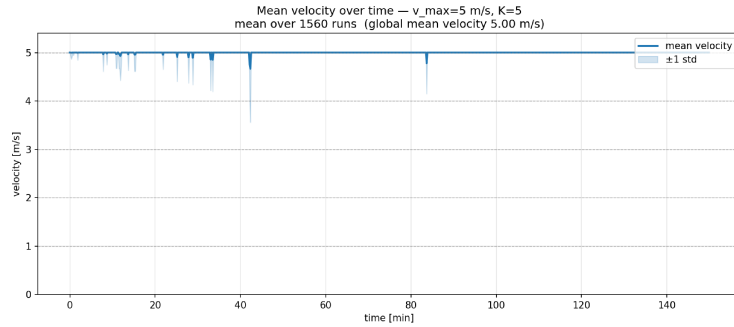
(a) $v_{\max}=1$ m/s(b) $v_{\max}=3$ m/s(c) $v_{\max}=5$ m/s

Figura 5.7: PPO adattivo: velocità media (su tutte le run) nel tempo, stadi $v_{\max}=1,3,5$. In media l'agente resta prossimo a $v_{\max}$ ma modula il passo con rallentamenti correttivi, più evidenti allo stadio intermedio e quasi assenti agli estremi.

5.5.4 Formazione a singola vs. doppia corona

La doppia corona (Tabella 5.1) aggiunge alla corona locale a $r_s = 20$ m una **seconda corona** di 8 slave a $r_{s2} = 50$ m, per dare al master informazione sul plume a scala spaziale più larga. La domanda è: questa seconda corona **serve**, e **a quali velocità**?

Un primo confronto, e perché è viziato. Il primo tentativo ha addestrato il modello a doppia corona *da zero*, con la stessa configurazione del riferimento **minimal**

(Tabella 5.7).

Metrica	Singola corona (116)	Doppia corona (196)
SR globale	96,9%	93,0%
Step medi (succ.)	129,2	114,4
V1	99,0%	90,3%
V2	88,7%	82,3%
Q1/2	93,5%	86,2%

Tabella 5.7: Primo confronto fra singola corona (*minimal*, $r_s=20$ m) e doppia corona (20 m + 50 m), entrambe a passo fisso e addestrate **da zero**. Il confronto è viziato: cambia insieme il sensore e la difficoltà di ottimizzare da zero una rete più grande. La Tabella 5.8 lo rifà in modo controllato.

Letto così, il risultato sembrerebbe chiudere la questione: la doppia corona *non* supera il riferimento, con un calo di 3,9 punti concentrato su V1, V2 e Q1/2. Il confronto è però **viziato**, perché cambia due cose insieme: il sensore *e* la dimensione della rete da ottimizzare da zero. Una rete con 80 ingressi in più è più difficile da addestrare a partire da pesi casuali, e il calo osservato confonde l'effetto dell'informazione con quello dell'ottimizzazione. Non si può quindi concludere che la seconda corona sia dannosa: si può solo dire che addestrare da zero una rete più grande rende meno.

Un protocollo equo: warm-start. Per isolare il *solo* effetto del sensore, la doppia corona è ottenuta per **warm-start dal corrispondente modello a singola corona**: per ogni $v_{\max} \in \{1, \dots, 5\}$ il modello a doppia corona parte da quello a singola corona di pari velocità, allargando lo strato di ingresso da 116 a 196 dimensioni e **azzerando** le colonne della corona-2. All'avvio la rete è quindi *identica* alla singola corona — ignora la corona-2 — e impara durante il fine-tuning se e come usarla. Ogni eventuale differenza è così attribuibile al solo sensore, non al protocollo. Ogni modello è valutato su 1560 episodi. L'allargamento dello strato d'ingresso conserva le colonne condivise e azzerava quelle della corona-2:

```
# doppia corona (196) warm-startata dalla singola (116):
# tutti i pesi copiati dal single; nello strato d'ingresso le colonne
  condivise
# sono rimappate e le 80 colonne della corona-2 sono AZZERATE.
W_in_dual[:, shared_cols] = W_in_single      # resto identico al single
W_in_dual[:, ring2_cols]   = 0.0             # corona-2 non
  contribuisce all'avvio
# le statistiche di VecNormalize della corona-2 restano neutre (media
  0, var 1).
```

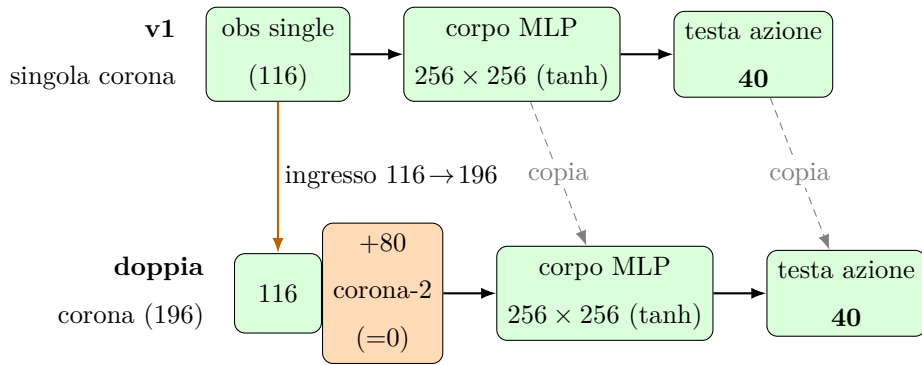


Figura 5.8: Warm-start della doppia corona (esempio a $v_{\max}=1$; a ogni livello è analogo, dal rispettivo modello a singola corona). **Corpo** e **testa d'azione** sono **copiati** invariati (verde); si allarga solo lo **strato d'ingresso** da 116 a 196 (arancio), aggiungendo 80 colonne per la corona-2 con **pesi inizializzati a zero**. All'avvio la rete è identica a v_1 e ignora la corona-2; il fine-tuning decide se e come usarla.

$v_{\max}$	SR sing.	SR dopp.	Step sing.	Step dopp.	Δ step
1	97,1%	95,8%	119,6 (19,9 min)	97,8 (16,3 min)	−18,2%
2	99,7%	99,7%	69,5 (11,6 min)	49,8 (8,3 min)	−28,3%
3	99,9%	100,0%	46,9 (7,8 min)	31,3 (5,2 min)	−33,2%
4	99,9%	99,9%	38,2 (6,4 min)	25,3 (4,2 min)	−33,6%
5	99,9%	100,0%	27,8 (4,6 min)	20,9 (3,5 min)	−24,8%

Tabella 5.8: Singola vs. doppia corona a ogni $v_{\max}$ (ogni doppia corona warm-startata dal modello a singola corona omologo; 1560 episodi per cella). “Step” = step medi al successo (minuti tra parentesi). Δ step = variazione relativa dei passi (verde = doppia più veloce).

Il quadro **si ribalta** rispetto alla lettura precedente, e per due motivi distinti. Primo, **il costo di SR è confinato a $v_{\max}=1$** : già con il solo warm-start la penalità scende da $-3,9$ a $-1,3$ — prova che gran parte del calo originale era difficoltà di ottimizzazione, non informazione dannosa — e ai livelli superiori non si ripresenta affatto: a v_2 – v_5 l’SR della doppia corona è *pari o superiore* a quello della singola. Secondo, **il guadagno di step non si assottiglia ma cresce**: da -18% (v_1) sale a un picco di $\sim -34\%$ (v_3 – v_4), per poi ridursi a -25% a v_5 per effetto pavimento (la singola è già a ~ 28 step, vicino al minimo geometrico).

Sguardo ravvicinato a $v_{\max}=1$. A $v_{\max}=1$ — l’unico livello con un costo di SR — il residuo $-1,3$ è concentrato in **V2** ($89,7 \rightarrow 86,2$) e **Q3/4** ($96,3 \rightarrow 90,4$), mentre su Q1/2 la doppia corona è in vantaggio ($+2,1$). Appaiando i 1560 episodi per (sorgente, vento, chunk, episodio) si isolano i casi in cui le due configurazioni divergono (Tabella 5.9).

Esito appaiato	N. episodi
Entrambi successo	1463
Entrambi fallimento	13
Singola OK, doppia FALLISCE	52
Singola fallisce, doppia OK	32
Netto	-20 ($= -1,3\%$)

Tabella 5.9: Matrice degli esiti appaiati singola vs. doppia corona a $v_{\max}=1$.

I 52 fallimenti differenziali hanno una firma nettissima (Figura 5.9): sono **tutti timeout** (nessuna uscita dal dominio), l’81% è su **vento V2** ($42/52$) nei chunk a corrente sostenuta, avvengono su **spawn lontani** (distanza iniziale media 1070 m contro 690 m della media generale) e **non sono near-miss**: $36/52$ non scendono mai sotto 150 m dalla sorgente (distanza minima mediana 322 m). L’agente, in altre parole, **si perde**; non “non riesce a frenare”. Il crollo è tutto nel regime “lontano + corrente forte”: in V2 la SR per fascia di distanza iniziale passa da $+1,1$ (300–700 m) a $-15,2$ (1100–1600 m) fino a $-83,3$ (oltre 1600 m).

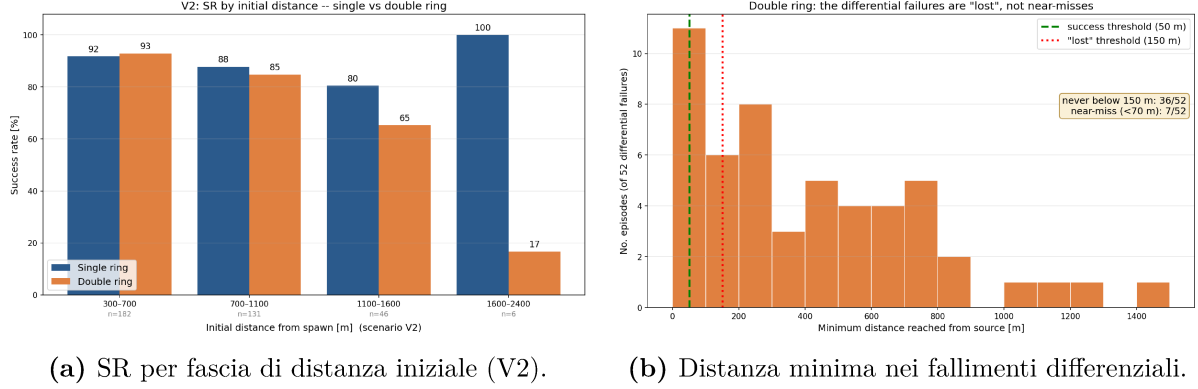


Figura 5.9: Firma del costo di affidabilità della doppia corona a $v_{\max}=1$ nello scenario V2. A sinistra: sugli spawn vicini la doppia corona è pari o migliore, ma sulle fasce lontane (>1100 m) la SR crolla rispetto alla singola. A destra: i fallimenti differenziali non sono *near-miss* — la distanza minima dalla sorgente resta elevata, l’agente “si perde” anziché non riuscire a frenare.

Vanno però letti anche i **32 casi opposti**, in cui la singola fallisce e la doppia riesce. Anch’essi sono tutti timeout della singola e concentrati in V2 (28/32), ma con firma *complementare*: il chunk dominante è **Q1/2** (25/32) anziché Q3/4, e gli spawn sono a distanza **moderata** (media 749 m contro 1070 m). La doppia corona non è dunque uniformemente peggiore: **redistribuisce** i successi. Sui chunk Q1/2 il segnale a 50 m aiuta a tenere la rotta ed è in attivo (25 recuperi contro 14 perdite); su Q3/4, con spawn più lontani e corrente più sostenuta, lo stesso segnale inganna ed è in passivo (7 recuperi contro 38 perdite). Il bilancio netto, -20 episodi, è uno **spostamento della frontiera di successo**, non una regressione generalizzata.

Interpretazione. Il filo che tiene insieme tutti i risultati è uno solo: la corona-2 sente il campo **più lontano** (50 m), e il valore di questa “vista lontana” dipende da *quanto lontano l’agente sta per muoversi*, cioè dalla lunghezza del passo. A $v_{\max}$ alto le falcate sono grandi (fino a 50 m) e una falcata mal puntata scavalca la rotta, costando correzioni; la corona-2, vedendo esattamente “un passo avanti”, permette di puntarle bene ed evita gli *overshoot* che rallentano la singola — da cui il guadagno crescente con $v_{\max}$. Il costo di affidabilità a $v_{\max}=1$ è lo stesso meccanismo ribaltato: a passi piccoli l’agente resta più a lungo immerso nel campo e in V2, con plume fortemente avvettato, il segnale a 50 m può puntare *lungo la corrente* anziché verso la sorgente; sugli spawn lontani l’agente si fida del segnale distante, sbaglia direzione e va in timeout. Alle velocità superiori attraversa quel regime troppo in fretta perché l’errore si accumuli, e il costo sparisce.

Vale infine la pena chiedersi perché l’agente non *ignori* semplicemente la corona-2 quando non conviene. Ignorarla equivarrebbe a pesi d’ingresso esattamente nulli, una soluzione che la discesa del gradiente non raggiunge: anche partendo da zero, come nel warm-start, i gradienti spostano quei pesi. Soprattutto, usarla *aumenta* il reward — lo **step_penalty** premia l’arrivare prima e la corona-2 dà traiettorie più dirette — quindi l’ottimizzatore **sceglie** di usarla. In sintesi: la seconda corona è un **vantaggio netto a partire da** $v_{\max}=2$ (stessa affidabilità, 25–34% di passi in meno), e solo a $v_{\max}=1$ si osserva un piccolo scambio robustezza/velocità. La conclusione del primo confronto — “per questo dominio la singola corona è sufficiente” — era un artefatto dell’aver addestrato da zero e dell’aver guardato un solo livello di velocità.

5.5.5 Test di modulazione: raggio di successo a 10 m

I risultati della catena a velocità variabile mostrano che, in media, l’agente resta prossimo a $v_{\max}$. Sorge quindi una domanda: l’agente *sa* davvero rallentare, o gli conviene sempre andare al massimo? Con un raggio di successo di 50 m la domanda è mal posta, perché il raggio è grande quanto un passo pieno (50 m a $v_{\max}=5$): l’agente può **piombare** sulla sorgente a piena velocità e centrare comunque il bersaglio, senza alcun bisogno di rallentare.

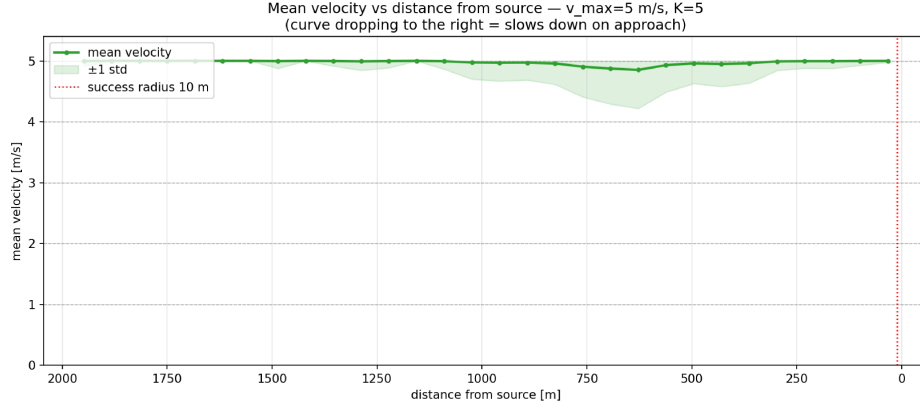
Per rendere il compito più stringente si **restringe il raggio di successo a 10 m**. Ora un passo pieno da 50 m **scavalca** un bersaglio da 10 m, e centrarlo non è più banale: per riuscirci l’agente impara a *rallentare* in avvicinamento, oppure adotta un’altra strategia? Il modello è ottenuto per fine-tuning a partire dallo stadio $v_{\max}=5$, cambiando *solo* il raggio di successo (50 $\rightarrow$ 10 m), con soglia di stagnazione ridotta, $lr = 10^{-4}$ e curriculum $\alpha = 0,8$, per 2 M timestep; la valutazione è su 1560 episodi, identica agli altri stadi.

Raggio	SR%	Step (min)	V0	V1	V2	V3	Q1/4	Q1/2	Q3/4
$v_{\max}=5$, 50 m	99,9	27,8 (4,6)	100,0	100,0	99,7	100,0	100,0	100,0	99,8
$v_{\max}=5$, 10 m	99,2 -0,7	54,8 (9,1) +27,0	100,0 0,0	98,5 -1,5	98,5 -1,2	100,0 0,0	98,7 -1,3	99,4 -0,6	99,6 -0,2

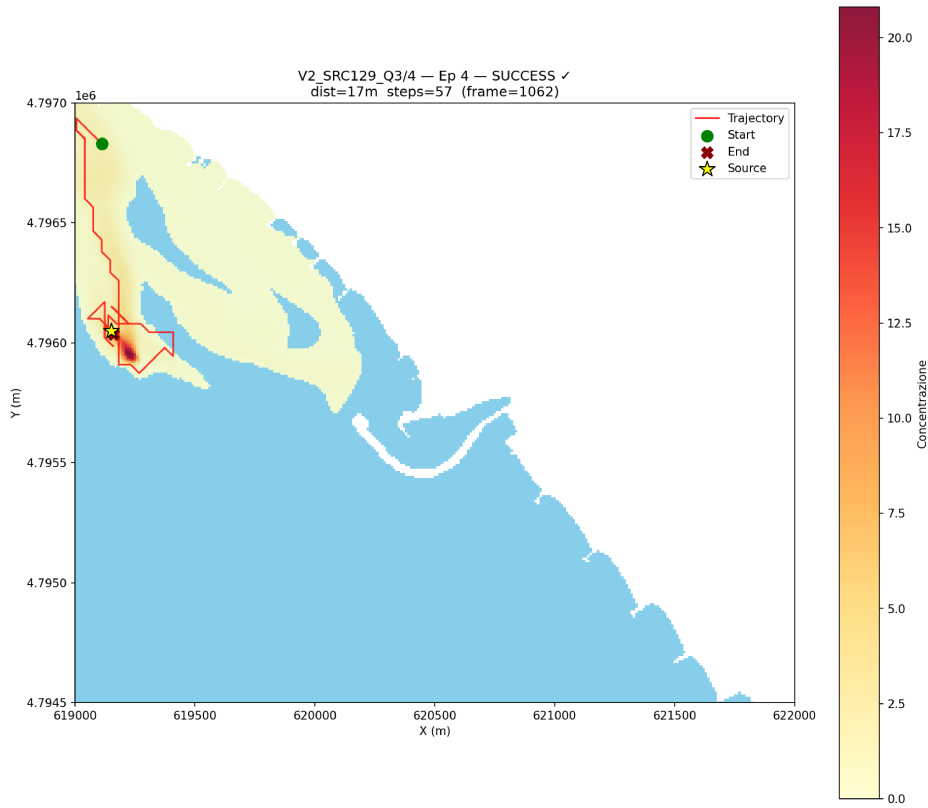
Tabella 5.10: Effetto del restringimento del raggio di successo ($50 \rightarrow 10$ m) sullo stadio $v_{\max}=5$ (1560 episodi). Sotto ogni valore a 10 m, il delta rispetto alla configurazione a 50 m (in rosso i cali di SR; in grigio l'aumento — atteso — degli step).

Con un bersaglio cinque volte più piccolo l'agente continua a raggiungerlo quasi sempre (99,2%), ma **impiega circa il doppio degli step** ($27,8 \rightarrow 54,8$, ovvero $4,6 \rightarrow 9,1$ minuti). Il dato interessante è *dove* va a finire questo tempo extra: **non** in un rallentamento. La velocità media resta infatti prossima al massimo (4,98 m/s), praticamente identica alla configurazione a 50 m.

Il motivo è nel criterio di cattura: il successo è valutato sul **segmento percorso** (posizione precedente $\rightarrow$ corrente), non solo sull'endpoint. Un passo pieno da 50 m la cui traiettoria passa entro 10 m dalla sorgente conta già come successo; l'agente quindi **non è obbligato a frenare** — gli basta indirizzare una falcata che sfiori il bersaglio. Centrare un target da 10 m con passi da 50 m richiede però più manovre: l'agente **scavalca e ri-approccia** ripetutamente attorno alla sorgente, accumulando step a velocità quasi piena. Il costo del bersaglio più piccolo si paga dunque in **lunghezza del percorso**, non in velocità: la politica sa *manovrare* finemente attorno a un target stretto, più di quanto sappia rallentare sistematicamente.



(a) Velocità media in funzione della distanza dalla sorgente (asse invertito: lontano a sinistra, vicino a destra; linea rossa = raggio di successo).



(b) Traiettoria con manovra attorno alla sorgente: scavalcamenti e ri-approcci (57 step).

Figura 5.10: Esperimento a raggio 10 m. La velocità media resta prossima al massimo a ogni distanza dalla sorgente (a): l'agente non frena. Scende invece lungo il plume a piena velocità e **manovra ripetutamente** attorno alla sorgente (b) per far passare una falcata entro i 10 m.

5.5.6 Heatmap di occupazione dell'agente

Per un riscontro *visivo* dei risultati numerici si riportano alcune **heatmap di occupazione**: dato uno scenario (sorgente, vento, chunk) si eseguono 10 episodi e si accumula

la densità di presenza dell'agente sul dominio, su sfondo geografico (mare, costa, sorgente $\star$). Si confrontano tre agenti alla **stessa velocità massima**: FCM (5 m/s), PPO a singola corona ($v_{\max}=5$) e PPO a doppia corona ($v_{\max}=5$), su scenari *difficili* (vento V1/V2, chunk Q1/2 o Q3/4). La scala colori è **comune ai tre pannelli**, fissata sui due modelli PPO; l'FCM, che oscilla a lungo negli stessi punti, ha densità di picco molto più alte e quindi **satura**.

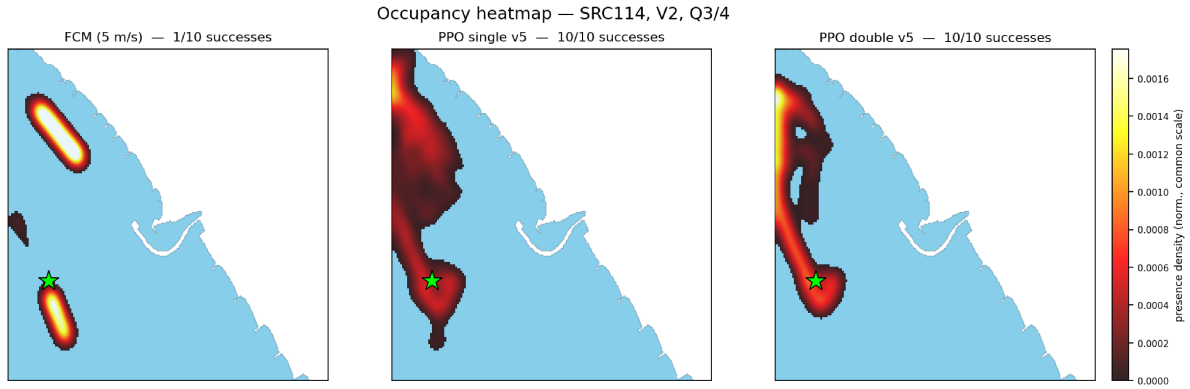


Figura 5.11: Heatmap di occupazione — SRC114, V2, Q3/4. FCM (sinistra) vs. PPO singola corona (centro) vs. PPO doppia corona (destra), $v_{\max}=5$, scala comune.

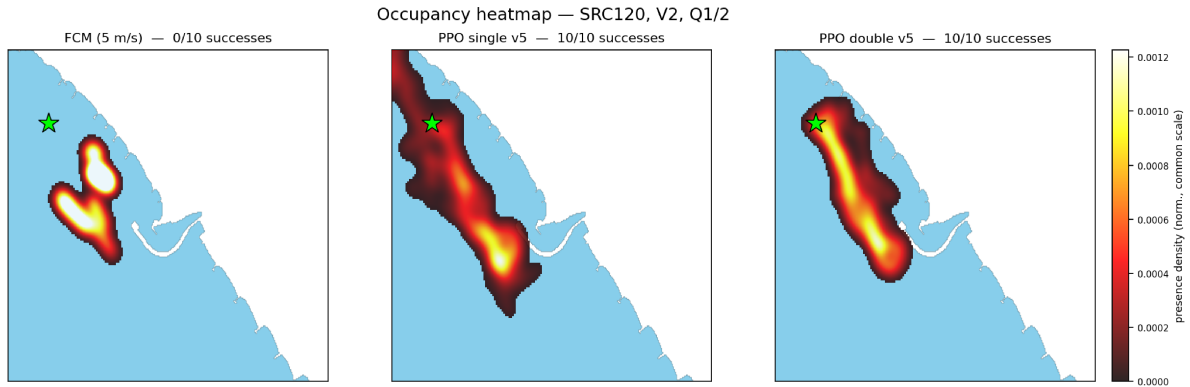


Figura 5.12: Heatmap di occupazione — SRC120, V2, Q1/2. Stessa disposizione e stessa scala della figura precedente.

Se ne leggono due fatti. Nel confronto **FCM vs. PPO**, l'FCM resta **intrappolato** in macchie oscillanti lontane dalla sorgente e quasi sempre fallisce, mentre il PPO forma una scia che **converge sulla sorgente**: è la controparte visiva del divario di success rate (Sezione 5.5.8) — la risalita di gradiente *locale*, senza memoria né contesto, si perde nel plume avvevato. Nel confronto **singola vs. doppia corona**, entrambe raggiungono la sorgente, ma la doppia concentra la presenza in un **corridoio più stretto e definito**

(più luminoso a parità di scala) mentre la singola è più dispersa: è la controparte visiva del -34% di step della Tabella 5.8. Va letta con un'avvertenza: la luminosità indica la *densità di passaggio*, non la velocità — un tratto più luminoso vicino alla sorgente riflette la convergenza di tutte le run in un corridoio comune, non un rallentamento (la velocità resta $\sim v_{\max}$).

5.5.7 Analisi delle spawn map

Per indagare se l'esito di un episodio dipenda dalla posizione iniziale del master, sono state generate le **spawn map** per le configurazioni vento-chunk con success rate inferiore al 100%. Per ciascuna si selezionano le 5 sorgenti con il maggior numero di fallimenti; per ogni terna (sorgente, v , q) si eseguono 10 episodi e si registrano le coordinate di spawn, contrassegnate con un **cerchio verde** (successo) o una **croce rossa** (fallimento), sovrapposte al plume di concentrazione della specifica sorgente all'istante Q^* .

Spawn map SRC114

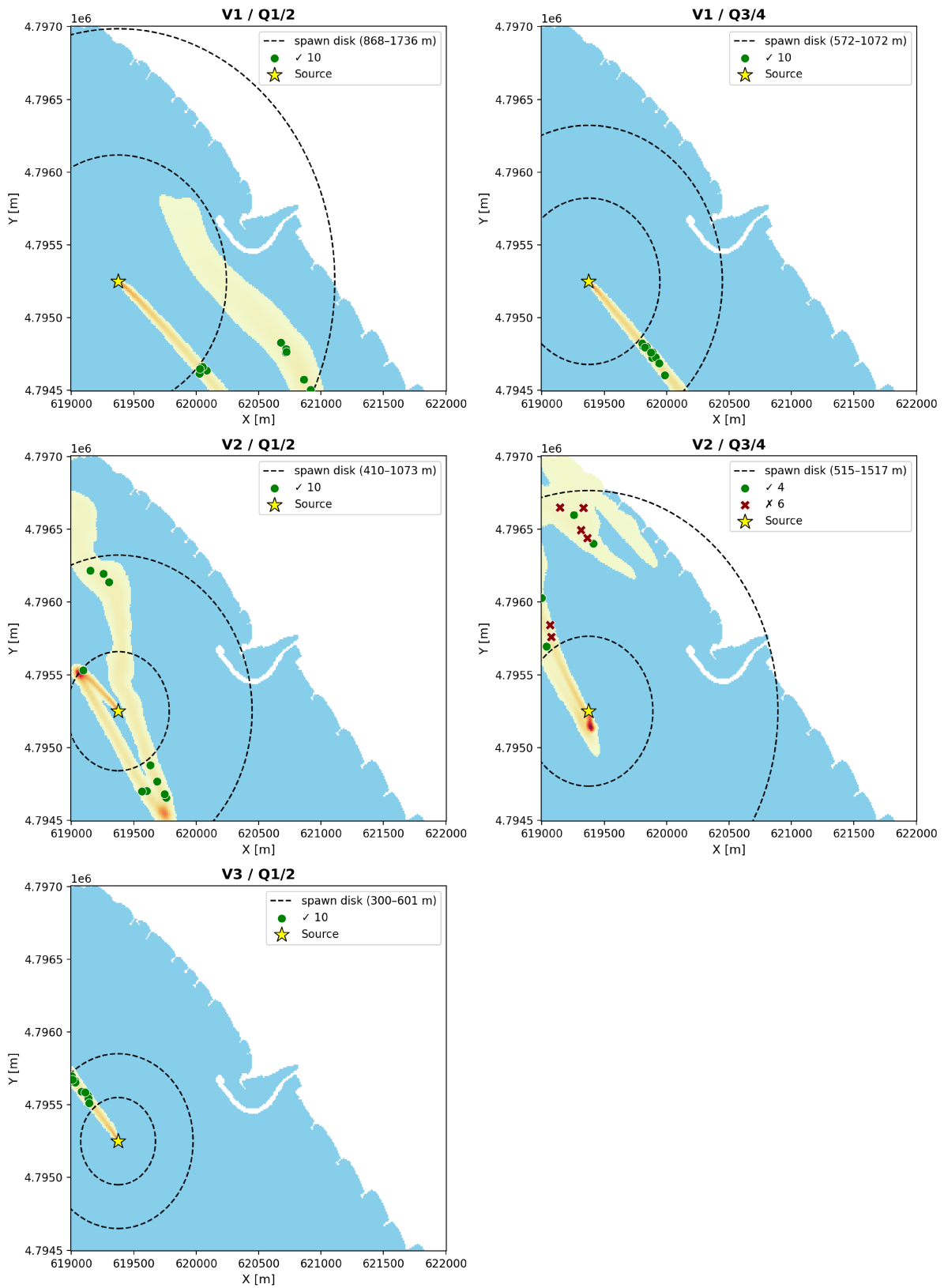


Figura 5.13: Spawn map — SRC114. Cerchi verdi (successo) e croci rosse (fallimento) risultano spazialmente intrecciati: non emerge alcuna regione di partenza sistematicamente sfavorevole.

Spawn map SRC119

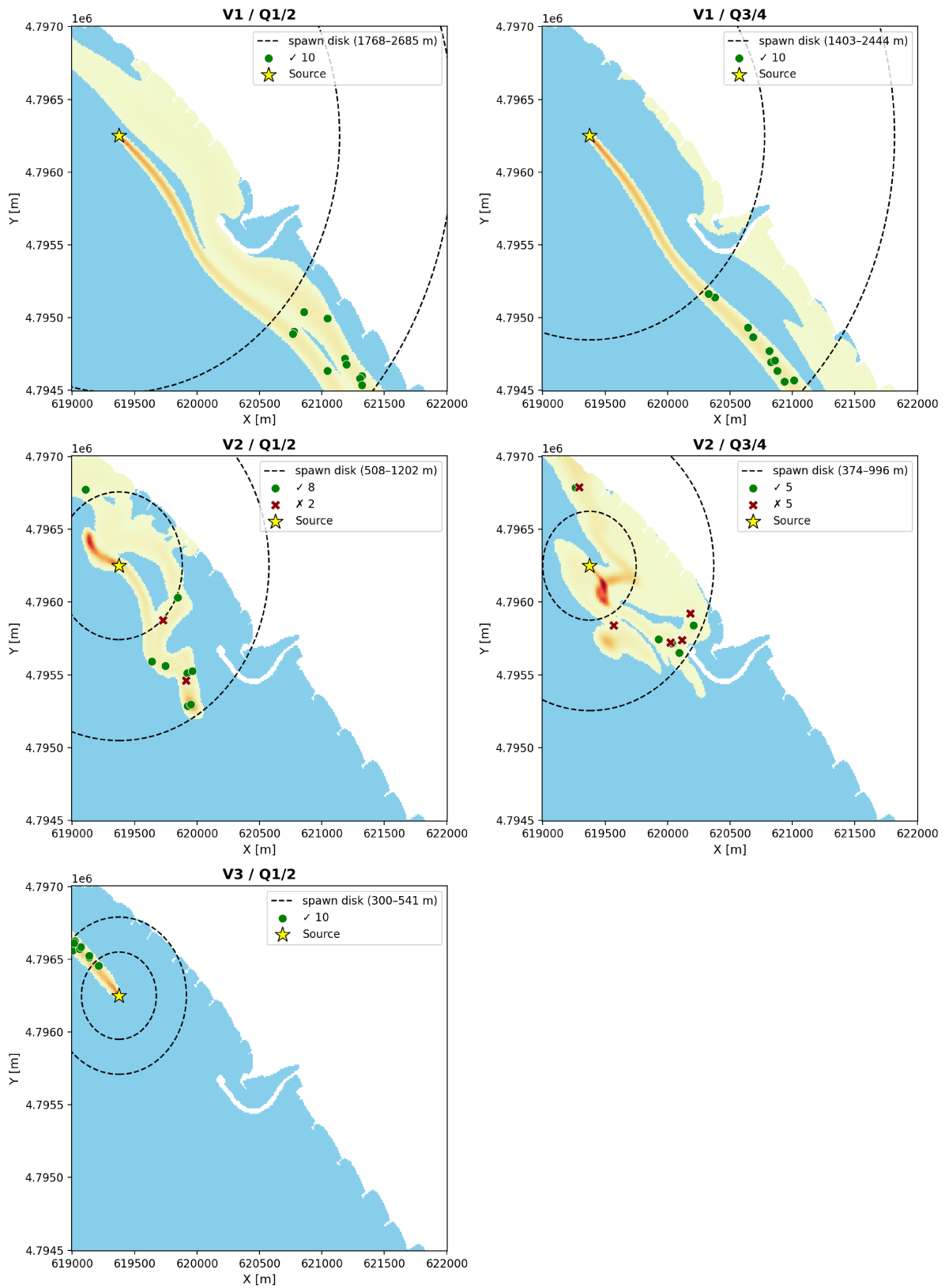


Figura 5.14: Spawn map — SRC119. Anche qui successi e fallimenti si alternano nelle stesse zone del plume: l'esito non è separabile in base alla posizione di spawn.

Spawn map SRC132

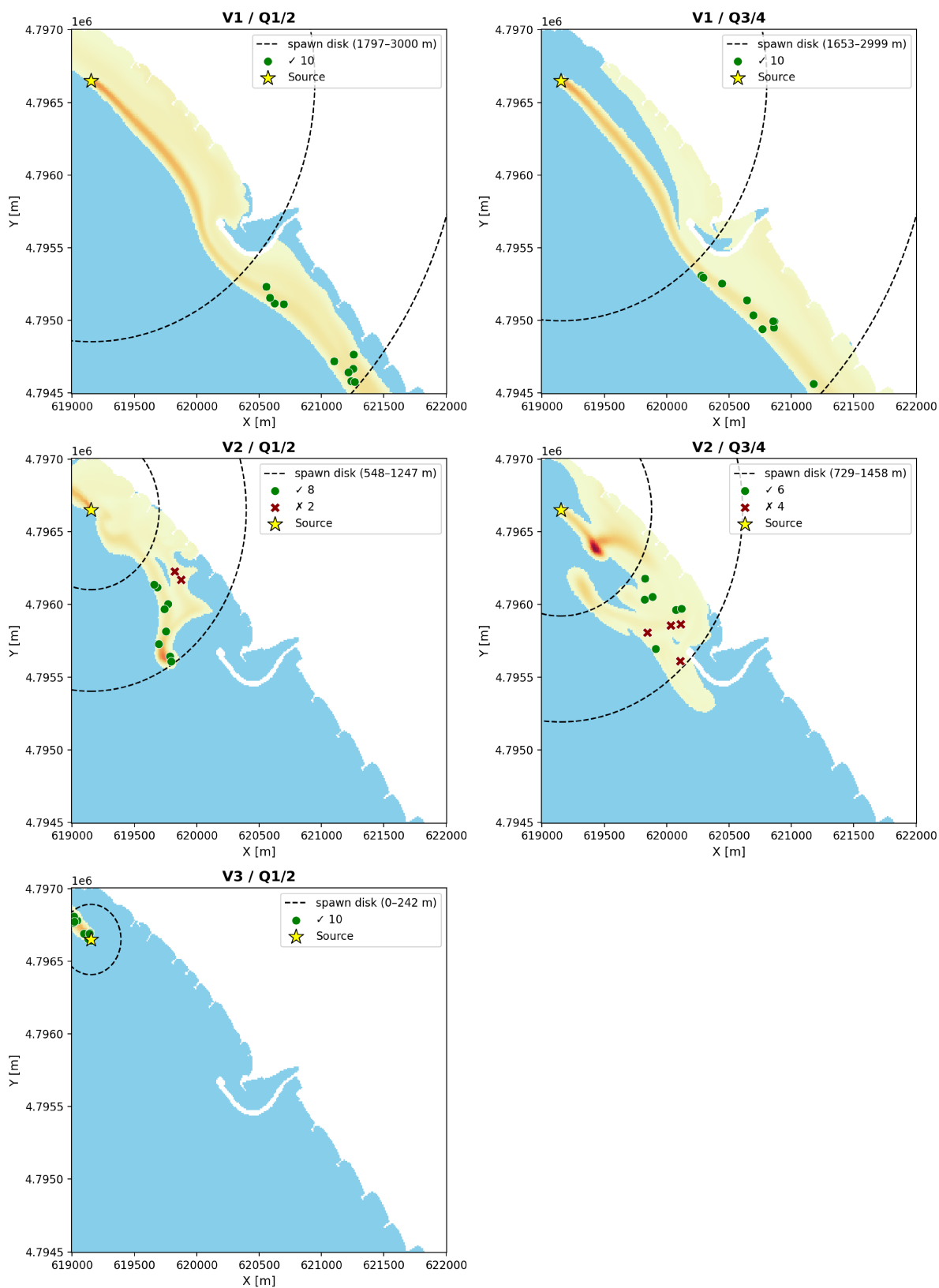


Figura 5.15: Spawn map — SRC132. La sovrapposizione di esiti opposti dagli stessi punti di partenza conferma che il fallimento dipende dalle condizioni idrodinamiche del chunk, non dalla geometria dello spawn.

In tutte le sorgenti analizzate, i punti di partenza che portano al successo e quelli che portano al fallimento **non mostrano alcuna separazione spaziale**: non emergono zone di spawn sistematicamente “difficili” o “facili”. L’esito di un episodio non dipende dunque dalla posizione iniziale del master, bensì dalle condizioni idrodinamiche locali (intensità e direzione del plume, turbolenza) proprie di quel chunk temporale e di quello scenario di vento. I fallimenti residui del modello sono da attribuire a caratteristiche intrinseche degli scenari più difficili, non a limitazioni geometriche dello spazio di spawn.

5.5.8 Confronto con FCM puro e adattivo

La Tabella 5.11 riassume il confronto tra le due varianti del metodo del gradiente (Capitolo 4) e il modello RL di riferimento (`minimal`), valutati sulle stesse sorgenti di test.

Metrica	FCM puro	FCM adattivo	RL (<code>minimal</code>)
SR globale	32,9%	63,5%	96,9%
V0	39,7%	84,6%	100,0%
V1	16,0%	35,3%	99,0%
V2	23,7%	41,7%	88,7%
V3	51,9%	92,3%	99,7%
Q1/4	50,5%	90,9%	100,0%
Q1/2	11,5%	44,2%	93,5%
Q3/4	36,5%	55,3%	97,1%

Tabella 5.11: Confronto diretto tra FCM puro ($r_s=50$ m), FCM con passo adattivo nella configurazione migliore ($lr=40$ m, Tabella 4.2) e modello RL di riferimento `minimal`. Il FCM adattivo adotta il disco adattivo di spawn, mentre il FCM puro e l’RL `minimal` usano la cascata (Sezione 3.1.4); il confronto RL vs. FCM a parità di spawn e velocità è nella Sezione 5.5.9.

Il modello RL supera nettamente entrambe le varianti del metodo del gradiente su ogni scenario: +64 punti percentuali sul success rate globale rispetto al FCM puro e +33 rispetto al FCM adattivo. Il divario è massimo proprio negli scenari a plume diffuso (V1, V2, Q1/2), dove il gradiente locale degenera e i metodi reattivi perdono la direzione.

Il confronto della Tabella 5.11 è per giunta *sfavorevole* all’RL sul piano cinematico: il modello **minimal** si muove a passo fisso di 10 m per timestep (1 m/s), mentre l’FCM adattivo a $lr = 40$ m compie passi fino a quattro volte più lunghi (4 m/s). L’RL vince dunque pur essendo quattro volte più lento. Poiché entrambi i metodi dispongono ora di una versione a velocità variabile — la catena della Sezione 5.5.3 per il PPO, lo sweep su lr per l’FCM — è possibile un confronto **a parità di velocità massima**, che neutralizza questo fattore: un passo FCM di lr metri per timestep corrisponde a $lr/10$ m/s, e si accoppia quindi con lo stadio PPO di pari $v_{\max}$ (Tabella 5.12).

Vel. / metodo	SR%	V0	V1	V2	V3	Q1/4	Q1/2	Q3/4
FCM ($lr=10$)	54,6	73,7	29,5	35,9	79,5	77,9	30,3	55,8
PPO ($v_{\max}=1$)	97,1 +42,5	99,5 +25,8	100,0 +70,5	89,7 +53,8	99,2 +19,7	100,0 +22,1	95,0 +64,7	96,3 +40,6
FCM ($lr=20$)	58,2	80,1	32,1	34,6	85,9	81,7	38,0	54,8
PPO ($v_{\max}=2$)	99,7 +41,5	100,0 +19,9	99,5 +67,4	99,2 +64,6	100,0 +14,1	100,0 +18,3	99,0 +61,1	100,0 +45,2
FCM ($lr=30$)	60,7	84,6	30,8	37,2	90,4	86,1	39,4	56,7
PPO ($v_{\max}=3$)	99,9 +39,2	100,0 +15,4	100,0 +69,2	99,7 +62,6	100,0 +9,6	100,0 +13,9	99,8 +60,4	100,0 +43,3
FCM ($lr=40$)	63,5	84,6	35,3	41,7	92,3	90,9	44,2	55,3
PPO ($v_{\max}=4$)	99,9 +36,4	100,0 +15,4	100,0 +64,7	99,5 +57,8	100,0 +7,7	100,0 +9,1	100,0 +55,8	99,6 +44,3
FCM ($lr=50$)	62,2	84,6	34,6	37,2	92,3	89,4	41,8	55,3
PPO ($v_{\max}=5$)	99,9 +37,8	100,0 +15,4	100,0 +65,4	99,7 +62,6	100,0 +7,7	100,0 +10,6	100,0 +58,2	99,8 +44,5

Tabella 5.12: FCM adattivo vs. PPO adattivo **a parità di velocità massima** (FCM: $lr/\Delta t$; PPO: $v_{\max}$), per scenario di vento (V) e chunk temporale (Q). Sotto ogni valore PPO, in verde, il surplus sul FCM in punti percentuali.

Neutralizzato il fattore cinematico, il quadro non cambia: a **ogni** livello di velocità il PPO supera l’FCM di 36–42 punti di success rate globale, e il divario resta massimo negli scenari avversi (V1, V2, Q1/2), dove tocca i 55–70 punti. Il vantaggio dell’RL non è dunque un artefatto della velocità: è strutturale. L’RL ha appreso strategie non

riducibili al gradiente locale — sfruttamento della memoria storica, navigazione in zone a bassa concentrazione, recupero da situazioni di stallo — che emergono dall’ottimizzazione su milioni di episodi e non sono accessibili a un metodo che decide passo-passo senza contesto.

5.5.9 Estensione all’intero range di velocità

Le analisi precedenti coprono i valori interi $v_{\max} \in \{1, \dots, 5\}$ m/s. Per completare il quadro sull’*intero* intervallo operativo del veicolo, la catena è stata estesa sia verso il basso, con le **velocità ridotte** $v_{\max} \in \{0,1, 0,4, 0,7\}$, sia negli **stadi intermedi** $v_{\max} \in \{1,2, 1,5, 1,7, 1,9\}$, per entrambe le formazioni (singola e doppia corona). Ogni stadio è ottenuto per fine-tuning a catena dal modello adiacente già addestrato — ascendente per gli intermedi, discendente per i valori bassi — e valutato sui consueti 1560 episodi held-out. In parallelo, il metodo del gradiente adattivo è stato valutato al learning rate corrispondente ($lr = 10 v_{\max}$, cfr. Sezione 4.3) su tutte le stesse velocità, così da estendere il confronto *a parità di velocità massima* lungo l’intero range. Le Figure 5.16–5.17 riportano l’andamento di success rate e step medi al successo; la Tabella 5.14 ne raccoglie i valori.

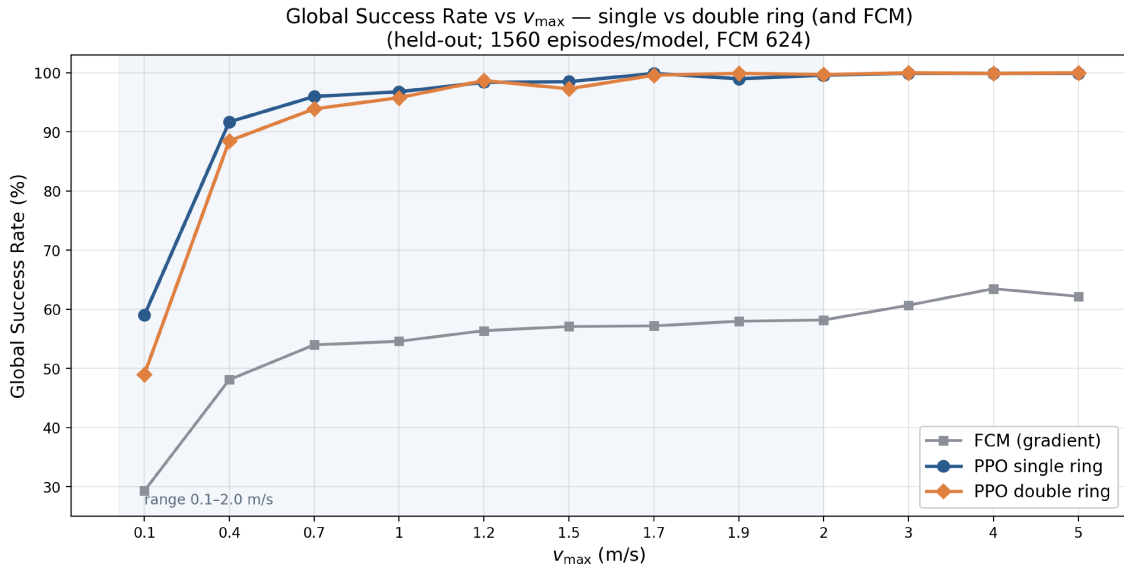


Figura 5.16: Success rate globale in funzione di $v_{\max}$ sull’intero range (0,1–5 m/s): PPO singola corona, PPO doppia corona e FCM adattivo (held-out; 1560 episodi per modello, 624 per l’FCM). Le due curve PPO saturano già a $v_{\max} \gtrsim 1,2$, mentre l’FCM resta confinato attorno al 55–63% alle velocità operative, con un ulteriore calo (fino al $\sim 29\%$) a quelle più basse.

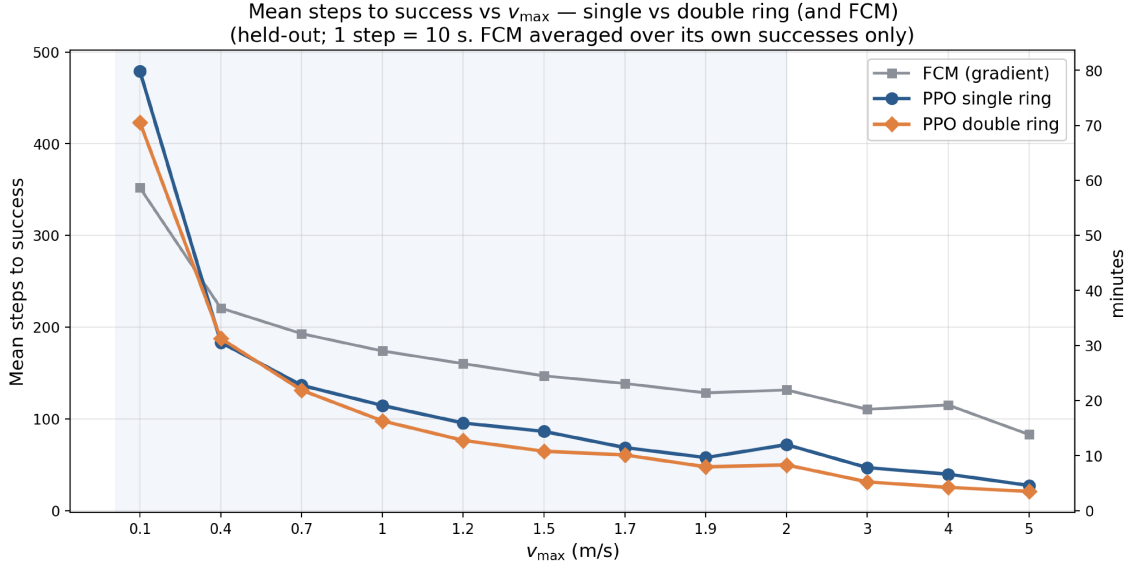


Figura 5.17: Step medi al successo in funzione di $v_{\max}$ (asse secondario in minuti, 1 step = 10 s). Dalla velocità nominale (1 m/s) in su la doppia corona raggiunge la sorgente in meno passi della singola; alle velocità più basse i due profili sono sostanzialmente sovrapposti. Il tempo al successo cala in modo netto all’aumentare di $v_{\max}$.

La Tabella 5.13 dettaglia il confronto *singola vs. doppia corona* a ciascuna velocità, riportando per entrambe success rate e passi al successo con le rispettive differenze.

$v_{\max}$	SR singola	SR doppia	Δ SR	step singola	step doppia	Δ step
0,1	59,0	49,0	−10,0	478,9	423,2	−11,6%
0,4	91,7	88,5	−3,2	183,4	187,7	+2,3%
0,7	96,0	93,9	−2,1	136,6	131,3	−3,9%
1,0	96,8	95,8	−1,0	114,6	97,8	−14,7%
1,2	98,4	98,7	+0,3	95,5	76,5	−19,9%
1,5	98,5	97,3	−1,2	86,3	64,7	−25,0%
1,7	99,9	99,6	−0,3	68,7	60,7	−11,6%
1,9	99,0	99,9	+0,9	57,8	47,7	−17,5%
2,0	99,6	99,7	+0,1	71,9	49,8	−30,7%
3,0	99,9	100,0	+0,1	46,8	31,3	−33,1%
4,0	99,9	99,9	0,0	39,7	25,3	−36,3%
5,0	99,9	100,0	+0,1	27,4	20,9	−23,7%

Tabella 5.13: Singola vs. doppia corona di sensori a ciascun $v_{\max}$ (held-out, 1560 episodi per modello). “step” = passi medi al successo (1 step = 10 s); Δ SR = doppia – singola (punti percentuali); Δ step = variazione relativa dei passi (verde = doppia più veloce/migliore, rosso = doppia peggiore). I valori provengono dalla campagna di valutazione unificata sull’intero range e possono differire di poco da quelli della Tabella 5.8.

$v_{\max}$	SR singola	SR doppia	SR FCM	PPO – FCM
0,1	59,0	49,0	29,3	+29,7
0,4	91,7	88,5	48,1	+43,6
0,7	96,0	93,9	54,0	+42,0
1,0	96,8	95,8	54,6	+42,2
1,2	98,4	98,7	56,4	+42,3
1,5	98,5	97,3	57,1	+41,4
1,7	99,9	99,6	57,2	+42,7
1,9	99,0	99,9	58,0	+41,9
2,0	99,6	99,7	58,2	+41,5
3,0	99,9	100,0	60,7	+39,3
4,0	99,9	99,9	63,5	+36,4
5,0	99,9	100,0	62,2	+37,8

Tabella 5.14: Success rate globale sull'intero range di velocità: PPO singola corona, PPO doppia corona e FCM adattivo (held-out). L'ultima colonna riporta il surplus della migliore configurazione PPO sull'FCM di pari velocità. I valori ai punti interi possono differire di meno di un punto da quelli delle Tabelle 5.6 e 5.8, in quanto ottenuti da una campagna di valutazione unificata sull'intero range.

Quattro fatti emergono con nettezza. Primo, la **saturatione precoce** del PPO: già a $v_{\max} = 1,2 \text{ m/s}$ entrambe le formazioni superano il 98% e si stabilizzano attorno al 99% fino a 5 m/s; il margine di velocità aggiuntivo si traduce in tempi al successo più brevi (Figura 5.17), non in maggiore affidabilità. Secondo, il **plateau dell'FCM**: il metodo del gradiente non supera il 64%, e la sua curva cresce lentamente e in modo quasi lineare con la velocità, senza mai avvicinarsi al PPO — conferma che il limite dell'FCM è *strutturale* (la natura reattiva e locale), non cinematico. Terzo, il **crollo condiviso alle velocità molto basse**: a $v_{\max} = 0,1 \text{ m/s}$ anche il PPO scende al 59%, perché a 0,1 m/s il veicolo è più lento delle correnti che deve risalire e nessuna strategia di navigazione può compensare un deficit puramente fisico di propulsione; è l'unico regime in cui il divario

PPO–FCM si riduce (a +29,7), pur restando ampiamente a favore del PPO. Su tutto il resto del range il surplus del PPO si mantiene fra +36 e +44 punti percentuali, ribadendo che il vantaggio dell'apprendimento è sistematico e indipendente dalla velocità operativa scelta. Quarto, il confronto **singola vs. doppia corona** (Tabella 5.13): la seconda corona a 50 m non è sempre conveniente, ma il suo valore dipende dalla velocità, con uno spartiacque attorno a $v_{\max} \approx 1$ m/s. *Sotto* tale soglia la doppia corona **costa** success rate (fino a -10 punti a 0,1 m/s): con passi piccoli l'agente resta a lungo immerso nel campo e, negli scenari avversi, il segnale lontano può puntare lungo la corrente anziché verso la sorgente, senza lasciare margine per correggere. *Da* $v_{\max} \geq 1,2$ m/s, invece, la doppia corona **mantiene la stessa affidabilità** (differenze di SR tra -1,2 e +0,9 punti, rumore vicino alla saturazione) **riducendo però i passi al successo del 12–37%**: a falcate lunghe la “vista lontana” aiuta a mirare meglio la direzione ed evitare gli overshoot. Il secondo anello è dunque un vantaggio netto solo dalle velocità intermedie in su.

Capitolo 6

Explainability della politica appresa

6.1 Motivazione

Il Capitolo 5 stabilisce *che* la politica funziona: sul protocollo held-out il modello a doppia corona con $v_{\max} = 2 \text{ m/s}$ risolve il 99,7% degli episodi. Non dice però *perché* funzioni, né su quale parte delle 196 dimensioni dell'osservazione si regga. È una lacuna che conta in un contesto operativo: sapere quali grandezze la politica sfrutta davvero significa sapere quali sensori sono indispensabili a bordo del veicolo, quali sono ridondanti e quali degradazioni sensoristiche la missione tollera.

Il capitolo separa due domande che la letteratura tende a confondere:

- **Che cosa la rete guarda** — quali blocchi dell'osservazione influenzano l'uscita della politica. È una domanda *osservazionale* sulla funzione appresa, e si risponde con l'attribuzione (Sezione 6.5).
- **Che cosa serve al compito** — quali blocchi, se rimossi, degradano la prestazione. È una domanda *causale* sul task, e si risponde con un intervento (Sezione 6.4).

Le due domande non hanno la stessa risposta, e il punto interessante è proprio dove divergono: una feature può avere attribuzione alta e impatto nullo perché ridondante rispetto ad altre, oppure attribuzione modesta ed effetto grande perché non sostituibile da nessun'altra. Il capitolo si chiude (Sezione 6.8) mettendo a confronto le due letture.

6.2 Metodi post-hoc: rassegna e criterio di scelta

Lo strumentario standard dell'explainability post-hoc model-agnostic è essenzialmente duplice. **LIME** [17] approssima il modello nell'intorno di una singola predizione con un surrogato lineare sparso, stimato su perturbazioni del punto pesate da un kernel di prossimità. **SHAP** [18] riconduce invece un'intera famiglia di metodi al *valore di Shapley* [19] della teoria dei giochi cooperativi: le feature sono i giocatori, la predizione è il payoff da ripartire, e il valore di Shapley è l'unica ripartizione che soddisfa gli assiomi di efficienza, simmetria, dummy e additività. Accanto a questi si colloca la famiglia delle attribuzioni basate sul gradiente, di cui gli *Integrated Gradients* [20] sono il rappresentante assiomatico.

È importante distinguere due livelli che il nome “SHAP” tende a sovrapporre: il valore di Shapley è il *concetto*, mentre KernelSHAP, DeepSHAP e TreeSHAP sono *stimatori* di quel valore, resi necessari dal fatto che il numero di coalizioni cresce come 2^n nel numero n di giocatori. La distinzione è decisiva per il seguito.

Nel contesto di questa tesi l'applicazione meccanica di questi strumenti sarebbe inadeguata, per tre ragioni.

Granularità e collinearità. L'osservazione non è un insieme di 196 grandezze indipendenti ma un insieme di *blocchi* fortemente collineari al loro interno: 72 valori sono la storia della corona interna, altri 72 quella della corona esterna. Chiedersi quanto valga la singola feature numero 87 — il sensore di nord-est di cinque passi fa — non ha una risposta sensata, perché essa è quasi determinata da una dozzina di altre. Il problema non è solo di leggibilità: in presenza di feature perfettamente correlate il valore di Shapley ripartisce il merito fra di esse in parti uguali, diluendo l'attribuzione di un blocco importante su decine di componenti. KernelSHAP aggiunge a ciò un'ipotesi di indipendenza fra le feature nel campionamento delle coalizioni, palesemente violata qui, con distorsioni documentate [21]. La risposta corretta non è abbandonare Shapley ma cambiarne i giocatori: è la costruzione nota come *groupShapley* [22], in cui i giocatori sono gruppi di feature semanticamente coerenti. I gruppi adottati sono definiti nella Sezione 6.3 e sono, non a caso, le stesse unità di cui si parla nel Capitolo 5: la seconda corona, il vento, la memoria degli spostamenti.

Esattezza. Con dodici gruppi le coalizioni sono $2^{12} = 4096$: un numero enumerabile in pochi secondi. Gli stimatori servono quando l’enumerazione è impraticabile; qui non lo è, e i valori di Shapley si calcolano in forma *esatta*. Si evita così sia l’errore di approssimazione sia la scelta arbitraria del numero di campioni, e l’assioma di efficienza diventa un test numerico di correttezza del calcolo anziché una proprietà asserita (Sezione 6.5).

La possibilità di intervenire. È la ragione più profonda. Nel supervised learning i metodi post-hoc sono necessari perché il mondo non è rieseguibile: il modello è dato, i dati sono dati, e l’importanza di una feature può solo essere *stimata* da un surrogato — da cui la *model reliance* [23] e le sue varianti a permutazione. Nel reinforcement learning con simulatore questa limitazione non sussiste. Bloccare un gruppo di feature e rieseguire la politica misurando il success rate effettivo non è un surrogato dell’importanza: è un *intervento*, cioè evidenza diretta. Inoltre la politica, a differenza di un classificatore, non è un fine ma un mezzo per un ritorno misurabile, il che rende disponibile una metrica di importanza definita sul compito e non sulla funzione di perdita — uno status che l’ablation ha nell’explainability per RL e non nel supervised [24]. Rinunciarvi per affidarsi soltanto a metodi osservazionali significherebbe lasciare inutilizzata la prova migliore a disposizione.

Perché non LIME. LIME è escluso come strumento principale per ragioni indipendenti. È *locale* per costruzione: spiega una singola decisione, mentre le domande di questo capitolo sono globali (il vento viene usato? la seconda corona serve?); rispondervi richiederebbe di aggregare centinaia di spiegazioni locali, ricostruendo in modo deteriorato un metodo globale. Le sue perturbazioni campionano inoltre le feature indipendentemente, il che con blocchi collineari produce punti lontani dalla varietà dei dati — la storia dei sensori è quasi deterministica dati i sensori correnti. Infine le spiegazioni LIME sono note per essere instabili rispetto alla larghezza del kernel e al seed di campionamento [25]: fondare su di esse un’affermazione del tipo “la feature X conta” è metodologicamente fragile. Resta a LIME una nicchia legittima, che questo capitolo non sfrutta ma che segnaliamo fra gli sviluppi possibili: lo studio di caso di una singola decisione critica, dove la domanda è effettivamente locale.

Il criterio adottato. Ne segue l'impianto del capitolo: valori di Shapley *esatti* sui gruppi per stabilire che cosa la politica guarda, ablation *causale* sugli stessi gruppi per stabilire che cosa serve al compito, e infine sonde comportamentali per caratterizzare la strategia appresa. I tre livelli sono complementari e vengono confrontati nella Sezione 6.8.

6.2.1 Due accortezze tecniche

Due dettagli dell'implementazione hanno conseguenze sostanziali sui risultati e vanno dichiarati.

Logit pre-maschera. MaskablePPO [8] pone a $-\infty$ i logit delle azioni che porterebbero l'agente a terra o fuori dominio (Sezione 5.3). Attribuzioni e sonde comportamentali sono calcolate sui logit *prima* del mascheramento: diversamente si misurerebbe la geometria della costa invece della politica. L'ablation, al contrario, gira con il mascheramento attivo, perché in quel caso l'oggetto di interesse è il compito reale, di cui il mascheramento è parte.

Spazio normalizzato. La rete non vede le grandezze fisiche ma la loro versione normalizzata da `VecNormalize`, che sottrae la media e divide per la deviazione standard stimate in addestramento. Ciò ha due implicazioni favorevoli. Primo, tutte le feature hanno varianza unitaria, quindi le magnitudini di attribuzione sono confrontabili fra blocchi senza ulteriori riscalature. Secondo, il vettore nullo nello spazio normalizzato corrisponde esattamente alla *media di addestramento*: è quindi una baseline naturale, con la semantica di “sensore che riporta il valore tipico”, e la stessa baseline serve sia per le coalizioni vuote dello Shapley sia per l'ablation. Fa eccezione la lettura dei sensori grezzi nelle sonde comportamentali, dove la normalizzazione per-feature altererebbe l'ordinamento e va quindi invertita.

6.3 I gruppi semantici dell'osservazione

L'analisi è condotta sul modello a **doppia corona con** $v_{\max} = 2 \text{ m/s}$ (Sezione 5.5.4), che ha osservazione a 196 dimensioni e spazio d'azione Discrete(8×5). Le 196 feature sono raggruppate nei dodici blocchi della Tabella 6.1, che ricalcano la struttura dell'os-

servazione (Tabella 5.1). La valutazione usa il protocollo held-out del Capitolo 5: 26 sorgenti mai viste, 4 scenari di vento, 3 chunk temporali.

Gruppo	Indici	Dim.
Concentrazione (istante)	[0]	1
Concentrazione (storia)	[1:10]	9
Spostamenti (storia)	[10:28]	18
Corona 20 m (istante)	[28:36]	8
Corona 50 m (istante)	[36:44]	8
Corona 20 m (storia)	[44:116]	72
Corona 50 m (storia)	[116:188]	72
Vento (u, v)	[188:190]	2
Corrente (u, v)	[190:192]	2
Vettore verso max conc.	[192:194]	2
Valore max conc.	[194]	1
Età ultimo contatto	[195]	1
Totale		196

Tabella 6.1: I dodici gruppi semantici in cui è partizionata l’osservazione a doppia corona. Sono i giocatori dello Shapley (Sezione 6.5) e le unità dell’ablation (Sezione 6.4).

6.4 Ablation causale di gruppo

Protocollo. Per ogni gruppo si azzerano le sue componenti nello spazio normalizzato — equivalente, come detto, a sostituirle con la media di addestramento — e si riesegue la politica misurando success rate, step al successo e distanza finale. Il disegno è **appaiato**: per ogni spawn si valutano la baseline e tutte e dodici le ablazioni a partire dallo stesso seme, quindi dallo stesso punto di partenza e dallo stesso campo. Senza questo accorgimento la varianza fra spawn — la distanza iniziale varia da 300 a oltre 2000 m — sommergerebbe differenze di pochi punti. Sono stati valutati **624 spawn appaiati** $\times$ 13 condizioni, per un totale di circa 8100 episodi; gli intervalli di confidenza al 95% sono

ottenuti per bootstrap ricampionando *gli spawn* (non gli episodi), così da preservare l'appaiamento. L'ablazione di un gruppo si riduce ad azzerarne gli indici nell'osservazione già normalizzata, ad ogni passo dell'episodio:

```
def run_episode_ablated(model, vec_env, ablate_idx, seed):
    obs = vec_env.reset()                # obs normalizzata (
        VecNormalize)
    obs[:, ablate_idx] = 0.0             # azzerare il gruppo ->
        media di training
    done = False
    while not done:
        action, _ = model.predict(obs, deterministic=True,
                                   action_masks=vec_env.env_method("
                                   action_masks")[0])
        obs, _, done, info = vec_env.step(action)
        obs[:, ablate_idx] = 0.0         # mantiene il blocco
            azzerato ad ogni step
    return info
```

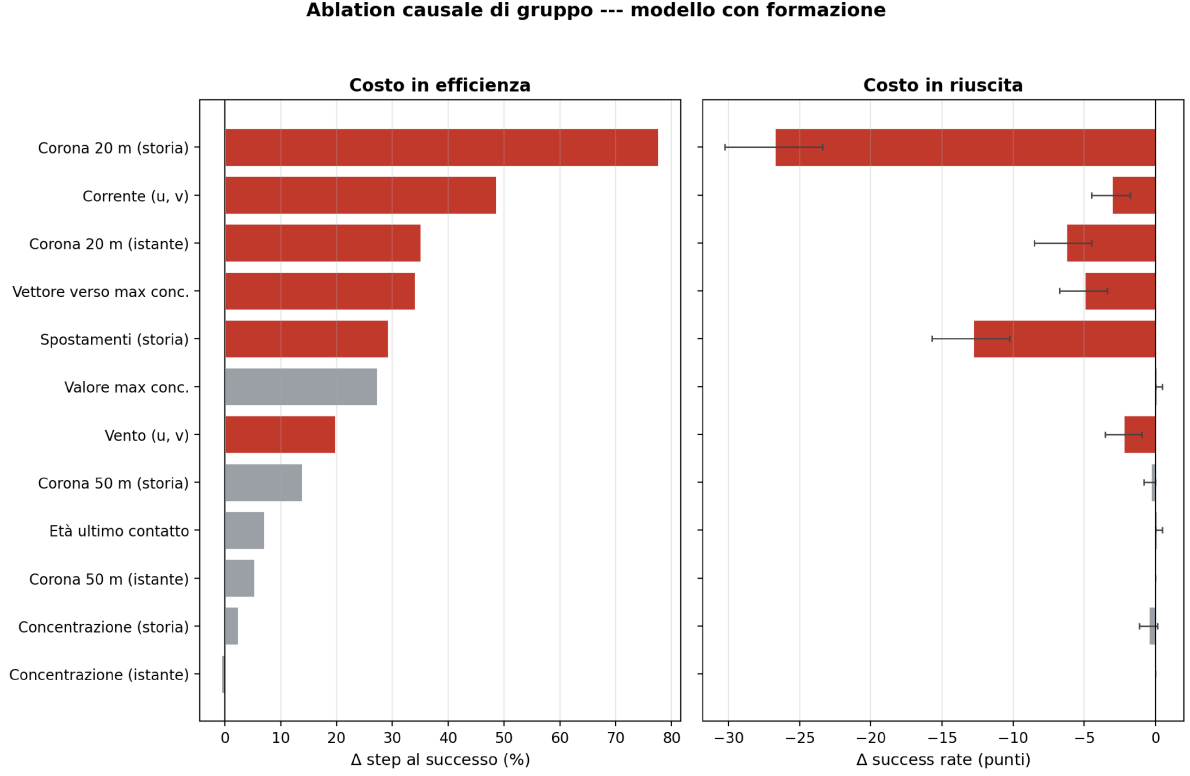


Figura 6.1: Ablation causale di gruppo sul modello a doppia corona $v_{\max}=2$: 624 spawn appaiati, IC 95% bootstrap. A sinistra il costo in efficienza (step al successo), a destra quello in riuscita (success rate). L'ordinamento segue l'efficienza. Il colore indica la *classe di evidenza*: rosso se l'intervallo esclude lo zero, grigio se lo contiene.

Risultati. La baseline raggiunge il 99,8% di success rate in 47,6 step medi — valori misurati sui 624 spawn appaiati di questo protocollo, e quindi lievemente diversi dal 99,7% / 49,8 step riportati per lo stesso modello nella Tabella 5.8 (valutazione held-out a 1560 episodi). Il risultato dominante è la **storia della corona a 20 m**: rimuoverla costa $-26,8$ punti di success rate (IC $[-30,3, -23,4]$) e $+77,7\%$ di step. È di gran lunga il blocco più portante dell'intera osservazione, e il fatto che a pesare sia la *storia* — non la lettura istantanea, che costa $-6,2$ punti — dice che la politica integra il campo chimico nel tempo anziché reagire all'istante. Seguono la memoria degli **spostamenti** ($-12,8$ punti), cioè il senso di dove si sta andando, e il **vettore verso la massima concentrazione vista** ($-5,0$).

All'estremo opposto, tre gruppi risultano **indistinguibili dalla baseline** su entrambe le metriche: la concentrazione istantanea, la sua storia e la corona a 50 m istantanea hanno intervalli di confidenza che contengono lo zero. Il caso della **concentrazione istantanea** è il più netto: azzerarla non cambia nulla ($-0,7\%$ di step, IC $[-5,5, +4,5]$;

success rate invariato). La lettura al centro della formazione è ridondante — gli otto sensori direzionali la circondano a 20 m e ne contengono già l'informazione — e la politica, di fatto, la ignora.

La seconda corona. Il verdetto sulla corona a 50 m è sfumato e merita attenzione, perché incrocia il risultato della Sezione 5.5.4. La sua lettura istantanea non ha effetto misurabile (+5,3% di step, IC $[-0,4, +12,4]$), mentre la sua *storia* ha un effetto piccolo ma reale sull'efficienza (+13,9%, IC $[+7,0, +21,9]$) e nullo sulla riuscita ($-0,3$ punti). Il quadro è coerente con il confronto per riaddestramento del Capitolo 5, dove la doppia corona a $v_{\max}=2$ risultava 28,3% più veloce a parità di success rate: i due esperimenti — rimuovere il sensore da un modello *congelato* e riaddestrare senza di esso — convergono sullo stesso ordine di grandezza e sulla stessa conclusione qualitativa, cioè che la seconda corona compra **velocità, non affidabilità**. La convergenza di due protocolli indipendenti è un riscontro non banale.

6.5 Valori di Shapley esatti

Con dodici giocatori le $2^{12} = 4096$ coalizioni sono state enumerate integralmente su un buffer di 1500 stati raccolti dai rollout della politica piena. Il payoff è $v(S) = \pi(a^* | x_S)$, con a^* l'azione scelta a osservazione completa. La media di $\pi(a^*)$ passa da 0,201 alla baseline a 0,707 a osservazione piena: i dodici gruppi si ripartiscono quindi 0,506 di massa di probabilità. L'assioma di efficienza è verificato numericamente con un errore massimo di $3,2 \times 10^{-8}$, che conferma l'esattezza del calcolo.

La Figura 6.2 riporta, per ciascuno dei dodici gruppi, la magnitudine media del valore di Shapley $|\phi_i|$: la lunghezza di ogni barra misura quanto quel blocco contribuisce, in media, alla probabilità che la politica assegna all'azione scelta. È quindi una graduatoria di *ciò che la rete guarda*, ossia una lettura *osservazionale* della funzione appresa, distinta dalla *necessità* causale misurata dall'ablazione (Sezione 6.4); come si vedrà, le due letture divergono, e il confronto è ripreso nella Sezione 6.8.

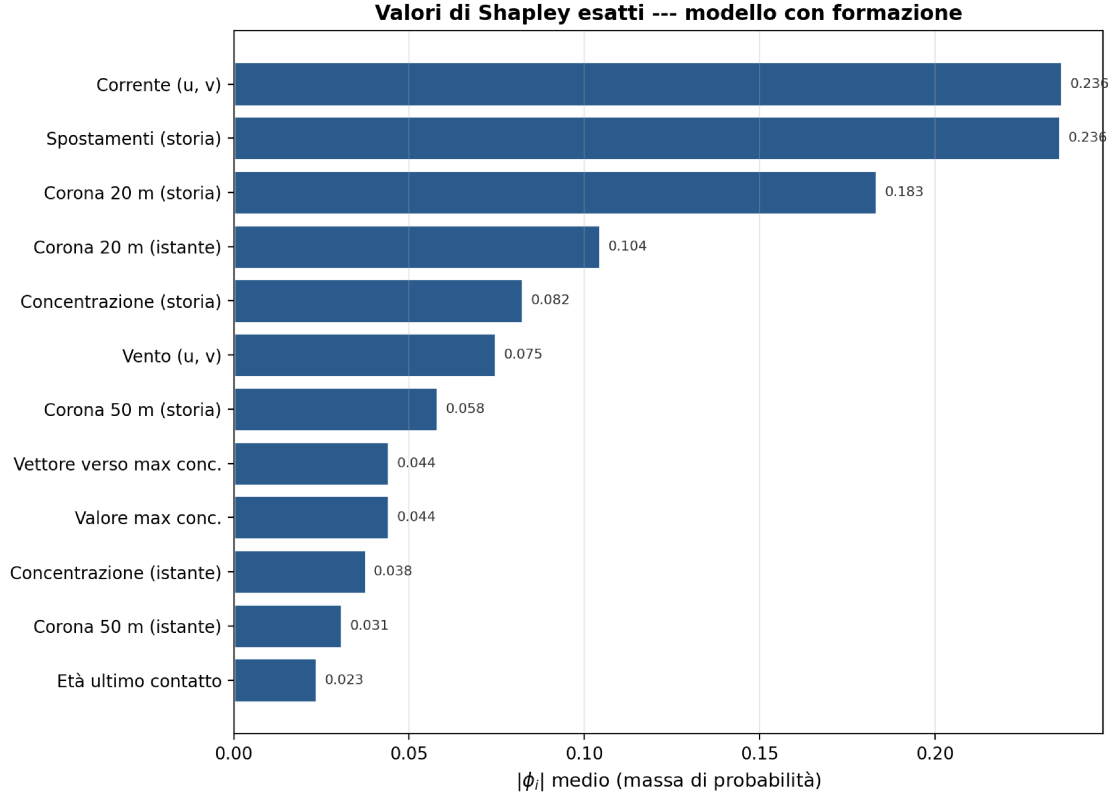


Figura 6.2: Valori di Shapley esatti sui dodici gruppi: media di $|\phi_i|$ su 1500 stati, 4096 coalizioni enumerate. La magnitudine è sull'asse, non nel colore.

La graduatoria di ciò che la rete *guarda* è guidata a pari merito dalla **corrente** ($|\phi| = 0,236$) e dalla **memoria degli spostamenti** (0,236), seguite dalla storia della corona a 20 m (0,183). In coda stanno l'età dell'ultimo contatto (0,023), la corona a 50 m istantanea (0,031) e la concentrazione istantanea (0,038).

6.6 Analisi comportamentale

Non è risalita di gradiente. La domanda più naturale su una politica di source seeking è se si riduca a seguire il sensore più intenso. Per rispondere si confronta, su 13 738 stati con contatto col plume, la direzione del sensore massimo della corona a 20 m con la direzione scelta. La lettura richiede un'accortezza: gli spawn stanno sul plume, che è avvezzato *a valle* della sorgente, quindi la sorgente cade quasi sempre in una direzione sistematica e il marginale $P(\text{azione})$ è fortemente sbilanciato. Confrontare la diagonale con $1/8$ sarebbe quindi fuorviante: va confrontata con il valore *atteso sotto indipendenza*, che incorpora quel bias.

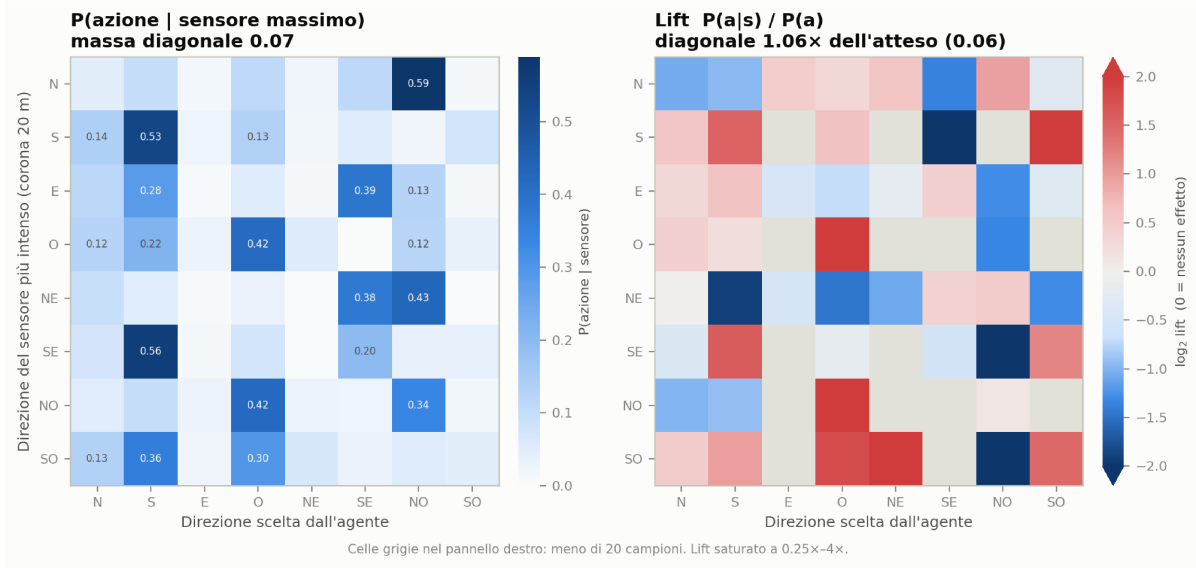


Figura 6.3: Sensore massimo (corona 20 m) contro direzione scelta. A sinistra la condizionale $P(a|s)$, dominata dal bias direzionale degli scenari; a destra il *lift* $P(a|s)/P(a)$, che divide per il marginale e isola il solo contributo del sensore (1 = nessun effetto).

La massa sulla diagonale è 0,068 contro un atteso di 0,064: un **lift di appena 1,06x**. La politica muove verso il sensore più intenso solo il 6% delle volte in più di quanto predica il suo stesso bias direzionale. **La politica non fa risalita di gradiente locale:** pur essendo la corona a 20 m il blocco più portante dell'osservazione (Sezione 6.4), non la usa per scegliere la direzione istantanea di massima concentrazione. I due risultati non sono in contraddizione, anzi si spiegano a vicenda: ciò che conta non è la corona ma la *sua storia*, cioè l'integrazione temporale del campo, esattamente la strategia che il Capitolo 5 attribuiva all'RL per spiegare il divario con l'FCM — il quale, per costruzione, la risalita di gradiente locale la fa.

Il vento è letto ma non seguito. Ruotando il vento a 360° e tenendo fermo il resto dell'osservazione, la direzione preferita dalla politica insegue il controvento con un allineamento di appena 0,109 (dove 1 sarebbe anemotassi perfetta e 0 indifferenza). La politica, cioè, **non fa anemotassi**. Il dato è coerente con l'addestramento: questo modello discende dalla catena *minimal*, in cui i reward di allineamento a vento e corrente sono *disattivati* (Sezione 5.5.2) proprio perché controproducenti. Il risultato vale quindi anche come validazione del metodo: l'analisi ritrova, senza saperlo, una proprietà nota del setup di addestramento.

Va segnalato il limite della sonda: ruotare il vento tenendo fermo il campo di concentrazione produce osservazioni fuori dalla varietà dei dati, perché vento e forma del plume sono fisicamente accoppiati. La response curve risponde quindi alla domanda “l’uscita della politica dipende dall’ingresso vento?”, non a “cosa accadrebbe in un mondo con vento diverso?”.

Il critic stima la distanza. La funzione valore $V(s)$ correla negativamente con la distanza dalla sorgente ($r = -0,33$ su tutti gli scenari): il critic ha appreso una nozione di prossimità senza che gliela si sia mai fornita esplicitamente. La correlazione è moderata perché aggregata su 26 sorgenti e 12 combinazioni vento–chunk, ciascuna con il proprio livello di difficoltà: $V(s)$ stima il ritorno atteso, che dipende anche dallo scenario, non solo dalla distanza.

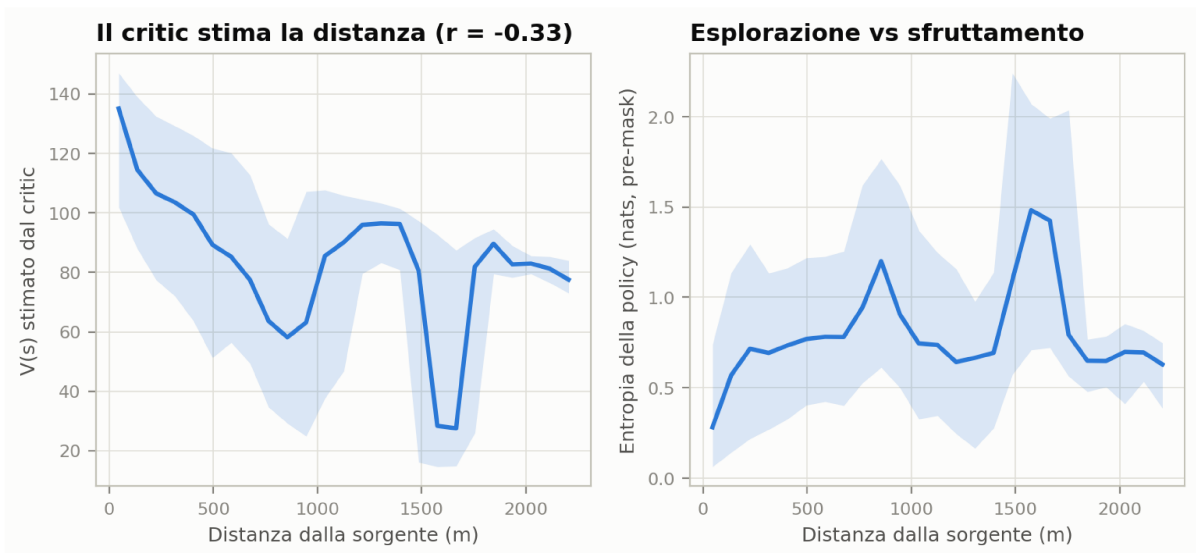


Figura 6.4: A sinistra $V(s)$ stimato dal critic in funzione della distanza dalla sorgente (mediana e intervallo interquartile per bin); a destra l’entropia della politica, calcolata sui logit pre-maschera.

6.7 Il valore della formazione: ablazione per riaddestramento

Le ablazioni fin qui condotte operano su un modello **congelato**: si azzerava un gruppo di feature in inferenza e si misura il danno. Questo protocollo, però, *sovrastima* l’importanza di una feature, perché l’agente non può adattarsi alla sua assenza. Per misurare

il valore *irriducibile* della formazione — l'insieme dei sensori direzionali che distingue la nostra squadra master-slave da un singolo veicolo — si adotta un protocollo più equo: **rimuovere la formazione e riaddestrare**.

Protocollo. A partire dal modello a singola corona con $v_{\max} = 2 \text{ m/s}$ si esegue un fine-tuning (2M step) in cui i sensori direzionali della formazione — correnti e storici — sono **azzerati** ad ogni passo (*masking* della formazione). L'architettura non cambia (osservazione a 116 dimensioni, spazio d'azione Discrete(8×5)): l'agente riparte dai pesi del modello con formazione e impara una nuova strategia con la sola informazione che gli resta — la concentrazione nella *propria* posizione, le memorie di concentrazione e di spostamento, il vento e la corrente. Tutto il resto (velocità, protocollo held-out a 1560 episodi, dataset) è identico, così che il calo di prestazioni sia attribuibile alla *sola* rimozione della formazione.

Costo della rimozione. La Tabella 6.2 confronta l'agente singolo con il modello a formazione di pari velocità.

Metrica	Con formazione	Senza formazione	Δ
SR globale	99,7%	93,3%	−6,4
Step medi (succ.)	69,5	100,6	+44,7%
V0	100,0%	97,2%	−2,8
V1	99,5%	97,9%	−1,6
V2	99,2%	86,4%	−12,8
V3	100,0%	91,8%	−8,2
Q1/4	100,0%	97,9%	−2,1
Q1/2	99,0%	90,0%	−9,0
Q3/4	100,0%	92,1%	−7,9

Tabella 6.2: Modello con vs. senza formazione a $v_{\max}=2 \text{ m/s}$ (1560 episodi ciascuno). Il Δ è il costo *netto* della rimozione della formazione, dopo che l'agente ha avuto modo di adattarsi.

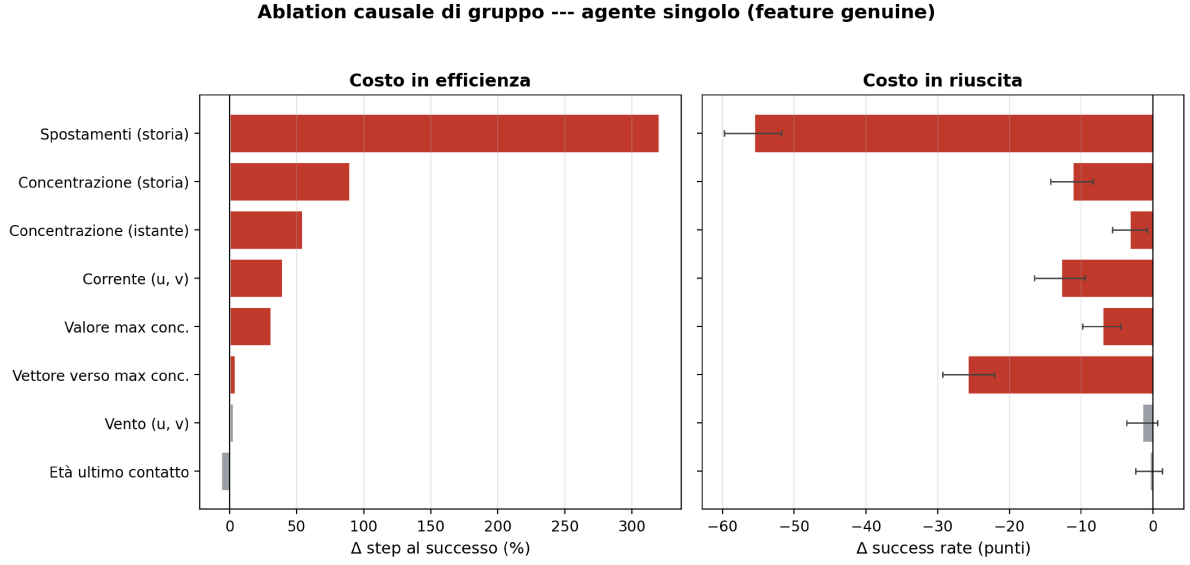
La formazione vale 6,4 **punti** di success rate globale e circa il 45% in velocità. Il costo non è uniforme: è trascurabile negli scenari facili (V0/V1, Q1/4, entro -3 punti) e diventa marcato dove il plume è avvettato e diffuso — **V2** ($-12,8$) e **Q1/2** ($-9,0$) — ossia dove il gradiente *spaziale* istantaneo delle otto letture simultanee è più prezioso. Il calo resta comunque **moderato**: l'agente singolo riaddestrato recupera gran parte delle prestazioni, e soprattutto rimane molto sopra il metodo del gradiente — l'FCM adattivo a pari velocità ($lr = 20\text{m}$) si ferma al 58,2%, circa **35 punti** sotto l'agente singolo (93,3%). È un fatto notevole: un agente RL che vede solo la propria concentrazione batte nettamente un metodo reattivo che dispone dell'intera formazione. Ciò che separa il successo dal fallimento non è il sensore, ma la strategia appresa.

Come compensa l'agente singolo. L'explainability del modello senza formazione (stesso protocollo: ablazione di gruppo e Shapley esatti) rivela uno **spostamento del baricentro informativo dal sensing spaziale a quello temporale** (Tabella 6.3, Figura 6.5). Privato della corona, l'agente si regge in modo schiacciante sulla **memoria degli spostamenti** (ablazione $-55,6$ punti, Shapley $|\phi| = 0,27$): correla i propri movimenti con le variazioni della concentrazione che misura sotto di sé, ricostruendo nel *tempo* il gradiente che la formazione forniva nello *spazio*. Seguono l'*homing* verso la massima concentrazione vista ($-25,8$), la corrente ($-12,8$) e la storia della concentrazione propria ($-11,2$) — proprio la feature che il modello *con* formazione ignorava (≈ 0).

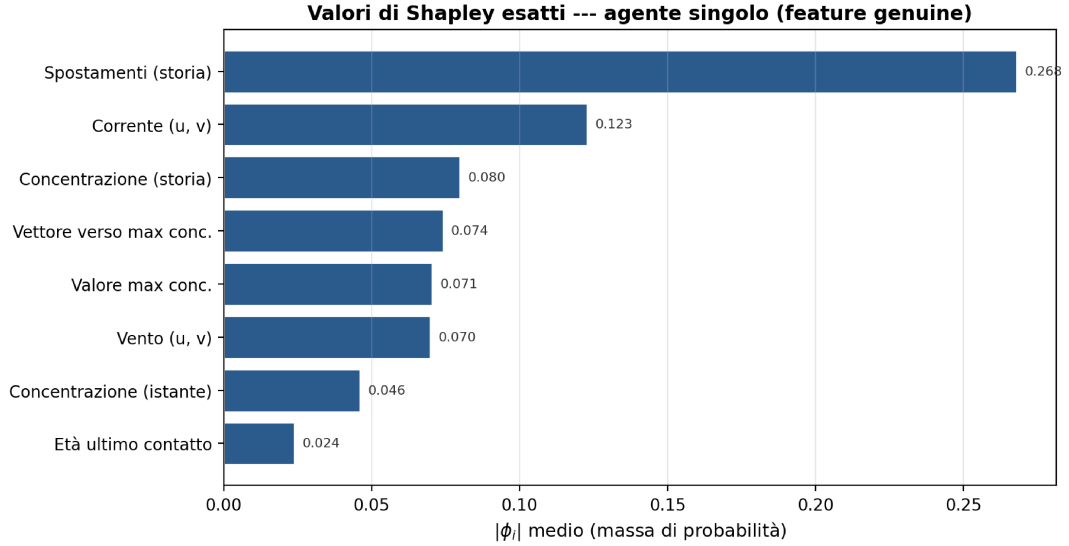
Come per il modello con formazione (Sezione 6.8), anche qui *attribuzione* e *necessità* divergono, e nello stesso modo istruttivo. Il **vettore verso il massimo di concentrazione** è solo quarto per attribuzione ($|\phi| = 0,07$) ma **secondo per impatto causale** ($-25,8$): l'*homing* di memoria è insostituibile per il compito pur essendo poco “guardato” dalla rete. Specularmente, la **corrente** è seconda per attribuzione ($|\phi| = 0,12$) ma pesa meno all'ablazione ($-12,8$), confermando — ora anche in assenza della corona — che ciò che la rete osserva non coincide con ciò che le è strettamente necessario.

Gruppo (Δ SR)	Con formazione	Senza formazione
Corona 20 m (storia)	−26,8	mascherata
Spostamenti (storia)	−12,8	−55,6
Corona 20 m (istante)	−6,2	mascherata
Vettore verso max conc. (homing)	−5,0	−25,8
Corrente (u, v)	−3,0	−12,8
Vento (u, v)	−2,2	−1,4
Concentrazione (storia)	−0,5	−11,2
Corona 50 m (storia)	−0,3	—
Concentrazione (istante)	0,0	−3,2
Corona 50 m (istante)	0,0	—
Valore max conc.	+0,2	−7,1
Età ultimo contatto	+0,2	−0,5

Tabella 6.3: Spostamento del baricentro informativo: costo in success rate (Δ SR, punti percentuali) di *ogni* gruppo, per il modello *con* formazione (doppia corona, ablazioni della Sezione 6.4) e per l’agente *senza* formazione (singola corona, formazione mascherata). Con la formazione la politica si regge sul gradiente *spaziale* (storia della corona); rimossa, il peso si trasferisce sul sensing *temporale* — spostamenti, homing, storia e valore di concentrazione, tutti nettamente più costosi. “mascherata”: corona azzerata, effetto spurio escluso (Figura 6.5); “—”: gruppo assente, poiché l’agente singolo ha una sola corona.



(a) Ablazione di gruppo.



(b) Valori di Shapley esatti.

Figura 6.5: Explainability dell’agente singolo (senza formazione), sui soli gruppi *genuini*: la memoria degli spostamenti è la feature portante, seguita dall’homing verso il massimo di concentrazione e dalla corrente. I gruppi della corona sono **esclusi** perché mascherati a zero: **VecNormalize** li rende una costante non nulla il cui azzeramento perturberebbe il bias appreso, producendo un effetto spurio privo di contenuto informativo.

Il quadro chiude il cerchio dell’analisi: la rete non “perde un pezzo” quando le si toglie la formazione, ma **ricompone** la stima della direzione con gli strumenti che le restano. La formazione è un vantaggio reale ma *sostituibile in parte* da una strategia temporale appresa; è negli scenari a plume diffuso, dove il segnale temporale degenera più in fretta, che il suo contributo spaziale resta insostituibile.

6.8 Sintesi: che cosa la rete guarda, che cosa le serve

Il confronto fra le due graduatorie è la parte più istruttiva del capitolo, perché **divergono**.

Gruppo	Shapley $ \phi $	ΔSR	Lettura
Corona 20 m (storia)	0,183 (3°)	−26,8	indispensabile
Spostamenti (storia)	0,236 (1°)	−12,8	indispensabile
Corona 20 m (istante)	0,104 (4°)	−6,2	necessaria
Vettore verso max conc. (homing)	0,044 (8°)	−5,0	necessaria, poco guardata
Corrente (u, v)	0,236 (1°)	−3,0	guardata, poco necessaria
Vento (u, v)	0,075 (6°)	−2,2	guardata, poco necessaria
Concentrazione (storia)	0,082 (5°)	−0,5	guardata, poco necessaria
Corona 50 m (storia)	0,058 (7°)	−0,3	ridondante
Concentrazione (istante)	0,038 (10°)	0,0	ridondante
Corona 50 m (istante)	0,031 (11°)	0,0	ridondante
Valore max conc.	0,044 (8°)	+0,2	ridondante
Età ultimo contatto	0,023 (12°)	+0,2	ridondante

Tabella 6.4: Attribuzione (Shapley $|\phi|$: *che cosa la rete guarda*) contro necessità causale (ablation ΔSR in punti percentuali: *che cosa serve al compito*), per tutti i dodici gruppi, ordinati per necessità decrescente. Le due graduatorie **divergono**: la *corrente* è la più guardata (Shapley 1°) ma poco necessaria (−3,0), mentre la *storia della corona* è solo terza per attribuzione ma di gran lunga la più necessaria (−26,8); specularmente l'*homing* è poco guardato (8°) ma utile (−5,0).

Il caso più istruttivo è la **corrente marina**: è il gruppo con l'attribuzione più alta in assoluto ($|\phi| = 0,236$, primo a pari merito), eppure rimuoverla costa appena 3 punti

di success rate. La rete la legge molto, ma il compito ne ha poco bisogno — segno che l'informazione che la corrente veicola è in larga parte *recuperabile* dagli altri blocchi, in primo luogo dalla storia della corona, che del trasporto porta già la traccia. Simmetricamente, la **storia della corona a 20 m** è solo terza per attribuzione ma prima, e di gran lunga, per impatto causale: è l'unico blocco davvero non sostituibile.

È precisamente la divergenza che giustifica l'impianto metodologico della Sezione 6.2. Un capitolo fondato sui soli metodi post-hoc avrebbe concluso che la corrente è la grandezza più importante per la politica; l'intervento mostra che, se la si toglie, la politica se la cava quasi altrettanto bene. Le due domande — che cosa la rete guarda, che cosa serve al compito — hanno risposte diverse, e solo la seconda è quella che conta per decidere quali sensori montare a bordo.

Ne emerge un ritratto coerente della politica appresa. Essa **non** è un risalitore di gradiente: la direzione istantanea di massima concentrazione non guida la scelta (lift $1,06\times$), e la concentrazione al centro della formazione è addirittura ignorata. Ciò su cui si regge è l'**integrazione temporale** del campo chimico locale — la storia della corona a 20 m, il blocco più portante, unita alla memoria di dove si è andati — affiancata da una memoria esplicita del punto migliore incontrato. La seconda corona a 50 m serve a puntare meglio le falcate lunghe, comprando velocità e non affidabilità, in accordo con quanto il Capitolo 5 aveva stabilito per riaddestramento. Vento e corrente vengono letti ma non usati come bussola. È, in sintesi, esattamente la strategia che il Capitolo 5 ipotizzava per spiegare il divario con il metodo del gradiente — con la differenza che qui è misurata anziché supposta.

Capitolo 7

Conclusioni

Questa tesi ha affrontato il problema del *source seeking* marino — la localizzazione autonoma della sorgente di un inquinante a partire dalle letture locali di concentrazione — confrontando due famiglie di approcci nell’ambito del progetto HYDRAS, su un dataset di simulazioni idrodinamiche realistiche della baia di Cecina.

7.1 Risultati principali

Ripercorriamo il cammino della tesi dall’inizio alla fine, raccogliendone i risultati salienti.

Il problema e i dati. Il punto di partenza è un problema concreto: localizzare in modo autonomo la sorgente di uno sversamento inquinante dalle sole letture locali di concentrazione, in un ambiente costiero dove la dispersione è governata da una dinamica complessa di correnti, vento e turbolenza. Nell’ambito del progetto HYDRAS, il compito è stato affrontato su un dataset di simulazioni idrodinamiche ad alta risoluzione (MIKE21) della baia di Cecina (Capitolo 2), forzate dai dati meteo-oceanografici di CMEMS ed ERA5: 132 sorgenti equispaziate, ciascuna simulata sotto quattro scenari di vento (V0–V3) che generano plume dalle morfologie marcatamente diverse. Per una valutazione rigorosa, 26 sorgenti (SRC107–132) sono state riservate alla sola inferenza — mai viste in addestramento — e ogni simulazione è stata suddivisa in tre finestre temporali, definendo il protocollo *held-out* su cui entrambi gli approcci sono stati misurati a parità di condizioni.

L'ambiente condiviso. Su questi dati è stato costruito un unico ambiente di simulazione (Capitolo 3), condiviso dai due approcci: una squadra *master-slave* in cui un agente master decide il movimento e otto slave, disposti in corona lungo le direzioni cardinali e diagonali, campionano la concentrazione e gliela riportano. L'ambiente definisce lo spazio d'azione discreto, l'*action masking* che impedisce le collisioni con la costa, la procedura di inizializzazione sul plume e un segnale di reward. Far operare i due metodi sullo stesso ambiente — stessa dinamica, stessi vincoli, stesse osservazioni — è ciò che rende il confronto pulito: l'unica variabile è il modulo che, ricevuta l'osservazione, sceglie l'azione.

Il metodo del gradiente. Il primo modulo è il **Field Climbing Method** (Capitolo 4), la baseline gradient-based. Nella versione a passo fisso raggiunge un success rate globale di appena il **32,9%**, limitato dalla sua natura puramente reattiva. L'introduzione dell'ottimizzatore **Adam** per il calcolo del passo — con momentum sul gradiente stimato e normalizzazione per la varianza storica — lo porta fino al **63,5%** ($lr = 40$ m), con un guadagno di oltre trenta punti: il momentum smorza le oscillazioni dovute al rumore del sensore. Il passo ottimale non è però il più ampio — oltre i 40 m falcate troppo lunghe scavalcano la struttura del plume — e i valori più performanti implicano velocità superiori a quella nominale del veicolo. Al di là di questi affinamenti, il limite resta *strutturale*: negli scenari a plume diffuso il gradiente locale degenera e il metodo, privo di memoria e di contesto, si blocca.

L'apprendimento per rinforzo. Il secondo modulo è un agente addestrato con **PPO** (Capitolo 5). Uno studio di ablation sulla funzione di reward ha mostrato che la variante più semplice (**minimal**), basata sul solo gradiente chimico come segnale direzionale, ottiene il success rate più alto (**96,9%**): i reward di allineamento a vento e corrente si sono rivelati addirittura controproducenti, perché fondati sull'ipotesi — non valida in ambiente costiero — di una sorgente sempre controvento. Rendere poi la **velocità una scelta dell'agente** (spazio Discrete(8×5), addestrato a catena alzando progressivamente $v_{\max}$) ha portato il success rate al 99,7–99,9% e ridotto di oltre quattro volte il tempo al successo (da ~ 20 a ~ 5 minuti simulati). Su questa catena, la **doppia corona** di sensori si è rivelata un vantaggio netto a partire da $v_{\max} = 2$ m/s — stessa affidabilità

e 25–34% di passi in meno — correggendo la lettura di un primo confronto in cui, addestrata da zero, sembrava dannosa. Approfondimenti mirati hanno infine mostrato che l’agente sa *manovrare* finemente attorno alla sorgente (test a raggio di successo ridotto) e che l’esito di un episodio **non dipende dalla posizione di partenza** (analisi delle *spawn map*), ma solo dalle condizioni idrodinamiche dello scenario.

Il confronto. Il confronto diretto è netto: il modello RL supera il metodo del gradiente su ogni scenario. Poiché entrambi dispongono di una versione a velocità variabile, il confronto più equo è quello **a parità di velocità massima**, che neutralizza il vantaggio cinematico di un passo più lungo: a ogni livello di velocità il PPO domina l’FCM di 36–42 **punti** di success rate, con il divario massimo proprio negli scenari a plume diffuso, dove il gradiente locale degenera. Esteso all’intero range operativo (da 0,1 a 5 m/s), il quadro non cambia: il PPO supera il 98% già a 1,2 m/s e si stabilizza attorno al 99%, mentre l’FCM resta confinato attorno al 55–63% alle velocità operative; l’unico regime in cui il divario si assottiglia è quello delle velocità più basse, dove un deficit puramente fisico di propulsione — il veicolo più lento delle correnti da risalire — penalizza ogni strategia.

L’interpretazione della politica. Stabilito *che* la politica funziona, il Capitolo 6 ne ha indagato il *perché*. Tramite ablation causale di gruppo, valori di Shapley esatti e sonde comportamentali è emerso un ritratto coerente: la politica **non** è un risalitore di gradiente locale (muove verso il sensore più intenso solo il 6% delle volte in più di quanto predica il suo stesso bias direzionale), ma si regge sull’**integrazione temporale** del campo chimico — la storia della corona di sensori — unita alla memoria di dove si è andati. È emersa inoltre una divergenza istruttiva fra *ciò che la rete guarda* e *ciò che le serve*: la corrente marina è la grandezza più “guardata” eppure la sua rimozione costa pochissimo, mentre la storia della corona è il solo blocco davvero insostituibile. Un esperimento di riaddestramento *senza* formazione ne ha infine quantificato il valore: la formazione vale circa 6,4 **punti** di success rate, ma è in parte sostituibile da una strategia temporale appresa — l’agente singolo riaddestrato resta al 93,3%, circa trentacinque punti sopra l’FCM che pure dispone dell’intera formazione. Ciò che separa il successo dal fallimento, in definitiva, non è il sensore ma la **strategia appresa**.

7.2 Sviluppi futuri

La direzione di sviluppo più naturale e ambiziosa è il passaggio da un sistema a *singolo decisore* a un **sistema multi-agente cooperativo “vero”**. Nell’architettura master–slave adottata in questa tesi un solo agente (il master) decide il movimento, mentre gli otto slave sono sensori passivi in formazione rigida: di fatto il problema resta a singolo agente. La visione complessiva del progetto HYDRAS prevede invece una squadra di n robot autonomi, ciascuno dotato di una *propria* politica di navigazione, che collaborano — coordinando l’esplorazione, ripartendosi il dominio e scambiandosi informazioni — per localizzare la sorgente più rapidamente e con maggiore robustezza di un sistema centralizzato.

L’algoritmo MAPPO. Per addestrare una tale squadra, l’estensione naturale del PPO impiegato in questa tesi è **MAPPO** (*Multi-Agent PPO*) [10], che adatta il PPO al contesto multi-agente cooperativo secondo il paradigma **CTDE** (*Centralized Training, Decentralized Execution*). L’idea di fondo è separare ciò che serve in addestramento da ciò che serve in esecuzione:

- ogni agente possiede una **politica decentralizzata** (attore) $\pi_\theta(a_i \mid o_i)$ che sceglie l’azione a partire dalla *sola* osservazione locale o_i (le proprie letture di concentrazione, la propria cinematica). In esecuzione, quindi, ogni robot è autonomo: naviga senza un coordinatore centrale e senza comunicazione a banda piena;
- in addestramento, un **critico centralizzato** (*value function*) $V_\phi(s)$ riceve informazione *globale* sullo stato congiunto della squadra — non disponibile ai singoli attori — e ne stima il valore atteso. È questo critico “onnisciente” a stabilizzare l’apprendimento: valuta le azioni di ciascun agente tenendo conto di ciò che fanno gli altri, mitigando la *non-stazionarietà* che affligge gli approcci in cui ogni agente apprende in modo del tutto indipendente (IPPO, *Independent PPO*).

Il critico viene usato *solo* durante il training e scartato in esecuzione, dove sopravvivono le sole politiche locali — da cui il nome CTDE. La scelta di cosa fornire in ingresso al critico è un punto delicato: gli autori mostrano che uno stato *agent-specific* (lo stato globale dell’ambiente concatenato all’osservazione locale dell’agente, con le feature ridondanti

rimosse) rende meglio sia della semplice concatenazione di tutte le osservazioni sia del solo stato globale dell'ambiente.

Sul piano meccanico MAPPO conserva l'impianto del PPO di questa tesi — obiettivo *clipped*, stima del vantaggio con GAE, normalizzazione del vantaggio — applicato per-agente, con due accorgimenti che si rivelano decisivi. Il primo è la **condivisione dei parametri** tra agenti omogenei: attori e critici usano gli stessi pesi per tutti i robot, il che moltiplica i dati utili e migliora l'efficienza campionaria quando i veicoli sono identici. Il secondo è la **normalizzazione del value target** tramite media e deviazione standard correnti, che evita l'instabilità quando i ritorni variano molto tra episodi. Gli autori raccomandano inoltre poche epoche PPO per aggiornamento (5–15, meno nei task difficili) e al più due mini-batch, per contenere la deriva delle politiche fra un update e l'altro. Il risultato principale del loro lavoro è che, **all'aumentare del numero di agenti**, il vantaggio del critico centralizzato di MAPPO cresce nettamente rispetto agli approcci indipendenti: è proprio nei regimi a molti agenti — quelli di interesse per una squadra numerosa di robot marini — che l'algoritmo dà il meglio.

Adattamento al source seeking. Calato nel nostro dominio, ciascun agente sarebbe un veicolo autonomo con la propria corona di sensori; il critico centralizzato, in addestramento, potrebbe accedere alle posizioni e alle letture dell'*intera* squadra (e, in simulazione, all'intero campo MIKE21), mentre in mare ogni veicolo naviga sulla sola informazione locale e su una comunicazione sparsa con i vicini. Rispetto al master-slave qui adottato, ciò trasformerebbe gli slave da sensori passivi in decisori attivi, aprendo a strategie di copertura e di ripartizione del dominio precluse a un singolo agente — esattamente l'architettura *multi-agente distribuita* che costituisce l'obiettivo di lungo periodo del progetto HYDRAS.

Restano infine aperte altre direzioni complementari:

- **Validazione su veicolo reale:** trasferire la politica appresa in simulazione a un AUV fisico, affrontando il *sim-to-real gap* dovuto al rumore reale dei sensori e alla dinamica del veicolo;
- **Generalizzazione geografica:** addestrare e validare gli agenti su domini marini diversi dalla baia di Cecina, per valutare la trasferibilità delle strategie apprese;

- **Architetture ricorrenti:** sostituire la policy MLP con reti dotate di memoria esplicita (LSTM, Transformer) per modellare in modo più ricco la dinamica temporale del plume.

I risultati ottenuti confermano il potenziale del Reinforcement Learning come strumento per la navigazione autonoma in ambienti marini complessi, e quantificano il contributo dell'apprendimento rispetto agli approcci gradient-based classici.

Bibliografia

- [1] Jennie Cochran and Miroslav Krstić. Nonholonomic source seeking with tuning of angular velocity. *IEEE Transactions on Automatic Control*, 54(4):717–731, 2009.
- [2] Jay A. Farrell, Shuo Pang, and Wei Li. Chemical plume tracing via an autonomous underwater vehicle. *IEEE Journal of Oceanic Engineering*, 30(2):428–442, 2005.
- [3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, 2018.
- [4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [5] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, 2018.
- [6] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*, 2015.
- [7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [8] Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. In *The International FLAIRS Conference Proceedings*, volume 35, 2022.

-
- [9] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
 - [10] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 35, 2022.
 - [11] DHI. MIKE 21 flow model: Hydrodynamic module — user guide. *DHI Water and Environment*, 2017.
 - [12] Copernicus Marine Service. Copernicus marine environment monitoring service (cmems): Mediterranean sea physics analysis and forecast. *E.U. Copernicus Marine Service Information*, 2022.
 - [13] Paolo Oddo and Nadia Pinardi. Lateral open boundary conditions for nested limited area models: A scale selective approach. *Ocean Modelling*, 20(2):134–156, 2008.
 - [14] Hans Hersbach, Bill Bell, Paul Berrisford, et al. The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730):1999–2049, 2020.
 - [15] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.
 - [16] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
 - [17] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. “why should I trust you?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.
 - [18] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

-
- [19] Lloyd S. Shapley. A value for n -person games. In *Contributions to the Theory of Games II*, volume 28 of *Annals of Mathematics Studies*, pages 307–317. Princeton University Press, 1953.
 - [20] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International Conference on Machine Learning (ICML)*, 2017.
 - [21] Kjersti Aas, Martin Jullum, and Anders Løland. Explaining individual predictions when features are dependent: More accurate approximations to Shapley values. *Artificial Intelligence*, 298:103502, 2021.
 - [22] Martin Jullum, Annabelle Redelmeier, and Kjersti Aas. groupShapley: Efficient prediction explanation with Shapley values for feature groups. *arXiv preprint arXiv:2106.12228*, 2021.
 - [23] Aaron Fisher, Cynthia Rudin, and Francesca Dominici. All models are wrong, but many are useful: Learning a variable’s importance by studying an entire class of prediction models simultaneously. *Journal of Machine Learning Research*, 20(177): 1–81, 2019.
 - [24] Stephanie Milani, Nicholay Topin, Manuela Veloso, and Fei Fang. Explainable reinforcement learning: A survey and comparative review. *ACM Computing Surveys*, 56(7), 2024.
 - [25] David Alvarez-Melis and Tommi S. Jaakkola. On the robustness of interpretability methods. *arXiv preprint arXiv:1806.08049*, 2018.